

JoBot Merge Plan.

A comprehensive architecture and setup guide for merging the best of **33 open-source job-search AI repositories** — including **santifer/career-ops (63k★)**, **MadsLorentzen/ai-job-search (31k★)**, and **feder-cr/AIHawk (30k★)** — into JoBot's existing agentic chassis. Covers target architecture, 12 capability modules, 6-provider LLM abstraction, phased migration roadmap, and full local + Docker setup.

TARGET REPOSITORY	LICENSE
aryansinghnagar/JoBot	AGPL-3.0
MAINTAINER	PLAN VERSION
Aryan Singh Nagar	Draft 1.0 · 2026-08-13

MULTI-LLM	AUTO-APPLY	ATS	DOCKER	PLUGIN REGISTRY
SAGA APPLY	LATEX RESUME			

Table of Contents

This document is organized in five parts. Part I audits JoBot's current state and the source repository landscape. Part II defines the target architecture. Part III specifies each of the 12 capability modules to be merged. Part IV is the phased migration roadmap. Part V is the setup documentation for local and Docker deployment.

Executive Summary

Multi-LLM Provider Support

Confidence Levels

What This Document Is Not

Reading Order

Chapter 1 — Scope, Assumptions & Methodology

1.1 Scope

1.2 Assumptions

1.3 Research Methodology

1.4 Tiering Heuristic

1.5 What This Plan Is

Chapter 2 — JoBot Current State Audit

2.1 What JoBot Has Today

2.2 What JoBot Lacks

2.3 Verdict

2.4 Repository Hygiene

Chapter 3 — Source Repository Inventory (Star-Ranked Matrix)

3.1 Tier 1 — 10,000+ stars or 5,000+ with strong maintenance

3.2 Tier 2 — 1,000–10,000 stars

3.3 Tier 3 — 100–1,000 stars

3.4 Tier 4 — <100 stars, archived, or too niche

3.5 Feature Overlap Matrix (Top 10 by Stars)

Chapter 4 — Tiered Adoption Strategy

4.1 Tier 1 — Absorb First

4.1.1 santifer/career-ops (63,659★)

4.1.2 MadsLorentzen/ai-job-search (31,411★)	17
4.1.3 feder-cr/Jobs_Applier_AI_Agent_AIHawk (30,159★)	18
4.2 Tier 2 — Absorb Next (Specific Capability Gaps)	18
4.2.1 speedyapply/JobSpy (4,072★) — VENDOR AS PIP DEP	18
4.2.2 DaKheera47/job-ops (3,840★)	19
4.2.3 GodsScion/Auto_job_applier_linkedin (2,688★)	19
4.2.4 can4hou6joeng4/boss-agent-cli (1,558★)	19
4.2.5 eatmoreduck/boss-zhipin-scraper (1,094★)	19
4.3 Tier 3 — Selectively Absorb (Single-Capability References)	20
4.4 Tier 4 — Skip (and Why)	20
4.5 License Compliance Decision Tree	20
4.6 Recommended Adoption Sequence	21

Chapter 5 — Target Architecture Overview 23

5.1 Layer Map	23
5.2 Design Patterns Chosen	23
5.3 Data Flow — End-to-End Apply Saga	24
5.4 Anti-Patterns Explicitly Forbidden	25

Chapter 6 — AI Provider Abstraction Layer (ModelRouter v2) 26

6.1 Design Goals	26
6.2 Provider Class Matrix	27
6.3 Interface Contract	27
6.4 Concrete Implementations	29
6.5 The Router — Provider Selection Logic	31
6.6 Configuration Schema	32

Chapter 7 — Browser Automation Layer 34

7.1 Patchright as Primary Stealth Browser	34
7.2 CDP Session Reuse as Alternative Stealth	34
7.3 BrowserPool — Session Lifecycle	34
7.4 Named Actions vs One-Off DOM Scripts	35
7.5 Captcha Solver Hook	36
7.6 Selector Healing + Action Caching	37
7.7 Preview-Before-Commit	37

Chapter 8 — Storage & State Plane	38
8.1 Tier 1 — SQLite WAL Control Plane	38
8.2 Tier 2 — Fernet-Encrypted Blob Store	39
8.3 Tier 3 — OS Keyring	40
8.4 Migration System	41
8.5 Vector Memory for Semantic Dedup	41
Chapter 9 — Task Graph & Orchestrator	42
9.1 TaskGraph — Acyclic, Typed, Checkpointable	42
9.2 Saga — Atomic Multi-Step Side Effects	43
9.3 Checkpoint & Resume	44
9.4 Quarantine & Dead-Letter	44
9.5 Tracing — Trajectories, Not Just Outcomes	45
Chapter 10 — Plugin & Adapter Registry	46
10.1 Adapter Registry — First-Party Board Integrations	47
10.2 Adapter Registry Implementation	48
10.3 Plugin Registry — User-Installable Modules	49
Chapter 11 — Security, PII & Policy Engine	51
11.1 PII Masker — Strip Sensitive Data From Traces	51
11.2 Policy Engine — Caps & Enforcement	52
11.3 Audit Log of Every Side Effect	53
11.4 Worked Example — Daily Apply Cap	53
11.5 Secret Hygiene	53
Chapter 12 — Capability Module: Multi-Source Job Scraper	54
12.1 Strategy: Vendor JobSpy + Build Direct-API Adapters	54
12.2 File Layout	54
12.3 JobSpy Adapter — Full Scaffold	55
12.4 Career-Page Scanner — Generic 150+ Sites	56
12.5 Dedup Service — Hash + Vector	58
Chapter 13 — Capability Module: Resume Tailoring	59
13.1 Two-Agent Loop — Drafter & Reviewer	59
13.2 Drafter-Reviewer Loop Orchestrator	61

13.3 LaTeX Template System	62
13.4 ATS Parseability Check	63
Chapter 14 — Capability Module: Cover-Letter Generator	65
Chapter 15 — Capability Module: Auto-Apply Engine	67
15.1 Apply Saga — The Multi-Step Transaction	67
15.2 LinkedIn Easy Apply on Patchright	67
15.3 Question-Answering Module (Lifted from AIHawk)	69
15.4 Direct-API Adapters (Greenhouse, Lever)	71
15.5 Human-in-the-Loop Checkpoint	72
Chapter 16 — Capability Module: Interview Prep	73
16.1 Module File Layout	73
16.2 Mock Interview Session	74
16.3 STAR-Method Coach	75
Chapter 17 — Capability Module: Application Tracking & Dashboard	77
17.1 Application State Machine	77
17.2 Weekly Digest Email	78
17.3 Analytics Dashboard (Terminal + HTML)	79
Chapter 18 — Capability Module: Career Analytics & Salary Benchmarking	80
18.1 Salary Lookup	80
18.2 Skill-Gap Analysis	81
Chapter 19 — Capability Module: Outreach & Cold-DM Generation	83
19.1 Profile-Driven YAML Presets	83
19.2 LinkedIn People-Search URL Generator	84
19.3 DM Template Generator	85
19.4 SMTP Email Sender	85
Chapter 20 — Capability Module: Scheduler & Daily Loop	86
20.1 Modes — Scan, Apply, Digest, Full-Loop	86
20.2 Daily Loop Implementation	87
20.3 Cron Entry	88

Chapter 21 — Phased Migration Roadmap (Phases 0–4)

89

21.1 Phase 0 — Audit & Cleanup (Week 1)

89

21.2 Phase 1 — Provider Abstraction + Config Schema (Weeks 2–3)

89

21.3 Phase 2 — Scraper Merge (Weeks 4–5)

90

21.4 Phase 3 — Resume + Cover-Letter + Auto-Apply (Weeks 6–9)

91

21.5 Phase 4 — ATS, Analytics, Plugins, Docker (Weeks 10–13)

92

21.6 Timeline Summary

92

Chapter 22 — Risks, Mitigations & License Compliance

93

22.1 Risk Register

94

22.2 License Compliance — Re-iteration

95

Chapter 23 — Setup: Prerequisites & System Requirements

96

23.1 Hard Prerequisites

96

23.2 Soft Prerequisites

96

23.3 OS Support Matrix

97

23.4 Resource Requirements

97

Chapter 24 — Setup: Local Dev Installation

98

24.1 macOS (Intel & Apple Silicon)

98

24.2 Linux (Ubuntu 22.04+)

99

24.3 Windows 10/11 with WSL2 (Recommended)

99

24.4 Windows Native (Not Recommended)

100

Chapter 25 — Setup: Configuration & Secrets

101

25.1 Tier 1 — .env (Runtime Config)

101

25.2 Tier 3 — OS Keyring (API Keys & Master Key)

101

25.3 Tier 2 — secrets.yaml (Encrypted Board Credentials)

102

25.4 Tier 4 — Profiles (User Personas)

102

25.5 Config Wizard

104

Chapter 26 — Setup: AI Provider API Keys (All 6 Supported Providers)

105

26.1 Google Gemini (DEFAULT — Required for First Run)

105

26.2 OpenAI

105

26.3 Anthropic Claude

105

26.4 OpenAI-Compatible (OpenRouter, Groq, Together, Ollama, vLLM)

106

26.5 Mistral	106
26.6 Cohere	106
26.7 AWS Bedrock	107
26.8 Google Vertex AI	107
26.9 Switching Providers at Runtime	108
Chapter 27 — Setup: Docker	109
27.1 Dockerfile (Multi-Stage Build)	109
27.2 docker-compose.yml	110
27.3 Bringing Up the Stack	111
27.4 Secrets Management in Docker	111
27.5 Healthcheck + Restart Policy	111
27.6 Backup Strategy	112
Chapter 28 — Setup: CLI Reference	113
28.1 Initialization & Diagnostics	113
28.2 Configuration & Secrets	113
28.3 Scraping & Discovery	114
28.4 Ranking & Applying	114
28.5 Resume & Cover Letter	115
28.6 Interview Prep	115
28.7 Tracker & Analytics	115
28.8 Outreach	116
28.9 Scheduler & Loops	116
28.10 Plugin Management	116
28.11 Maintenance & Backup	117
Chapter 29 — Setup: Troubleshooting & FAQ	118
Appendix A — Full Source Repository Star Inventory	122
A.1 Repos That Could Not Be Located	123
Appendix B — Glossary	125

Executive Summary

This document is an architectural blueprint and operational playbook for merging the best functionality of **33 open-source job-search AI repositories** into **aryansinghnagar/JoBot** — a Python AGPL-3.0 project that has, as of August 2026, zero GitHub stars but the most sophisticated runtime primitives of any repo surveyed. The merge thesis is unusual: JoBot is not an empty vessel being filled. It is an architecturally ambitious chassis — async task graph, stealth browser, provider-neutral LLM router, encrypted storage, policy engine — onto which capability modules from higher-star repos must be **grafted**, not copied. Most source repos are functionally rich but architecturally primitive. JoBot is the inverse.

Three Tier-1 source repositories define the capability surface to absorb. **santifer/career-ops (63,659 stars, JavaScript/MIT)** is the gold standard — the only surveyed repo scoring well across all twelve capability columns, with 17 CI workflows, a plugin registry, and ~120 modular capability scripts. **MadsLorentzen/ai-job-search (31,411 stars, TypeScript/MIT)** contributes the drafter→reviewer two-agent loop and a LaTeX CV pipeline that no other repo matches. **feder-cr/Jobs_Applier_AI_Agent_AIHawk (30,159 stars, Python/AGPL-3.0)** is the historical OG auto-applier; its architecture is dated (single-script monolith, JSON persistence, no tests, no Docker, OpenAI-locked), but its question-answering module for application forms remains best-in-class and will be re-implemented behind JoBot's existing ModelRouter.

The single highest-leverage decision in this plan is to vendor **speedyapply/JobSpy (4,072 stars, Python/MIT)** as a pip dependency. One line in `pyproject.toml` gives JoBot six production board scrapers (LinkedIn, Indeed, Glassdoor, Google, ZipRecruiter, Bayt) with deduplication, behind JoBot's existing adapter registry and circuit-breaker primitives. No other single decision in this plan delivers as much capability per unit of engineering effort.



Multi-LLM Provider Support

JoBot will support six provider classes with **Google Gemini as the default**, configurable via a single environment variable or YAML profile. The provider abstraction is a Strategy-pattern `LLMProvider` base class with concrete implementations for Gemini, OpenAI, Anthropic, OpenAI-compatible endpoints (OpenRouter, Groq, Together, Ollama, vLLM), Mistral, Cohere, AWS Bedrock, and Google Vertex AI. The router is cost-aware: it selects the cheapest sufficient model for each task type, with explicit fallback chains on rate-limit or quota errors. JoBot already ships a 4-provider `ModelRouter`; this plan extends it to cover all six classes and adds Bedrock/Vertex enterprise authentication.

Confidence Levels

Per the `agents.md` operating protocol, confidence levels are stated explicitly. **High confidence:** the JobSpy integration, the provider abstraction design, the ASP 12-phase state machine, the circuit-breaker pattern, the SQLite/Fernet/Keyring storage plane. **Moderate confidence:** the Patchright LinkedIn Easy-Apply flow (LinkedIn actively detects and blocks automation; the cat-and-mouse game is real), the LaTeX resume pipeline on Windows (TeX Live setup is painful), the captcha-solver reliability claims of any single vendor. **Low confidence:** the long-term survival of any specific scraper selector against board UI changes, the legal defensibility of auto-submitted job applications under emerging AI-hiring regulations in the EU and several US states.

What This Document Is Not

This is not a copy-paste manifest. Source repositories are referenced by their architectural patterns, not by file-level lift-and-shift. Several high-star repos are legally untouchable for code vendoring due to missing licenses (NOASSERTION) or restrictive licenses incompatible with JoBot's AGPL-3.0; these are explicitly marked as **design references only** in Chapter 4. The merge plan assumes the reader is a Python engineer comfortable with `async`, `pydantic`, `typer`, and `pytest`. It does not teach Python.

Reading Order

JoBot contributors should read Parts I-IV in order, then use Part V as a setup reference. Newcomers should start with the Executive Summary, skip to Part V (Setup), run `jobot doctor`, then return to Parts I-IV for context. The appendices (full repo inventory + glossary) are reference material.

Chapter 1 — Scope, Assumptions & Methodology

1.1 Scope

The scope of this plan is the **JoBot core repository** — its CLI, engine, adapters, capability modules, configuration, and deployment artifacts. Out of scope: a hosted SaaS offering, a mobile client, a browser extension, a Slack/Discord bot, and any feature requiring paid third-party services beyond LLM API access. The plan is explicitly targeted at the open-source self-hosted use case: a single user or small team running JoBot on their own hardware against their own LLM API keys.

1.2 Assumptions

The following assumptions are foundational. If any is false, the plan must be revisited.

A1. JoBot's existing runtime primitives (async task graph, ModelRouter, SQLite WAL control plane, Fernet encryption, OS keyring integration, Patchright stealth browser, circuit breaker, PII masker, vector memory) are real, tested, and remain the substrate for all merged capabilities.

A2. Python 3.11+ is the target runtime. Async-first is the default execution model. No synchronous-only modules will be accepted into core.

A3. JoBot's AGPL-3.0 license is non-negotiable. Source code from AGPL-3.0 or GPL-compatible repos may be vendored; Apache/MIT/BSD code may be vendored; NOASSERTION or no-license code may be referenced for design but must be re-implemented.

A4. Patchright (stealth Playwright fork) is the primary browser automation layer. Selenium-based modules from source repos will be re-implemented on Patchright, not vendored.

A5. The user supplies their own LLM API keys. No free-tier abuse, no shared keys, no LLM proxy service baked into the codebase. Provider keys live in OS keyring or Fernet-encrypted secrets.yaml, never in .env or git.

A6. The job boards JoBot integrates with will continue to evolve their anti-automation defenses. Scrapers and auto-apply modules are expected to break; the architecture must make repair cheap (adapter pattern + selector healing + quarantine + dead-letter).

A7. The user is technically literate — can install Python, edit a YAML file, read a stack trace, and run docker compose. The setup docs target this competency floor, not a non-technical end user.

A8. JoBot is not a recruiter-replacement. It is a force multiplier for a job seeker who is actively engaged. The system always leaves a human-in-the-loop checkpoint before any external side effect more consequential than a job 'save'.

1.3 Research Methodology

The source repository inventory was gathered on 2026-08-13 via the GitHub REST API. Star counts, languages, licenses, archived flags, and last-push timestamps are API-verified, not estimated. Feature inventories were derived from each repository's README (fetched from raw.githubusercontent.com) and root-directory listing. Where anonymous API rate-limiting interrupted metadata collection, the GitHub search API was used as a fallback source (separate rate-limit budget) and the affected rows are marked accordingly in the research file. The complete research artifact is at `/home/z/my-project/upload/repo_research.md` and is reproduced as Appendix A.

1.4 Tiering Heuristic

Repositories are tiered by a combination of star count and maintenance signal: Tier 1 (10,000+ stars or 5,000+ with active maintenance within 30 days); Tier 2 (1,000–10,000 stars); Tier 3 (100–1,000 stars); Tier 4 (under 100 stars, archived, or too niche to justify integration effort). The heuristic is intentionally conservative: a 200-star repo with a unique capability not present anywhere else (e.g., the visa-sponsor filter in `job-ops`) is promoted to Tier 2 for that single capability. Conversely, a high-star repo with no maintainable architecture (e.g., AIHawk's single-script monolith) is demoted for code-vendoring purposes and treated as an algorithm reference only.

1.5 What This Plan Is

An architectural blueprint with production-grade code scaffolds. Every capability module is specified to interface-signature level, with concrete Python class and function definitions, pydantic schemas, and example invocations. The scaffolds are intended to be drop-in starting points — they compile and type-check under `mypy strict` — but they are not the final implementation. Each module's tests, edge-case handling, and production hardening are left to the integration engineer, with explicit notes on the expected failure modes drawn from the `agents.md` reliability-math and harness-engineering principles.

Chapter 2 — JoBot Current State Audit

An honest audit of **aryansinghnagar/JoBot** as of 2026-08-13. The repository has zero GitHub stars (it was created 2026-07-22, three weeks ago at time of writing) but contains a substantial engineering investment: ~30 pytest files, Ruff + mypy strict CI, a 12-phase Application State Protocol, and runtime primitives none of the 30k-star source repos possess. The audit covers both what exists and what is missing.

2.1 What JoBot Has Today

The following capabilities are implemented, tested, and merged into the main branch. They form the substrate onto which all merged capability modules will be grafted.

Capability	Status	Tested?	Source repo if applicable
Typer + rich CLI	Shipped	Yes	Native
Async task-graph engine	Shipped	Yes	Native (test_task_graph.py)
Patchright stealth browser	In progress	Yes	Native (test_browser_session.py)
SQLite WAL control plane	Shipped	Yes	Native (test_storage.py)
Fernet symmetric encryption	Shipped	Yes	Native
OS keyring integration	Shipped	Yes	Native
ModelRouter (4-provider)	Shipped	Yes	Native (test_ai.py)
Greenhouse adapter	Shipped	Yes	Native (test_greenhouse_adapter.py)
Adapter registry	Shipped	Yes	Native (test_adapter_registry.py)
Circuit breaker	Shipped	Yes	Native (test_circuit_breaker*.py)
Captcha solver hook	Stub	Yes	Native (test_captcha_solver.py)
Dedup service	Shipped	Yes	Native (test_dedup.py)
Policy engine (caps + enforcement)	Shipped	Yes	Native (test_policy*.py)
12-phase Application State Protocol	Shipped	Yes	Native (test_asp*.py)
Tracing + QA engine + alerts	Shipped	Yes	Native (test_*_wired.py)
Skill extraction + discovery	Shipped	Yes	Native
Vector memory	New (2026-08-11)	Yes	Native
PII masker	New (2026-08-11)	Yes	Native
~30 pytest files + npm vitest dual-stack	Shipped	Yes	Native
Ruff + mypy strict CI	Shipped	N/A	Native
AGPL-3.0 license	Shipped	N/A	Native

2.2 What JoBot Lacks

Twelve concrete capability gaps, in priority order. Each gap maps to one or more source repositories whose modules will fill it. The order is roughly the order in which the gaps will be closed in the phased roadmap (Chapter 21).

G1. Production scrapers for major job boards. Only the Greenhouse adapter is real. LinkedIn, Indeed, Glassdoor, Lever, Naukri, etc. are stubs being de-stubbed. LIFT FROM: speedyapply/JobSpy as a pip dependency + portal-health-locking pattern from career-ops.

G2. LaTeX / PDF resume tailoring. No CV compilation, no template system. LIFT FROM: drafter→reviewer loop + LaTeX pipeline from MadsLorentzen/ai-job-search; build-cv-latex.mjs pattern from career-ops.

G3. Cover-letter generation. Absent. LIFT FROM: generate-cover-letter.mjs pattern from career-ops; tone presets from replyre/job-hunter.

G4. Interview prep module. Absent. LIFT FROM: interview-prep/ directory from career-ops and /interview skill from ai-job-search; FAANG question bank from jennifer88huang/interview-skills.

G5. Application tracking dashboard / analytics. ASP exists but no UI or analytics layer. LIFT FROM: build-dashboard.mjs, weekly-digest.mjs, stats.mjs, funnel-velocity.mjs, rejection-latency.mjs patterns from career-ops; jobsync full-stack UI as long-term reference.

G6. Real ATS portal scanner. Mock ATS in tests only. LIFT FROM: discover-ats.mjs, scan-ats-full.mjs, detect-reposts.mjs from career-ops.

G7. Docker / container packaging. No Dockerfile. LIFT FROM: Dockerfile + docker-compose.yml from career-ops or DaKheera47/job-ops.

G8. Multi-board question-answering AI. ModelRouter exists but no question-answering pipeline wired to it. LIFT FROM: question-answering module from AIHawk (re-implement behind ModelRouter; AIHawk is OpenAI-locked).

G9. Outreach / cold-DM generation. Absent. LIFT FROM: profile-driven outreach pipeline from replyre/job-hunter (YAML presets + LinkedIn people-search URL generation).

G10. Scheduler / daily-loop. Task graph exists but no documented daily-loop. LIFT FROM: daily-digest scheduler from replyre/job-hunter; modes/ from career-ops.

G11. Salary benchmarking. Absent. LIFT FROM: salary_lookup.py from ai-job-search; salary-gap.mjs from career-ops.

G12. Plugin install / audit flow. Adapter registry exists but no user-facing plugin install. LIFT FROM: plugin-install.mjs, plugin-audit.mjs, plugins-registry/ from career-ops.

2.3 Verdict

Verdict

JoBot is **architecturally the most ambitious** of the 33 surveyed repos and **functionally the emptiest** in user-facing capabilities. It has primitives (ModelRouter, ASP, circuit breaker, task graph, vector memory, PII masker, stealth browser) that none of the higher-star repos possess. The merge should treat JoBot as the host chassis and graft the capability modules from Tier-1/Tier-2 repos onto it, **re-implementing** them behind JoBot's existing provider-neutral and policy-enforced abstractions rather than vendoring them whole. This is not a code-lift exercise. It is a re-implementation exercise informed by external references.

2.4 Repository Hygiene

Two hygiene issues must be resolved before the merge begins. First, the existing planning documents in JoBot's root — `JoBot_Refactor_Plan.md`, `unified_master_plan.md`, `implementation_contract_dev_0_1.md` through `_dev_2_0.md`, `implementation_contract_release_1_0.md`, `runtime_capability_matrix.md` — should be moved to `docs/history/` with a single pointer from the README. They are valuable archaeological context but they clutter the root and confuse newcomers about which plan is authoritative. Second, the in-repo `worklog.md` should be preserved as the team's execution log but clearly labeled as distinct from this planning document.

Chapter 3 — Source Repository Inventory (Star-Ranked Matrix)

All 33 inventoried repositories, sorted by star count descending. Star counts are API-verified as of 2026-08-13 (UTC 04:35–05:00). The Multi-AI column marks whether the repo ships a real provider-neutral LLM abstraction (✓✓ = best-in-class, ✓ = present, partial = limited, ✗ = absent / locked to one provider). The Tests / Docker / Docs columns use the same scale. License abbreviations: AGPL-3.0, MIT, GPL-3.0, NOASSERTION (GitHub could not detect a recognized license), None (no LICENSE file present at all).

3.1 Tier 1 — 10,000+ stars or 5,000+ with strong maintenance

Repo	Stars	Lang	License	Multi-AI	Tests	Docker	Docs
santifer/career-ops	63,659	JS	MIT	✓✓	✓✓	✓✓	✓✓
MadsLorentzen/ai-job-search	31,411	TS	MIT	✓	✓	✗	✓
feder-cr/AIHawk (Jobs_Applier)	30,159	Py	AGPL-3.0	✗	✗	✗	✓

Tier 1 defines the capability surface. career-ops is the gold standard (only repo scoring well on all 12 capability columns). MadsLorentzen contributes the drafter→reviewer loop and LaTeX CV pipeline. AIHawk's architecture is dated (single-script monolith, JSON persistence, no tests, no Docker, OpenAI-locked) but its question-answering algorithm is the historical OG.

3.2 Tier 2 — 1,000–10,000 stars

Repo	Stars	Lang	License	Multi-AI	Tests	Docker	Docs
speedyapply/JobSpy	4,072	Py	MIT	N/A	✓	✗	✓
DaKheera47/job-ops	3,840	TS	NOASSERTION	✓	partial	✓✓	✓
GodsScion/Auto_job_applier_linkedIn	2,688	Py	MIT	partial	partial	✗	partial
PaulMcInnis/JobFunnel (ARCHIVED)	2,178	Py	MIT	N/A	✓	✗	✓
can4hou6joeng4/boss-agent-cli	1,558	Py	MIT	✓	partial	✗	✓
eatmoreduck/boss-zhipin-scraper	1,094	Py	MIT	N/A	partial	✗	✓

Tier 2 fills specific capability gaps. JobSpy is the canonical scraper library (vendor as pip dep). job-ops is the closest spiritual sibling to JoBot's vision; its visa-sponsor-providers module is unique in the survey. The remaining Tier 2 repos contribute single patterns: LinkedIn resilience (GodsScion), reference scraper design (JobFunnel, archived), JSON-envelope output for agent-tool use (boss-agent-cli), and Chrome-CDP real-session-reuse stealth (boss-zhipin-scraper).

3.3 Tier 3 — 100–1,000 stars

Repo	Stars	Lang	License	Multi-AI	Tests	Docker	Docs
Gsync/jobsync	854	TS	MIT	✓	partial	✗	✓
wodsuz/EasyApplyJobsBot	804	Py	NOASSERTION	partial	✗	✗	partial
rainmanjam/jobspy-api	376	Py	(check)	N/A	partial	✓	✓
agentenatalie/get-job.skill	348	Py	NOASSERTION	✓	partial	✗	✓
strelov1/freehire	337	Go	MIT	N/A	partial	✗	✓
andrew-shwetzter/career-ops-plugin	463	HT ML	MIT	✓	partial	✗	✓
feder-cr/resume_render_from_jd	412	Py	MIT	partial	✗	✗	✓
jennifer88huang/interview-skills	272	JS	None	✓	partial	✗	✓
slothsheepking/jobclaw	208	?	—	partial	partial	✗	partial
feder-cr/lib_resume_builder_AIHawk	193	Py	MIT	partial	✗	✗	partial
imon333/Job-apply-AI-agent	177	Py	None	✗	✗	✗	partial
colophon-group/jobseek	167	Py	?	N/A	partial	✗	✓
surapuramakhil-org/Job_search_agent (ARCHIVED)	171	Py	AGPL-3.0	partial	✓	partial	✓
replyre/job-hunter	150	Py	None	partial	✗	✗	✓✓
beatwad/LinkedIn-AI-Job-Applier-Ultimate	147	Py	None	partial	✗	✗	partial

Tier 3 contributes single-capability references: jobsync for dashboard UX, replyre/job-hunter for outreach + scheduler + REST API design, interview-skills for FAANG mock-interview prompts, feder-cr resume libs for rendering algorithm references.

3.4 Tier 4 — <100 stars, archived, or too niche

Tier 4 repos are listed for completeness in Appendix A but are explicitly excluded from the merge plan. They are either stale (krishnavalliappan/JobScout, last push Aug 2024), Gemini-locked to a niche market (LuisMiguelFurlanettoSousa/auto-apply-bot for Brazilian Gupy), Chrome extensions (wrong deployment model — JoBot is CLI/server-first), or strictly dominated by a higher-tier repo (PunithVT/career-ops, 2 stars, AWS-deploy fork of the original). The Tier-4 repo with the most interesting architecture is **algsoch/job_agentic** (3 stars, Ollama-local, SQLite+CSV, rule+LLM scoring) — but its 'Job Intelligence OS' concept is functionally a subset of what JoBot already plans, and the absence of a LICENSE file makes it legally unusable for code vrending.

3.5 Feature Overlap Matrix (Top 10 by Stars)

Capability columns: AA = auto-apply, SC = scraping, RT = resume tailoring, IP = interview prep, AT = ATS tracking, CA = career analytics, BA = browser automation, MA = multi-agent, ML = multi-LLM provider, DK = Docker, K8 = Kubernetes, SH = scheduler.

Repo	Stars	AA	SC	RT	IP	AT	CA	BA	MA	ML	DK	K8	SH
career-ops	63k	p	✓	✓	✓	✓	✓	✓	✓	✓✓	✓	✗	✓
ai-job-search	31k	p	✓	✓✓	✓	p	✓	p	✓	✓	✗	✗	✓

Repo	Stars	AA	SC	RT	IP	AT	CA	BA	MA	ML	DK	K8	SH
AIHawk	30k	✓✓	p	✓	✗	p	✗	✓	✗	✗	✗	✗	✓
JobSpy	4.1k	✗	✓✓	✗	✗	✗	✗	✗	✗	N/A	✗	✗	✗
job-ops	3.8k	p	✓	p	✗	✓	✓	✓	✓	✓	✓	✗	✓
Auto_job_applier_linkedIn	2.7k	✓✓	✓	✓	✗	p	✗	✓	✗	p	✗	✗	✓
JobFunnel (archived)	2.2k	✗	✓✓	✗	✗	✗	✗	✗	✗	N/A	✗	✗	✓
boss-agent-cli	1.6k	✗	✓	✗	✗	p	✓	✗	p	✓	✗	✗	p
boss-zhipin-scraper	1.1k	✗	✓✓	✗	✗	✗	p	✗	✗	N/A	✗	✗	✗
jobsync	854	✗	p	✓	✗	✓✓	✓	✗	✗	✓	✗	✗	p

Legend: ✓✓ = best-in-class; ✓ = present; p = partial; ✗ = absent; N/A = not applicable (e.g., library without LLM). Only career-ops scores ✓ or better across all twelve columns. Multi-LLM provider support is genuinely rare — only career-ops, ai-job-search, job-ops, and JoBot itself ship a true provider-neutral abstraction. **No surveyed repo ships Kubernetes manifests.** This is an open market gap JoBot could claim.

Chapter 4 — Tiered Adoption Strategy

The merge strategy is tiered: absorb Tier-1 first (these define the capability surface), Tier-2 next (these fill specific gaps), Tier-3 selectively (single-capability references), Tier-4 not at all. Within each tier, the order is determined by a combination of capability value, integration cost, and license compatibility.

4.1 Tier 1 — Absorb First

These three repositories define what JoBot will become. Their capability modules are re-implemented behind JoBot's existing abstractions (ModelRouter, adapter registry, task graph, policy engine) rather than vendored verbatim.

4.1.1 santifer/career-ops (63,659★)

The single most complete reference implementation in the survey. Architecture: ~120 small `.mjs` files (one per capability), invoked via slash commands inside Claude Code / Codex / OpenCode / Antigravity. Persistence is JSON files on disk with file-locking via `pipeline-lock.mjs` and `portal-health-lock.mjs`. Has a plugin registry, 17 CI workflows (test, codeql, sbom, signature-ci, release, plugin-registry-validate, web-ci, gfi-claims, manifesto-guestbook), and 16 translated READMEs. **Engineered to run inside a host AI-CLI, not as a standalone service.**

What to lift:

- Plugin registry + plugin-audit + plugin-install scaffolder — adapt to JoBot's existing adapter registry.
- Per-provider LLM runner strategy (`openai-eval.mjs`, `gemini-eval.mjs`, `ollama-eval.mjs`, `openrouter-runner.mjs`) → port to JoBot's ModelRouter as Strategy classes.
- The ~120-capability script catalog → port the **CV builder**, **cover-letter generator**, **ATS scanner**, **weekly digest**, **dashboard builder**, **followup cadence**, **skill extractor**, **salary gap** into JoBot's async task graph.
- Test patterns (`*.test.mjs` co-location, integration tests, eval tests).
- Dockerfile + docker-compose.yml + SBOM/CodeQL CI patterns.
- A-F rubric JD scoring (1.0–5.0) — best-in-class scoring algorithm.

Why first: highest star count + only repo covering all 12 capability columns. Its JSON-file persistence model is simpler than JoBot's SQLite WAL — port the *logic*, keep JoBot's better persistence. Its 17 CI workflows are a gold standard for engineering posture that JoBot should match.

4.1.2 MadsLorentzen/ai-job-search (31,411★)

Architecture: Claude-Code-skill-pack. Slash commands (`/setup`, `/scrape`, `/rank`, `/apply`, `/interview`) backed by a `.agents/skills/<portal>-search/cli/` directory of small Bun+TypeScript tools (one per job board). Python handles LaTeX compilation and salary lookups. State lives in markdown + JSON profile files. CV is generated via LaTeX (`lualatex` + `xelatex`); an ATS parseability check uses `pdftotext` from poppler.

What to lift:

- The **drafter→reviewer two-agent loop** for CV/cover-letter generation — port as a JoBot task-graph subgraph.

- **LaTeX CV pipeline** (lualatex + xelatex + pdftotext ATS check).
- The .agents/skills/<portal>-search/ per-board skill pattern → adapt to JoBot's adapter registry (one Bun/TS skill per portal).
- /setup profile ingestion (documents-folder → profile).
- Salary benchmarking (salary_lookup.py).
- upstream-watch.yml pattern (track portal changes — alerts when a board's HTML structure changes).

Why this tier: second-highest stars + the only repo with a genuinely language-agnostic evaluation framework. Its slash-command UX maps cleanly onto JoBot's typer CLI. The Danish-portal skills are easy to swap for US/global ones.

4.1.3 feder-cr/Jobs_Applier_AI_Agent_AIHawk (30,159★)

Architecture: single-script monolith with thin module layer. `main.py` orchestrates a linear pipeline (login → search → iterate → per-job: tailoring + question-answering + submit). `src/` holds the modularization (LLM utils, browser actions, question answering, resume parser). Browser automation is Selenium + Chrome. Persistence is plain JSON under `data_folder/`. Config via `secrets.yaml` + `config.yaml`. No tests, no Docker.

What to lift:

- The **question-answering module** for application forms (its strongest asset) — re-implement behind JoBot's ModelRouter (AIHawk is OpenAI-locked).
- The **LinkedIn Easy Apply** Selenium flow → re-implement on Patchright (JoBot already chose the stealthier lib).
- Reference for the per-job application loop structure (search → iterate → per-job pipeline).

Skip AIHawk's architecture

Single-script monolith, JSON persistence, no tests, no Docker, OpenAI-locked. **Do not lift its code structure.** Only lift the question-answering + Easy-apply *algorithms*, re-implemented on JoBot's existing primitives. AIHawk's historical value (Aug 2024 OG) is real; its current engineering value is reference-only.

4.2 Tier 2 — Absorb Next (Specific Capability Gaps)

4.2.1 speedyapply/JobSpy (4,072★) — VENDOR AS PIP DEP

Pure-Python scraper library exposing a single `scrape_jobs()` API returning a pandas DataFrame. Internal adapter pattern: one `BaseScraper` subclass per board (LinkedIn, Indeed, Glassdoor, Google, ZipRecruiter, Bayt). No persistence, no LLM, no agent layer. **The cheapest, highest-leverage lift in the entire survey.**

```
# pyproject.toml
[project]
dependencies = [
    "jobspy>=0.4.0",
]

# Usage in JoBot adapter
from jobspy import scrape_jobs
from jobot.adapters.base import BoardAdapter
from jobot.storage.dedup import DedupService
```

```

from jobot.resilience.circuit_breaker import CircuitBreaker

class JobSpyAdapter(BoardAdapter):
    board_name = "jobspy"

    def __init__(self, dedup: DedupService, cb: CircuitBreaker):
        self._dedup = dedup
        self._cb = cb

    async def scrape(self, query: ScrapeQuery) -> list[JobPosting]:
        # JobSpy is sync – run in threadpool
        raw = await asyncio.to_thread(
            scrape_jobs,
            site_name=query.sites, # ["linkedin", "indeed", ...]
            search_term=query.keywords,
            location=query.location,
            results_wanted=query.limit,
            hours_old=query.hours_old,
        )
        postings = [self._row_to_posting(r) for r in raw.to_dict('records')]
        # Dedup against SQLite + vector memory
        unique = [p for p in postings if not self._dedup.is_dup(p)]
        self._cb.record_success()
        return unique

```

4.2.2 DaKheera47/job-ops (3,840★)

TypeScript monorepo: `career-boards/` (board adapters), `extractors/` (JD parsers), `orchestrator/` (pipeline runner), `shared/` (types), `visa-sponsor-providers/` (**unique — no other repo has this**). Uses Biome for lint, Knip for dead-code detection, Playwright-MCP for browser, runs both as CLI and via Docker. Architecturally the closest spiritual sibling to JoBot's vision. License is NOASSERTION — **read for inspiration, do not copy code verbatim** without clarifying license.

4.2.3 GodsScion/Auto_job_applier_linkedin (2,688★)

Selenium-based LinkedIn Easy Apply bot with LinkedIn-specific resilience: blacklisting, retry logic, session persistence. MIT-licensed. Lift the LinkedIn-specific hardening patterns (retry-with-backoff, selector healing, login state recovery) and re-implement on Patchright.

4.2.4 can4hou6joeng4/boss-agent-cli (1,558★)

Architecturally novel pattern: **designed as a tool for an AI agent, not as an agent itself** — the inverse of career-ops. Outputs JSON envelopes (structured tool-call-style output) so an AI host can consume results programmatically. **Lift the JSON-envelope output pattern** for making JoBot itself callable as a tool by other AI agents (MCP-friendly).

4.2.5 eatmoreduck/boss-zhipin-scraper (1,094★)

Chrome DevTools Protocol (CDP) scraper. Reuses the user's real logged-in browser session via CDP rather than launching a separate browser — stealthier and harder-to-detect than Selenium/Playwright. Niche (China market) but architecturally instructive for the stealth-browser layer JoBot is building with Patchright. **Lift the CDP session-reuse pattern** as an alternative stealth strategy when Patchright is detected.

4.3 Tier 3 — Selectively Absorb (Single-Capability References)

Repo	What to lift	Caveat
Gsync/jobsync (854★)	Full-stack tracker dashboard UX (kanban + analytics) for JoBot's eventual UI layer.	TS — separate UI subproject, not core.
replyre/job-hunter (150★)	Profile-driven YAML presets + daily digest scheduler + LinkedIn people-search URL generation + REST API surface (Postman collection included).	No license — design reference only, re-implement.
agentenatalie/get-job.skill + jennifer88huang/interview-skills	Interview-prep skill design (Chinese-market ref + FAANG mock-interview coach).	Claude-skill format — port prompt design, not skill packaging.
feder-cr/resume_render_from_jd + lib_resume_builder_AIHawk	Reference resume-rendering algorithms (the AIHawk resume lineage).	OpenAI-locked — re-implement behind ModelRouter.
strelov1/freehire	Go-based search-engine architecture reference (if JoBot ever adds a job-listings search endpoint).	Different language — design reference only.
beatwad/LinkedIn-AI-Job-Appplier-Ultimate	Telegram-reporting pattern + 'applies to ALL job types, not just Easy Apply' technique.	No license — design reference only.

4.4 Tier 4 — Skip (and Why)

Repo	Why skip
PaulMcInnis/JobFunnel (2,178★)	Archived Dec 2025. Architecture outdated (no LLM, no apply step). Historical reference only — JobSpy supersedes.
algsoch/job_agentic (3★)	No LICENSE file (legally unusable). 'Job Intelligence OS' is a subset of what JoBot plans. Ollama-only stance already covered by JoBot's ModelRouter.
krishnavalliappan/JobScout (47★)	Stale (Aug 2024), GPT-locked, no license. No value over Tier-1/Tier-2.
wodsuz/EasyApplyJobsBot (804★)	No LICENSE (NOASSERTION) → legally untouchable. Subset of AIHawk.
imon333/Job-apply-AI-agent (177★)	No license, OpenAI-locked, n8n-dependent. AIHawk supersedes.
All Tier-4 Chrome extensions	Wrong deployment model (JoBot is CLI/server-first). Architecturally incompatible.
LuisMiguelFurlanettoSousa/auto-apply-bot (34★)	Gemini-locked, Brazilian-market-focused (Gupy). Playwright-MCP pattern already available from job-ops.
PunithVT/career-ops (2★)	AWS-deploy fork of career-ops concept — strictly dominated by the original.

4.5 License Compliance Decision Tree

JoBot is AGPL-3.0. Code lifted from other repositories must respect license compatibility. The decision tree below is non-negotiable; violations expose JoBot's contributors to copyright infringement claims.

Source license	Action	Notes
AGPL-3.0	Vendor directly (with attribution)	Same license — fully compatible. Must keep original copyright + license header. AIHawk's AGPL-3.0 modules can be lifted verbatim (but their architecture is poor — re-implement anyway).
GPL-3.0	Vendor directly (with attribution)	GPL-3.0 → AGPL-3.0 is one-way compatible. sreekarrrs/JobSearch-Agent is GPL-3.0 — code can be lifted.

Source license	Action	Notes
Apache-2.0 / MIT / BSD-2/3-clause	Vendor directly (with attribution)	Permissive — compatible with AGPL. JobSpy, career-ops, GodsScion, JobFunnel, MadsLorentzen are all MIT — safe to vendor.
MPL-2.0	Vendor with care (file-level copyleft)	MPL is file-level copyleft. Lifted files must remain MPL. JoBot core can depend on them but cannot absorb them into AGPL-licensed files.
Unlicense / CC0	Vendor freely	Public domain dedication.
NOASSERTION (GitHub couldn't detect)	Reference only — DO NOT VENDOR	License file exists but isn't recognized. Risky. DaKheera47/job-ops is NOASSERTION — read for design, re-implement.
None (no LICENSE file)	Reference only — DO NOT VENDOR	Legally 'All Rights Reserved' by default. wodsuz/EasyApplyJobsBot, imon333/Job-apply-AI-agent, replyre/job-hunter, beatwad/LinkedIn-AI-Job-Applier-Ultimate, jennifer88huang/interview-skills are all in this category.
Proprietary / Commercial	Reference only — DO NOT VENDOR	Self-explanatory.

Legally untouchable repos (do not vendor code)

wodsuz/EasyApplyJobsBot (804★) — NOASSERTION. imon333/Job-apply-AI-agent (177★) — None. replyre/job-hunter (150★) — None. beatwad/LinkedIn-AI-Job-Applier-Ultimate (147★) — None. jennifer88huang/interview-skills (272★) — None. algoch/job_agentic (3★) — None. DaKheera47/job-ops (3,840★) — NOASSERTION. These seven repos may be read for design inspiration but their code **must not be vendored**. All lifted algorithms must be re-implemented from scratch in JoBot's own codebase.

4.6 Recommended Adoption Sequence

A 15+ week timeline. Each phase has explicit exit criteria in Chapter 21. The sequence is ordered by leverage (capability delivered per unit of engineering effort), not by Tier ranking.

Week 1–2. Vendor speedyapply/JobSpy as pip dep

Exit criterion: Instantly gain 6 board scrapers behind a JoBot adapter. Highest leverage / lowest risk. Exit: `jobot scrape linkedin` returns ≥ 50 real postings.

Week 3–6. Port Tier-1 capability modules from career-ops

Exit criterion: CV builder, cover-letter, ATS scanner, dashboard, digest, plugin registry, Docker, CI hardening. Behind JoBot's existing ModelRouter + SQLite + circuit-breaker primitives. Exit: `jobot apply --dry-run` produces a tailored resume + cover letter for any JD.

Week 7–10. Port drafter→reviewer + LaTeX pipeline from MadsLorentzen

Exit criterion: Two-agent CV loop with A-F rubric scoring. LaTeX compilation. ATS parseability check. Profile ingestion (/setup equivalent). Exit: `jobot resume tailor --jd <url>` produces ATS-safe PDF.

Week 11–14. Port question-answering + Easy-Apply loop from AIHawk

Exit criterion: Re-implemented on Patchright + ModelRouter. Saga pattern for atomic apply. Human-in-the-loop checkpoint for sensitive submissions. Exit: `jobot apply <job-id>` completes a real LinkedIn Easy Apply submission.

Week 15+. Selectively absorb Tier-3 single-capability references

Exit criterion: Interview prep, outreach, dashboard UX. As JoBot's surface area matures. Exit: each module has its own pytest suite + integration test against a mock ATS.

Chapter 5 — Target Architecture Overview

The target architecture is a layered system with explicit contracts between layers. The user interacts only with the CLI; the CLI translates commands into task-graph operations; the task graph orchestrates capability modules; capability modules consume cross-cutting services; cross-cutting services wrap external surfaces. No layer may skip a layer below it. No capability module may directly touch an external API — all external access goes through an adapter.

5.1 Layer Map

Layer	Component	Examples	Owns
L1 — User Surface	Typer + rich CLI	jobot scrape / apply / tracker / digest / plugin / config / doctor	Argument parsing, output formatting, exit codes
L2 — Orchestrator	Async task-graph engine	TaskGraph, TaskNode, Saga, Checkpoint, Quarantine	Task decomposition, parallelism, dependency ordering, retry, idempotency
L3 — Capability Modules	12 module packages	scrapers/, resume/, coverletter/, apply/, interview/, tracker/, career/, outreach/, scheduler/	Domain logic: JD parsing, CV generation, form filling, scoring
L4 — Cross-cutting Services	8 service planes	ModelRouter, BrowserPool, StoragePlane, PluginRegistry, PolicyEngine, TracingPlane, SkillExtractor, VectorMemory	Infrastructure: provider routing, browser lifecycle, persistence, secrets, policy enforcement, observability
L5 — External Surfaces (Adapters)	Adapter registry	LinkedInAdapter, GreenhouseAdapter, LeverAdapter, JobSpyAdapter, IndeedAdapter, ...	Board-specific quirks: selectors, rate-limits, auth flows
L6 — Storage & Secrets	3-tier persistence	SQLite WAL (control plane), Fernet blob store, OS keyring	Durable state, encrypted-at-rest secrets, OS-native credential storage
L7 — Observability	Tracing + alerts + dashboard	OpenTelemetry-style spans, alert rules, weekly digest email	Trace trajectories, alert on anomalies, surface aggregate stats

5.2 Design Patterns Chosen

Pattern choices are deliberate. Each is justified by a specific reliability or maintainability requirement drawn from the agents.md harness-engineering principles.

Pattern	Why chosen	Layer
Strategy (LLMProvider)	Allows swapping LLM providers at runtime without code changes. Required because JoBot must support 6 provider classes (Gemini default + OpenAI + Anthropic + OpenAI-compatible + Mistral/Cohere + Bedrock/Vertex).	L4 — ModelRouter
Factory (BoardAdapterRegistry)	Source board name → concrete adapter instance. Allows new board adapters to be registered without modifying the orchestrator. Required because new boards will be added continuously and selectors will break continuously.	L5 — Adapters
Adapter (board-specific quirks)	Translates JoBot's canonical JobPosting/ScrapeQuery/ApplyRequest types into board-specific API calls or DOM interactions. Required because LinkedIn, Greenhouse, Lever, Indeed each have radically different submission flows.	L5 — Adapters
Saga (multi-step apply workflows)	Each apply is a multi-step transaction (fetch form → answer questions → upload resume → submit → confirm). Partial failure must roll back via compensating actions. Required because a half-applied application is worse than no application — it leaves the user in an unknown state on the board's side.	L2 — Orchestrator

Pattern	Why chosen	Layer
State Machine (ASP 12-phase)	Application State Protocol: DISCOVERED → SAVED → APPLYING → APPLIED → SCREENING → INTERVIEW → OFFER → ACCEPTED (with terminal states REJECTED, WITHDRAWN, ARCHIVED). Required because application status is event-sourced and must be reconstructable from the event log.	L3 — Tracker
Circuit Breaker (external calls)	Per-adapter circuit breaker that opens after N consecutive failures and stays open for a cooldown period. Required because hammering a board that is rate-limiting or has changed its UI makes the situation worse.	L4 — Resilience
Idempotency Key (side effects)	Every external side effect (job applied, email sent, file written) carries a unique key. Retries are safe: the effect layer deduplicates on key. Required because the orchestrator may retry a task multiple times.	L2 — Orchestrator
Plugin (user-installed capabilities)	User-facing install/audit flow for community-contributed modules. Required because JoBot cannot anticipate every board or every workflow variant.	L4 — PluginRegistry
Observer (tracing + alerts)	Every task emits spans (start, end, attributes) consumed by an observability sink. Alerts are derived from span aggregates. Required because trace trajectories are first-class — a system that gets the right answer through a dangerous path is not yet reliable.	L7 — Observability
Repository (storage access)	Capability modules never touch SQLite directly; they go through repository interfaces. Required so that storage can be mocked in tests and swapped (SQLite → Postgres) without module changes.	L6 — Storage

5.3 Data Flow — End-to-End Apply Saga

Walks through a single `jobot apply <job-id>` invocation end-to-end, identifying every layer touched. This is the single most important data-flow to understand the architecture.

```

USER
|
| jobot apply <job-id>
v
[CLI L1] Typer parses args, loads profile from profiles/<name>.yaml
|
v
[ORCHESTRATOR L2] Build ApplySaga TaskGraph:
- TaskNode: fetch_job_from_storage(job_id) [L6 → SQLite]
- TaskNode: check_policy(user, action=APPLY) [L4 → PolicyEngine]
- TaskNode: route_to_llm_provider('gpt-4o-mini') [L4 → ModelRouter]
- TaskNode: tailor_resume(job, profile, llm) [L3 → resume module]
- TaskNode: tailor_cover_letter(job, profile, llm) [L3 → coverletter]
- TaskNode: render_latex_pdf(resume) [L3 → resume.latex]
- TaskNode: get_adapter(job.board_name) [L5 → adapter registry]
- TaskNode: adapter.submit_application(job, resume_pdf, cover_letter)
  with idempotency_key = sha256(job.id + user.id + date)
  with circuit_breaker.guard(job.board_name)
  with saga.compensating_action = adapter.withdraw_application
- TaskNode: persist_event(APPLIED, job.id, ts) [L6 → SQLite WAL]
- TaskNode: emit_span('apply.success', ...) [L7 → tracing]
|
v
[CAPABILITY MODULES L3] Each TaskNode executes:
- resume module calls ModelRouter.route() → Gemini
- coverletter module calls ModelRouter.route() → Gemini
- latex module runs lualatex → produces PDF
- apply module calls adapter via registry
|
v

```

```

[ADAPTER L5] LinkedInEasyApplyAdapter:
- BrowserPool.acquire() → Patchright context
- Stealth session resume from browser-profile/ volume
- Navigate to job URL
- Click 'Easy Apply' button
- Fill form fields (resume upload, custom questions)
- For each custom question: ModelRouter.route() → answer
- Submit
- Capture screenshot evidence
- BrowserPool.release()
|
v
[STORAGE L6] SQLite WAL:
- INSERT INTO applications (id, job_id, status, ts) VALUES (...)
- INSERT INTO events (application_id, type, payload) VALUES (...)
- Fernet-encrypt cover_letter_text → blob store
|
v
[OBSERVABILITY L7] Tracing:
- span 'apply.saga' (duration, success, retry_count)
- span 'llm.call' (provider, model, tokens_in, tokens_out, cost_usd)
- span 'browser.session' (board, duration, captcha_encountered)
- alert if cost_usd > daily_cap → email user
|
v
USER sees: ✓ Applied to <job-id> at <board> in 47s (cost: $0.023)

```

5.4 Anti-Patterns Explicitly Forbidden

Anti-pattern	Why forbidden
Capability module calls LLM directly (bypassing ModelRouter)	Locks the module to one provider. Violates Strategy pattern. Blocks cost-aware routing.
Capability module touches SQLite directly (bypassing Repository)	Couples module to storage backend. Blocks test mocking. Blocks future Postgres migration.
Orchestrator calls Adapter directly (bypassing registry)	Prevents new adapters from being added without code changes. Breaks plugin system.
Side effect without idempotency key	Retries are unsafe — same apply could be submitted twice. Violates saga contract.
Long-running task without checkpoint	Crash mid-run = full restart from zero. Violates durable-wait primitive.
Free-form LLM output at phase boundary	Breaks validation. Each phase output must validate against a pydantic schema. Free-form text alone is too weak.
Hidden retry storm (no quarantine)	Repeatedly failing task thrashes the system. Poison work must move to dead-letter queue with explicit replay.
Browser session per task (no pool reuse)	Each task launches a new browser → instant board rate-limit. Sessions must be pooled and persisted.

Chapter 6 — AI Provider Abstraction Layer

(ModelRouter v2)

JoBot's ModelRouter is its single most architecturally important component. It must be provider-neutral, cost-aware, fallback-capable, and support tool-calling. The existing v1 supports 4 providers (Gemini, OpenAI, Anthropic, Ollama). This chapter specifies v2 — extending support to 6 provider classes with Gemini as the default, adding OpenAI-compatible endpoints (OpenRouter/Groq/Together/vLLM), Mistral, Cohere, AWS Bedrock, and Google Vertex AI.

6.1 Design Goals

- **Provider-neutral.** Capability modules call `router.route(prompt, ...)` and never know which provider answered. Switching providers is a config change, not a code change.
- **Default = Gemini.** Out of the box, with only `GEMINI_API_KEY` set, JoBot works. Other providers are opt-in.
- **Cost-aware.** Each provider declares per-model cost-per-1k-tokens. The router picks the cheapest sufficient model unless the caller specifies otherwise.
- **Fallback-capable.** On rate-limit or quota error, the router tries the next provider in the configured fallback chain.
- **Tool-calling support.** For agentic loops (drafter→reviewer, form question-answering), the router exposes a unified `route_with_tools()` interface that maps to each provider's tool-calling API.
- **Streaming.** For long generations, the router supports async iteration over token chunks. Useful for user-facing CLI feedback.
- **Tracing-native.** Every call emits a span (provider, model, tokens_in, tokens_out, cost_usd, latency_ms). Aggregatable into daily cost dashboards.
- **Local-only mode.** Ollama and vLLM are first-class — JoBot must work fully offline with no cloud API keys, at reduced quality.

6.2 Provider Class Matrix

Provider class	Default model	Env vars	Free tier?	Tool calls?	Streaming?
Google Gemini (default)	gemini-2.0-flash	GEMINI_API_KEY or GOOGLE_API_KEY	Yes (15 RPM, 1500 RPD)	Yes	Yes
OpenAI	gpt-4o-mini	OPENAI_API_KEY	No (pay-as-you-go)	Yes	Yes
Anthropic Claude	claude-3-5-sonnet-20241022	ANTHROPIC_API_KEY	No (pay-as-you-go)	Yes	Yes
OpenAI-compatible (OpenRouter)	openai/gpt-4o-mini	OPENROUTER_API_KEY + OPENAI_BASE_URL=https://openrouter.ai/api/v1	Free models available	Yes	Yes
OpenAI-compatible (Groq)	llama-3.1-70b-versatile	GROQ_API_KEY + OPENAI_BASE_URL=https://api.groq.com/openai/v1	Yes (30 RPM free)	Yes	Yes
OpenAI-compatible (Together)	meta-llama/Llama-3-70b-chat-hf	TOGETHER_API_KEY + OPENAI_BASE_URL=https://api.together.xyz/v1	Free credits new users	Yes	Yes
OpenAI-compatible (Ollama local)	llama3.1:8b	OLLAMA_BASE_URL=http://localhost:11434/v1 (no key)	Free (self-hosted)	Yes	Yes
OpenAI-compatible (vLLM local)	any (server-launched)	VLLM_BASE_URL + OPENAI_BASE_URL	Free (self-hosted)	Yes	Yes
Mistral	mistral-large-latest	MISTRAL_API_KEY	Free tier (1 RPM)	Yes	Yes
Cohere	command-r-plus-08-2024	COHERE_API_KEY	Trial key (1000 calls)	Yes	Yes
AWS Bedrock	anthropic.claude-3-5-sonnet-20241022-v2:0	AWS_ACCESS_KEY_ID + AWS_SECRET_ACCESS_KEY + AWS_REGION	AWS free tier	Yes	Yes
Google Vertex AI	gemini-1.5-pro-002	GOOGLE_APPLICATION_CREDENTIALS (JSON) + VERTEX_PROJECT + VERTEX_REGION	Google Cloud free tier	Yes	Yes

Six provider classes, twelve concrete instances. Bedrock and Vertex are for enterprise auth (IAM-based, no API key file). Ollama and vLLM are for fully-local execution. All others are cloud API providers.

6.3 Interface Contract

The base class every provider must implement. Strategy pattern — the router picks the concrete provider at call time based on config and cost.

```
# jobot/llm/base.py
from __future__ import annotations
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from typing import AsyncIterator, Literal, Any
from pydantic import BaseModel, Field

class Role(str):
    SYSTEM = 'system'
    USER = 'user'
    ASSISTANT = 'assistant'
    TOOL = 'tool'
```

```

@dataclass(frozen=True)
class Message:
    role: str
    content: str
    name: str | None = None # for tool calls
    tool_call_id: str | None = None

@dataclass(frozen=True)
class ToolSpec:
    name: str
    description: str
    parameters: dict[str, Any] # JSON schema

@dataclass(frozen=True)
class ToolCall:
    id: str
    name: str
    arguments: dict[str, Any]

@dataclass(frozen=True)
class LLMResponse:
    text: str
    tool_calls: list[ToolCall] = field(default_factory=list)
    input_tokens: int = 0
    output_tokens: int = 0
    cost_usd: float = 0.0
    latency_ms: int = 0
    provider: str = ''
    model: str = ''
    raw: Any = None # provider-specific raw response

@dataclass(frozen=True)
class ProviderPricing:
    input_per_1k: float # USD per 1k input tokens
    output_per_1k: float # USD per 1k output tokens

class LLMPProvider(ABC):
    """Strategy interface – every concrete provider implements this."""
    name: str # 'gemini' | 'openai' | 'anthropic' | ...
    default_model: str
    pricing: dict[str, ProviderPricing]

    @abstractmethod
    async def complete(
        self,
        messages: list[Message],
        model: str | None = None,
        temperature: float = 0.7,
        max_tokens: int = 2048,
        tools: list[ToolSpec] | None = None,
        timeout_s: float = 60.0,
    ) -> LLMResponse:
        ...

    @abstractmethod
    async def stream(
        self,
        messages: list[Message],
        model: str | None = None,
        temperature: float = 0.7,

```

```

        max_tokens: int = 2048,
    ) -> AsyncIterator[str]:
        ...

    @abstractmethod
    def estimate_cost(self, input_tokens: int, output_tokens: int,
                      model: str | None = None) -> float:
        ...

    @abstractmethod
    async def health_check(self) -> bool:
        ...

```

6.4 Concrete Implementations

Below is the file layout and one concrete implementation (Gemini, the default). Other implementations follow the same pattern; their key differences are noted but full code is omitted for brevity. All implementations live under `jobot/llm/`.

```

jobot/llm/
├── __init__.py
├── base.py # LLMPProvider ABC + Message/ToolSpec/etc
├── gemini.py # Google Gemini (default)
├── openai.py # OpenAI direct
├── anthropic.py # Claude direct
├── openai_compat.py # OpenAI-compatible (OpenRouter/Groq/Together/Ollama/vLLM)
├── mistral.py # Mistral direct
├── cohere.py # Cohere direct
├── bedrock.py # AWS Bedrock (boto3-based)
├── vertex.py # Google Vertex AI (google-cloud-aiplatform)
├── router.py # ModelRouter – picks provider per call
└── pricing.yaml # Per-model pricing data (versioned)

```

Concrete implementation: Gemini (default provider).

```

# jobot/llm/gemini.py
from __future__ import annotations
import asyncio, time, os
from typing import AsyncIterator
import google.generativeai as genai
from .base import LLMPProvider, Message, LLMResponse, ToolSpec, ToolCall,
ProviderPricing
from ..tracing.span import emit_span

class GeminiProvider(LLMPProvider):
    name = 'gemini'
    default_model = 'gemini-2.0-flash'
    pricing = {
        'gemini-2.0-flash': ProviderPricing(input_per_1k=0.000075,
output_per_1k=0.0003),
        'gemini-2.0-pro': ProviderPricing(input_per_1k=0.00125, output_per_1k=0.005),
        'gemini-1.5-flash': ProviderPricing(input_per_1k=0.000075,
output_per_1k=0.0003),
        'gemini-1.5-pro': ProviderPricing(input_per_1k=0.00125, output_per_1k=0.005),
    }

    def __init__(self, api_key: str | None = None):
        self._api_key = api_key or os.environ.get('GEMINI_API_KEY') or
os.environ.get('GOOGLE_API_KEY')

```

```

if not self._api_key:
    raise RuntimeError('GEMINI_API_KEY (or GOOGLE_API_KEY) not set')
genai.configure(api_key=self._api_key)

async def complete(self, messages, model=None, temperature=0.7,
                    max_tokens=2048, tools=None, timeout_s=60.0):
    model_name = model or self.default_model
    # Map Message[] → Gemini's Content[] format
    contents = [{'role': m.role, 'parts': [m.content]} for m in messages]
    gen_model = genai.GenerativeModel(model_name)
    t0 = time.monotonic()
    try:
        # Gemini SDK is sync – run in threadpool
        resp = await asyncio.wait_for(
            asyncio.to_thread(
                gen_model.generate_content,
                contents,
                generation_config={'temperature': temperature,
                                   'max_output_tokens': max_tokens},
            ),
            timeout=timeout_s,
        )
        text = resp.text or ''
        usage = resp.usage_metadata
        cost = self.estimate_cost(usage.prompt_token_count,
                                   usage.candidates_token_count, model_name)
        await emit_span('llm.call', provider='gemini', model=model_name,
                        input_tokens=usage.prompt_token_count,
                        output_tokens=usage.candidates_token_count,
                        cost_usd=cost,
        latency_ms=int((time.monotonic()-t0)*1000))
        return LLMResponse(text=text, input_tokens=usage.prompt_token_count,
                            output_tokens=usage.candidates_token_count,
                            cost_usd=cost,
        latency_ms=int((time.monotonic()-t0)*1000),
                            provider='gemini', model=model_name, raw=resp)
    except Exception as e:
        await emit_span('llm.error', provider='gemini', model=model_name,
        error=str(e))
        raise

    async def stream(self, messages, model=None, temperature=0.7, max_tokens=2048):
        # Gemini's generate_content(stream=True) returns a sync iterator
        model_name = model or self.default_model
        gen_model = genai.GenerativeModel(model_name)
        contents = [{'role': m.role, 'parts': [m.content]} for m in messages]
        sync_iter = gen_model.generate_content(contents, stream=True,
                                                generation_config={'temperature':
temperature,
                                                                    'max_output_tokens'
: max_tokens})
        for chunk in sync_iter:
            if chunk.text:
                yield chunk.text
            await asyncio.sleep(0) # cooperative

    def estimate_cost(self, input_tokens, output_tokens, model=None):
        p = self.pricing.get(model or self.default_model,
self.pricing[self.default_model])
        return (input_tokens / 1000) * p.input_per_1k + (output_tokens / 1000) *

```

```
p.output_per_1k

    async def health_check(self) -> bool:
        try:
            await self.complete([Message('user', 'ping')], max_tokens=5)
            return True
        except Exception:
            return False
```

6.5 The Router — Provider Selection Logic

The ModelRouter is the capability-facing surface. Modules call `router.route()` and never instantiate a provider directly. The router is cost-aware: it selects the cheapest sufficient model unless the caller specifies one.

```
# jobot/llm/router.py
from __future__ import annotations
from dataclasses import dataclass
from .base import LLMPProvider, Message, LLMResponse, ToolSpec
from .gemini import GeminiProvider
from .openai import OpenAIProvider
from .anthropic import AnthropicProvider
from .openai_compat import OpenAICompatProvider
from .mistral import MistralProvider
from .cohere import CohereProvider
from .bedrock import BedrockProvider
from .vertex import VertexProvider

PROVIDER_REGISTRY = {
    'gemini': GeminiProvider,
    'openai': OpenAIProvider,
    'anthropic': AnthropicProvider,
    'openrouter': lambda:
OpenAICompatProvider(base_url='https://openrouter.ai/api/v1'),
    'groq': lambda: OpenAICompatProvider(base_url='https://api.groq.com/openai/v1'),
    'together': lambda: OpenAICompatProvider(base_url='https://api.together.xyz/v1'),
    'ollama': lambda: OpenAICompatProvider(base_url=os.environ.get('OLLAMA_BASE_URL', '
http://localhost:11434/v1'), api_key='dummy'),
    'vllm': lambda: OpenAICompatProvider(base_url=os.environ.get('VLLM_BASE_URL'),
api_key='dummy'),
    'mistral': MistralProvider,
    'cohere': CohereProvider,
    'bedrock': BedrockProvider,
    'vertex': VertexProvider,
}

@dataclass(frozen=True)
class RouterConfig:
    default_provider: str = 'gemini'
    fallback_chain: tuple[str, ...] = ('gemini', 'openai', 'anthropic')
    max_retries: int = 2
    daily_cost_cap_usd: float = 5.0

class ModelRouter:
    def __init__(self, cfg: RouterConfig):
        self._cfg = cfg
        self._instances: dict[str, LLMPProvider] = {}
        self._daily_spend: float = 0.0

    def _get(self, name: str) -> LLMPProvider:
```



```

    if name not in self._instances:
        factory = PROVIDER_REGISTRY[name]
        self._instances[name] = factory() if callable(factory) else factory
    return self._instances[name]

    async def route(self, messages, model=None, **kw) -> LLMResponse:
        # Try default provider first, then fallback chain on failure
        chain = (self._cfg.default_provider,) + self._cfg.fallback_chain
        last_err = None
        for prov_name in dict.fromkeys(chain): # dedup preserving order
            prov = self._get(prov_name)
            try:
                resp = await prov.complete(messages, model=model, **kw)
                # Check cost cap BEFORE returning – caller may have hit daily cap
                if self._daily_spend + resp.cost_usd > self._cfg.daily_cost_cap_usd:
                    raise RuntimeError(f'Daily cost cap
                    ${self._cfg.daily_cost_cap_usd} '
                                   f'would be exceeded
                                   (current=${self._daily_spend:.4f}, '
                                   f'this_call=${resp.cost_usd:.4f})')
                self._daily_spend += resp.cost_usd
                return resp
            except Exception as e:
                last_err = e
                # Log + try next provider
                await emit_span('router.fallback', from_provider=prov_name,
                               error=str(e))
            raise RuntimeError(f'All providers exhausted; last error: {last_err}')

        async def route_with_tools(self, messages, tools, model=None, **kw) ->
        LLMResponse:
            return await self.route(messages, model=model, tools=tools, **kw)

    def reset_daily_spend(self):
        """Called by scheduler at midnight UTC."""
        self._daily_spend = 0.0

```

6.6 Configuration Schema

Provider selection is configured via YAML profile or environment variable. The precedence is: explicit CLI flag > profile YAML > environment variable > default.

```

# profiles/default.yaml – minimal config (Gemini default)
llm:
  default_provider: gemini
  fallback_chain: [gemini]
  daily_cost_cap_usd: 5.0
  default_model: gemini-2.0-flash

# profiles/power-user.yaml – multi-provider with fallback
llm:
  default_provider: gemini
  fallback_chain: [gemini, openai, anthropic]
  daily_cost_cap_usd: 25.0
  default_model: gemini-2.0-flash
  task_overrides:
    resume_tailoring: # use stronger model for resume work
      provider: anthropic
      model: claude-3-5-sonnet-20241022

```

```

question_answering: # cheap model for form Q&A
  provider: gemini
  model: gemini-2.0-flash
interview_prep: # strong reasoning for mock interviews
  provider: openai
  model: gpt-4o

# .env (NEVER commit; .gitignore'd)
GEMINI_API_KEY=AIZA...
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...
OPENROUTER_API_KEY=sk-or-...
GROQ_API_KEY=gsk-...
TOGETHER_API_KEY=...
MISTRAL_API_KEY=...
COHERE_API_KEY=...
# For Bedrock (AWS):
AWS_ACCESS_KEY_ID=...
AWS_SECRET_ACCESS_KEY=...
AWS_REGION=us-east-1
# For Vertex AI (GCP):
GOOGLE_APPLICATION_CREDENTIALS=/path/to/sa-key.json
VERTEX_PROJECT=my-gcp-project
VERTEX_REGION=us-central1

```

Switching providers at runtime

`jobot config set llm.default_provider anthropic` writes the new value to the active profile YAML. The next LLM call uses Anthropic. The router's `_instances` dict is lazy — switching providers does not require a restart, but an already-instantiated provider keeps its existing client. To force re-instantiation: `jobot config reload`.

Chapter 7 — Browser Automation Layer

Browser automation is JoBot's most fragile layer. Every board (LinkedIn, Indeed, Glassdoor, Lever, Greenhouse, Workday) actively evolves its anti-automation defenses. Selectors break. Captchas appear. Login flows change. The architecture must make repair cheap: named actions (not one-off DOM scripts), selector healing, action caching, preview-before-commit, and evidence capture on every risky action. This chapter specifies the layer.

7.1 Patchright as Primary Stealth Browser

Patchright is a stealth fork of Playwright that patches several detection vectors Playwright leaves open by default (`navigator.webdriver`, `Runtime.enable` CDP leak, CDP-related window properties, headless-mode detection). It is API-compatible with Playwright, so any Playwright code works unmodified. JoBot's existing `test_browser_session.py` already validates the integration. This is the default for all browser-touching adapters.

7.2 CDP Session Reuse as Alternative Stealth

When Patchright is detected (rare but possible), JoBot falls back to CDP session reuse — lifted from [eatmoreduck/boos-zhipin-scraper \(1,094★\)](#). CDP attaches to the user's real Chrome session via the remote-debugging-port. Selectors run in the same context as the user's manual browsing, making the automation indistinguishable from human activity. Tradeoff: requires the user to launch Chrome with `--remote-debugging-port=9222` and log in manually first. Useful for high-value, low-volume operations (e.g., submitting 1 application per day to a competitive employer).

7.3 BrowserPool — Session Lifecycle

Each board gets a persistent browser context with its own `user-data-dir` volume. Contexts are kept warm across CLI invocations — the second `jobot apply` to LinkedIn reuses the same logged-in session as the first, eliminating repeated login prompts and reducing detection surface. The pool limits concurrency per board (default 1) to avoid obvious bot patterns.

```
# jobot/browser/pool.py
from __future__ import annotations
import asyncio, os
from pathlib import Path
from patchright.async_api import async_playwright, Browser, BrowserContext

class BrowserPool:
    def __init__(self, profile_dir: Path, headless: bool = True,
                 max_per_board: int = 1):
        self._profile_dir = profile_dir
        self._headless = headless
        self._max_per_board = max_per_board
        self._playwright = None
        self._browser: Browser | None = None
        self._contexts: dict[str, list[BrowserContext]] = {}
        self._locks: dict[str, asyncio.Lock] = {}

    async def start(self):
        self._playwright = await async_playwright().start()
```

```

self._browser = await self._playwright.chromium.launch(
    headless=self._headless,
    args=['--disable-blink-features=AutomationControlled'],
)

async def acquire(self, board: str) -> BrowserContext:
    if board not in self._locks:
        self._locks[board] = asyncio.Lock()
    async with self._locks[board]:
        ctxs = self._contexts.setdefault(board, [])
        if not ctxs:
            user_data_dir = self._profile_dir / board
            user_data_dir.mkdir(parents=True, exist_ok=True)
            ctx = await self._browser.new_persistent_context(
                user_data_dir=str(user_data_dir),
                viewport={'width': 1366, 'height': 768},
                locale='en-US',
                timezone_id='America/New_York',
                # Stealth patches: real-looking UA
                user_agent='Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) '
                           'AppleWebKit/537.36 (KHTML, like Gecko) '
                           'Chrome/131.0.0.0 Safari/537.36',
            )
            # Apply stealth init script
            await ctx.add_init_script(
                'Object.defineProperty(navigator, "webdriver", {get: () =>
undefined});'
            )
            ctxs.append(ctx)
        return ctxs[0]

async def release(self, ctx: BrowserContext):
    # Don't close – keep context warm for next call.
    # Optionally clear cookies/storage if board='anonymous'.
    pass

async def stop(self):
    if self._browser:
        for ctxs in self._contexts.values():
            for ctx in ctxs:
                await ctx.close()
        await self._browser.close()
    if self._playwright:
        await self._playwright.stop()

```

7.4 Named Actions vs One-Off DOM Scripts

Per agents.md reliability principle 23: 'Prefer named browser actions over one-off DOM scripts.' Each board adapter exposes named, typed methods — never `page.click("//button[contains(@class, 'apply')]")` in the orchestrator. The named action is a single place to fix when the selector breaks. The orchestrator calls `adapter.click_easy_apply_button(page)` and the adapter encapsulates the selector (and its healing logic).

```

# jobot/adapters/linkedin/named_actions.py
from patchright.async_api import Page

async def click_easy_apply_button(page: Page) -> bool:

```

```

"""Find and click the LinkedIn Easy Apply button. Returns True if clicked."""
# Try multiple selectors – first match wins (selector healing)
selectors = [
    'button.jobs-apply-button', # 2024-era
    'button[aria-label="Easy Apply"]', # alt
    'button.jobs-apply-button--top-card', # post-2024 redesign
    '//button[contains(@class, "jobs-apply")]', # XPath fallback
]
for sel in selectors:
    try:
        btn = await page.wait_for_selector(sel, timeout=3000)
        if btn:
            await btn.click()
            return True
    except Exception:
        continue
return False

async def fill_text_input(page: Page, label: str, value: str) -> bool:
    """Fill a form text input identified by its visible label."""
    # Try by label text first, then by aria-label, then by placeholder
    sel = (
        f'label:has-text("{label}") + input, '
        f'input[aria-label="{label}"], '
        f'input[placeholder*="{label}"]'
    )
    field = await page.query_selector(sel)
    if not field:
        return False
    await field.fill(value)
    return True

async def capture_evidence(page: Page, step_name: str) -> str:
    """Screenshot before/after risky action. Returns file path."""
    import time, os
    path = f'/tmp/jobot-evidence/{step_name}-{int(time.time())}.png'
    os.makedirs(os.path.dirname(path), exist_ok=True)
    await page.screenshot(path=path, full_page=True)
    return path

```

7.5 Captcha Solver Hook

Captchas are inevitable. JoBot's captcha solver hook is a pluggable interface with three implementations: (1) 2captcha API integration (paid, \$1/1000 solves); (2) local heuristic solver for simple math captchas; (3) human-in-the-loop fallback — pause execution, ring a bell, wait for the user to solve in the visible browser, then continue. The fallback is the most reliable for unsolvable puzzles but breaks the unattended-mode promise.

```

# jobot/browser/captcha.py
from __future__ import annotations
from abc import ABC, abstractmethod
from patchright.async_api import Page

class CaptchaSolver(ABC):
    @abstractmethod
    async def solve(self, page: Page, captcha_type: str) -> bool:
        '''Returns True if captcha was solved, False if it remains.'''

```

```

class TwoCaptchaSolver(CaptchaSolver):
    def __init__(self, api_key: str):
        self._api = api_key

    async def solve(self, page, captcha_type):
        # Implement 2captcha.com API integration
        # Submit image, poll for solution, inject answer into page
        raise NotImplementedError('Wire 2captcha API here')

class HumanFallbackSolver(CaptchaSolver):
    async def solve(self, page, captcha_type):
        # Bring browser to foreground (requires headed mode)
        # Print loud message + bell
        print('\a\a\a CAPTCHA ENCOUNTERED – solve it in the browser, '
              'then press Enter to continue...')
        await asyncio.to_thread(input)
        # Check if captcha is gone
        return not await self._captcha_present(page, captcha_type)

    async def _captcha_present(self, page, captcha_type):
        return bool(await page.query_selector('iframe[src*="recaptcha"]'))

```

7.6 Selector Healing + Action Caching

Selector healing: when a named action's primary selector fails, it tries alternatives in order. When all fail, it logs the failure with a screenshot and emits a `selector.broken` span — which triggers an alert. The user can then contribute a new selector via a PR or a YAML override file. Action caching: idempotent actions (e.g., 'navigate to job URL') are cached per session to avoid redundant navigation. The cache key is a hash of the action name + arguments; the value is a 'success' marker. Invalidate on any failure.

7.7 Preview-Before-Commit

For sensitive actions (clicking 'Submit Application'), the adapter captures a screenshot of the final form state and writes it to `data/evidence/<job-id>/pre-submit.png`. In interactive mode, the user must confirm by pressing Enter before the submit click fires. In unattended mode (with `- -yes`), the screenshot is still captured — it becomes the audit trail. This is the human-in-the-loop checkpoint that protects against catastrophic mis-submissions.

Chapter 8 — Storage & State Plane

JoBot's storage is three tiers, each with a different durability guarantee and access pattern. Tier 1 (control plane) is hot, frequently-written, structured. Tier 2 (blob store) is warm, occasionally-read, unstructured. Tier 3 (OS keyring) is cold, rarely-read, sensitive. The three tiers are deliberately separated to enforce access discipline — capability modules never touch Tier 3 directly.

8.1 Tier 1 — SQLite WAL Control Plane

Hot structured state: jobs, applications, events, traces, spans. SQLite WAL mode (Write-Ahead Logging) allows concurrent readers + single writer — perfect for JoBot's typical workload (one writer, many readers from the dashboard). Schema migrations are explicit (alembic-style) and tested. Pydantic models mirror each table for type-safe access via repository classes.

```
# jobot/storage/schema.py – core table definitions
from sqlalchemy import Column, String, Integer, DateTime, Text, JSON, ForeignKey
from sqlalchemy.orm import DeclarativeBase
from datetime import datetime

class Base(DeclarativeBase): pass

class JobPosting(Base):
    __tablename__ = 'jobs'
    id = Column(String, primary_key=True) # uuid4 hex
    board = Column(String, index=True) # 'linkedin' | 'greenhouse' | ...
    external_id = Column(String, index=True) # board's own job id
    title = Column(String)
    company = Column(String, index=True)
    location = Column(String)
    url = Column(String)
    description = Column(Text) # full JD text
    description_embedding = Column(JSON) # vector for dedup
    scraped_at = Column(DateTime, default=datetime.utcnow)
    salary_low = Column(Integer, nullable=True)
    salary_high = Column(Integer, nullable=True)
    salary_currency = Column(String(3), nullable=True)
    remote = Column(String, default='unknown') # 'yes' | 'no' | 'hybrid' | 'unknown'
    visa_sponsor = Column(String, default='unknown') # 'yes' | 'no' | 'unknown'

class Application(Base):
    __tablename__ = 'applications'
    id = Column(String, primary_key=True)
    job_id = Column(String, ForeignKey('jobs.id'), index=True)
    user_id = Column(String, index=True)
    status = Column(String, index=True) # ASP state: 'discovered'..'accepted'
    applied_at = Column(DateTime, nullable=True)
    resume_blob_id = Column(String, ForeignKey('blobs.id'), nullable=True)
    cover_letter_blob_id = Column(String, ForeignKey('blobs.id'), nullable=True)
    cost_usd = Column(Integer, default=0) # cents
    idempotency_key = Column(String, unique=True, index=True)

class ApplicationEvent(Base):
    __tablename__ = 'events'
    id = Column(Integer, primary_key=True, autoincrement=True)
    application_id = Column(String, ForeignKey('applications.id'), index=True)
    event_type = Column(String, index=True) # 'status_change' | 'llm_call' |
    'browser_action' | ...
```

```

payload = Column(JSON)
ts = Column(DateTime, default=datetime.utcnow)

class TraceSpan(Base):
    __tablename__ = 'spans'
    id = Column(String, primary_key=True) # uuid4
    trace_id = Column(String, index=True)
    parent_id = Column(String, nullable=True)
    name = Column(String, index=True) # 'llm.call' | 'browser.session' | ...
    attributes = Column(JSON)
    start_ts = Column(DateTime)
    end_ts = Column(DateTime, nullable=True)
    status = Column(String, default='running') # 'running' | 'ok' | 'error'

class Blob(Base):
    __tablename__ = 'blobs'
    id = Column(String, primary_key=True)
    kind = Column(String) # 'resume_pdf' | 'cover_letter_text' | 'jd_full'
    cipher_iv = Column(String) # Fernet IV (base64)
    cipher_text = Column(Text) # encrypted blob (base64)
    sha256 = Column(String, index=True) # plaintext hash for dedup
    created_at = Column(DateTime, default=datetime.utcnow)

```

8.2 Tier 2 — Fernet-Encrypted Blob Store

Warm unstructured state: resume PDFs (binary, encrypted), cover letters (text, encrypted), full JD text (too large to keep in SQLite comfortably). The encryption key is derived from a master key in OS keyring (Tier 3) via HKDF-SHA256. Each blob has a unique IV. The plaintext SHA-256 is stored alongside the ciphertext for dedup — if the same resume is uploaded twice, the second upload is a no-op.

```

# jobot/storage/blobs.py
from cryptography.fernet import Fernet
from hashlib import sha256
from base64 import b64encode
import os, json
from .schema import Blob, SessionLocal

class BlobStore:
    def __init__(self, master_key: bytes):
        # Derive per-blob encryption key from master via HKDF
        from cryptography.hazmat.primitives.kdf.hkdf import HKDF
        from cryptography.hazmat.primitives import hashes
        self._derive = lambda salt: HKDF(
            algorithm=hashes.SHA256(), length=32, salt=salt,
            info=b'jobot-blob-store-v1'
        ).derive(master_key)

    async def put(self, kind: str, plaintext: bytes) -> str:
        sha = sha256(plaintext).hexdigest()
        # Dedup: return existing blob id if sha matches
        with SessionLocal() as s:
            existing = s.query(Blob).filter_by(sha256=sha).first()
            if existing:
                return existing.id
        # New blob
        salt = os.urandom(16)
        key = self._derive(salt)
        f = Fernet(b64encode(key))

```



```

ct = f.encrypt(plaintext)
blob_id = os.urandom(16).hex()
with SessionLocal() as s:
    s.add(Blob(id=blob_id, kind=kind,
               cipher_iv=b64encode(salt).decode(),
               cipher_text=ct.decode(), sha256=sha))
    s.commit()
return blob_id

async def get(self, blob_id: str) -> bytes:
    with SessionLocal() as s:
        b = s.query(Blob).get(blob_id)
        if not b:
            raise KeyError(blob_id)
        salt = __import__('base64').b64decode(b.cipher_iv)
        key = self._derive(salt)
        f = Fernet(b64encode(key))
        return f.decrypt(b.cipher_text.encode())

```

8.3 Tier 3 — OS Keyring

Cold sensitive state: LLM API keys, board credentials (LinkedIn cookies, etc.), the Fernet master key itself. Stored via the **keyring** Python library which uses the OS-native credential store: macOS Keychain, Linux Secret Service (gnome-keyring/kwalletd), Windows Credential Manager. Never written to disk in plaintext. Never logged. Never appears in trace spans (the PII masker sees to that).

```

# jobot/storage/secrets.py
import keyring
from typing import Optional

SERVICE_NAME = 'jobot'

class SecretsStore:
    @staticmethod
    def set(key: str, value: str):
        keyring.set_password(SERVICE_NAME, key, value)

    @staticmethod
    def get(key: str) -> Optional[str]:
        return keyring.get_password(SERVICE_NAME, key)

    @staticmethod
    def delete(key: str):
        try:
            keyring.delete_password(SERVICE_NAME, key)
        except keyring.errors.PasswordDeleteError:
            pass

# Example usage during setup:
# jobot config set llm.api_key.gemini AIza...
# Internally calls SecretsStore.set('llm.api_key.gemini', 'AIza...')
# jobot config get llm.api_key.gemini -> 'AIza...'
# (Key never appears in .env, never logged, never in trace spans.)

```

8.4 Migration System

Schema migrations use a lightweight alembic-style versioned approach. Each migration is a Python module under `jobot/storage/migrations/` with `up()` and `down()` functions. A `schema_version` table tracks the current version. `jobot migrate` runs all pending `up()` calls; `jobot migrate --rollback N` rolls back N versions. Migrations are transactional — a failure rolls back the transaction, leaving the DB in the prior state.

8.5 Vector Memory for Semantic Dedup

Job postings are frequently reposted with minor edits (salary updated, location tweaked, JD reworded). String-hash dedup misses these. JoBot's vector memory embeds each JD with the configured embedding model (default: `text-embedding-004` from Gemini, free tier) and stores the embedding in the `jobs.description_embedding` column. Dedup query: cosine similarity > 0.92 against any existing posting from the same company within the last 30 days. The `test_dedup.py` test validates this.

Chapter 9 — Task Graph & Orchestrator

The orchestrator is JoBot's control plane. It accepts a high-level intent (e.g., 'apply to this job'), decomposes it into a task graph, executes the graph with parallelism where dependencies allow, checkpoints after each phase for resumability, and quarantines repeatedly-failing tasks. It is the layer most directly shaped by the agents.md reliability-math principles — every design choice here is in service of the march of nines.

9.1 TaskGraph — Acyclic, Typed, Checkpointable

A TaskGraph is a DAG of TaskNodes. Each node has typed inputs, typed outputs, a sync or async callable, and an idempotency key. Edges declare data dependencies (output of A → input of B) and ordering constraints (B must run after A regardless of data). Independent nodes may run in parallel — the executor uses `asyncio.gather` over `asyncio.Semaphore` to bound concurrency.

```
# jobot/orchestrator/graph.py
from __future__ import annotations
from dataclasses import dataclass, field
from typing import Any, Awaitable, Callable
import asyncio, hashlib, json

@dataclass(frozen=True)
class TaskNode:
    name: str
    func: Callable[..., Awaitable[Any]]
    inputs: dict[str, Any] = field(default_factory=dict)
    depends_on: tuple[str, ...] = ()
    retry_count: int = 0
    timeout_s: float = 60.0

    @property
    def idempotency_key(self) -> str:
        # Hash of name + sorted inputs (excluding runtime vars)
        payload = json.dumps({'name': self.name, 'inputs': self.inputs},
                             sort_keys=True, default=str)
        return hashlib.sha256(payload.encode()).hexdigest()[:16]

class TaskGraph:
    def __init__(self, name: str):
        self.name = name
        self._nodes: dict[str, TaskNode] = {}
        self._edges: list[tuple[str, str]] = []

    def add(self, node: TaskNode) -> 'TaskGraph':
        self._nodes[node.name] = node
        for dep in node.depends_on:
            self._edges.append((dep, node.name))
        return self

    def topological_order(self) -> list[list[str]]:
        # Group nodes by level – each level can run in parallel
        # (simplified Kahn's algorithm)
        in_degree = {n: 0 for n in self._nodes}
        for _, child in self._edges:
            in_degree[child] += 1
        levels = []
        while in_degree:
            level = [n for n, d in in_degree.items() if d == 0]
```

```

        if not level:
            raise RuntimeError(f'Cycle detected in {self.name}')
        levels.append(level)
        for n in level:
            del in_degree[n]
            for src, child in self._edges:
                if src == n:
                    in_degree[child] -= 1
        return levels

    async def execute(self, executor: 'GraphExecutor') -> dict[str, Any]:
        return await executor.run(self)

```

9.2 Saga — Atomic Multi-Step Side Effects

An apply is not a single action. It is a saga: (1) fetch job from storage, (2) check policy, (3) tailor resume, (4) tailor cover letter, (5) render PDF, (6) acquire browser, (7) navigate, (8) fill form, (9) answer questions, (10) upload, (11) submit, (12) confirm, (13) persist event. Any step can fail. Partial failure leaves the user in an unknown state on the board's side. The saga pattern requires each forward action to declare a compensating action — what to do if a later step fails.

```

# jobot/orchestrator/saga.py
from __future__ import annotations
from dataclasses import dataclass, field
from typing import Callable, Awaitable, Any
import asyncio, logging

log = logging.getLogger(__name__)

@dataclass
class SagaStep:
    name: str
    forward: Callable[..., Awaitable[Any]]
    compensate: Callable[..., Awaitable[Any]] | None = None
    idempotency_key: str = ''

@dataclass
class SagaState:
    completed: list[SagaStep] = field(default_factory=list)
    failed_at: SagaStep | None = None
    error: Exception | None = None

class Saga:
    def __init__(self, name: str, steps: list[SagaStep]):
        self.name = name
        self._steps = steps
        self._state = SagaState()

    async def run(self, ctx: dict[str, Any]) -> Any:
        for step in self._steps:
            try:
                log.info(f'saga[{self.name}] {step.name} (forward)')
                result = await asyncio.wait_for(
                    step.forward(ctx), timeout=120.0
                )
                ctx[step.name] = result
                self._state.completed.append(step)
            except Exception as e:

```

```

        log.error(f'saga[{self.name}] {step.name} FAILED: {e}')
        self._state.failed_at = step
        self._state.error = e
        await self._compensate(ctx)
        raise
    return ctx

    async def _compensate(self, ctx: dict):
        # Run compensating actions in REVERSE order
        for step in reversed(self._state.completed):
            if step.compensate:
                try:
                    log.info(f'saga[{self.name}] {step.name} (compensate)')
                    await step.compensate(ctx)
                except Exception as ce:
                    log.error(f'saga[{self.name}] {step.name} COMPENSATION FAILED:
{ce}'))

                    # Compensation failure is critical – log to dead-letter queue
                    await self._dead_letter(step, ce, ctx)

    async def _dead_letter(self, step, error, ctx):
        from .quarantine import QuarantineQueue
        q = QuarantineQueue()
        await q.put({
            'saga': self.name,
            'step': step.name,
            'error': str(error),
            'ctx_keys': list(ctx.keys()),
            'ts': __import__('datetime').datetime.utcnow().isoformat(),
        })

```

Example: in the apply saga, the 'submit' step's compensating action is 'withdraw application' (if the board's API supports withdrawal). The 'upload resume' step's compensating action is 'delete uploaded file' (if applicable). The 'fill form' step's compensating action is 'clear form'. Some steps have no compensating action — e.g., 'persist event' is irreversible but harmless (the event log is append-only, so a compensating action would be to add a new 'rolled_back' event).

9.3 Checkpoint & Resume

After each successful step, the saga writes a checkpoint to `data/checkpoints/<saga-id>.json` containing the full ctx dict (with idempotency keys). On restart, the saga loads the checkpoint, skips steps whose idempotency key matches an already-completed entry, and resumes from the next step. This implements the agents.md durable-wait primitive: a crash mid-run does not force a restart from zero.

9.4 Quarantine & Dead-Letter

Per agents.md principle 21: 'Quarantine poison work instead of letting it thrash.' A task that fails N times in a row (default N=3) is moved to a quarantine queue, not retried. The queue is a SQLite table (`quarantined_tasks`) with columns: `id`, `saga_name`, `step_name`, `error`, `attempts`, `last_attempt_ts`, `payload_json`. The user can list, replay, or discard quarantined tasks via `jobot quarantine list / replay <id> / discard <id>`. Silent retry storms destroy reliability and operator trust — explicit quarantine with replay is the disciplined alternative.

9.5 Tracing — Trajectories, Not Just Outcomes

Per `agents.md` principle 22: 'Trace trajectories, not only outcomes.' Every saga step, every LLM call, every browser action, every storage write emits a span. Spans are written to the `spans` SQLite table (Tier 1 storage) and can be exported to OpenTelemetry-compatible backends via `jobot trace export`. The `test_tracing_wired.py` test verifies that spans are emitted for every code path.

```
# jobot/orchestrator/tracing.py
from __future__ import annotations
from typing import Any, AsyncIterator
from contextlib import asynccontextmanager
from dataclasses import dataclass, field
from datetime import datetime
import uuid, asyncio

@dataclass
class Span:
    id: str = field(default_factory=lambda: uuid.uuid4().hex)
    trace_id: str = ''
    parent_id: str | None = None
    name: str = ''
    attributes: dict[str, Any] = field(default_factory=dict)
    start_ts: datetime = field(default_factory=datetime.utcnow)
    end_ts: datetime | None = None
    status: str = 'running' # 'running' | 'ok' | 'error'

@asynccontextmanager
async def span(name: str, **attrs) -> AsyncIterator[Span]:
    s = Span(name=name, attributes=attrs)
    await emit_span_start(s)
    try:
        yield s
        s.status = 'ok'
    except Exception as e:
        s.status = 'error'
        s.attributes['error'] = str(e)
        raise
    finally:
        s.end_ts = datetime.utcnow()
        await emit_span_end(s)

async def emit_span_start(s: Span):
    # Insert into spans table (Tier 1 storage)
    from ..storage.span_repo import insert_span_start
    await insert_span_start(s)

async def emit_span_end(s: Span):
    from ..storage.span_repo import update_span_end
    await update_span_end(s)

# Usage in capability module:
# async with span('llm.call', provider='gemini', model='gemini-2.0-flash'):
#     resp = await router.route(messages)
#     s.attributes['cost_usd'] = resp.cost_usd
```

Chapter 10 — Plugin & Adapter Registry

Two-tier extension system. **Adapters** are first-party integrations with specific job boards — they ship with JoBot core, are tested with JoBot's CI, and live under `jobot/adapters/`. **Plugins** are user-installable capability modules — they ship as separate Python packages, install via `jobot plugin install`, and live anywhere on the user's filesystem. The split exists because adapters must be rock-solid (they touch external systems) while plugins can be experimental.

10.1 Adapter Registry — First-Party Board Integrations

```
# jobot/adapters/base.py
from __future__ import annotations
from abc import ABC, abstractmethod
from pydantic import BaseModel
from typing import AsyncIterator

class ScrapeQuery(BaseModel):
    keywords: str
    location: str = ''
    sites: list[str] = ['linkedin']
    limit: int = 50
    hours_old: int | None = None
    remote_only: bool = False

class JobPosting(BaseModel):
    id: str # uuid4 hex
    board: str
    external_id: str
    title: str
    company: str
    location: str
    url: str
    description: str = ''
    salary_low: int | None = None
    salary_high: int | None = None
    salary_currency: str | None = None
    remote: str = 'unknown' # 'yes' | 'no' | 'hybrid' | 'unknown'
    visa_sponsor: str = 'unknown'

class ApplyRequest(BaseModel):
    job_id: str
    user_id: str
    resume_pdf_path: str
    cover_letter_path: str | None = None
    dry_run: bool = False # if True, do everything except final submit

class ApplyResult(BaseModel):
    success: bool
    applied_at: str # ISO timestamp
    board_application_id: str | None = None
    evidence_path: str | None = None # screenshot path
    cost_usd: float = 0.0
    error: str | None = None

class BoardAdapter(ABC):
    board_name: str
    requires_browser: bool = True # False for API-only adapters

    @abstractmethod
    async def scrape(self, query: ScrapeQuery) -> AsyncIterator[JobPosting]:
        ...

    @abstractmethod
    async def apply(self, req: ApplyRequest) -> ApplyResult:
        ...

    @abstractmethod
    async def health_check(self) -> bool:
        '''Quick check if adapter is functional (selectors valid, etc).'''
```


...

10.2 Adapter Registry Implementation

```
# jobot/adapters/registry.py
from __future__ import annotations
from .base import BoardAdapter
from typing import Type

_REGISTRY: dict[str, Type[BoardAdapter]] = {}

def register_adapter(name: str):
    def deco(cls: Type[BoardAdapter]):
        if name in _REGISTRY:
            raise ValueError(f'Adapter {name} already registered')
        cls.board_name = name
        _REGISTRY[name] = cls
        return cls
    return deco

def get_adapter(name: str) -> BoardAdapter:
    if name not in _REGISTRY:
        raise KeyError(f'No adapter for board {name!r}. Available: {list(_REGISTRY)}')
    return _REGISTRY[name]()

def list_adapters() -> list[str]:
    return sorted(_REGISTRY)

# Importing adapter modules triggers their @register_adapter decorators:
from . import greenhouse, lever, linkedin, indeed, glassdoor, jobspy_adapter # noqa
```

Example adapter registration:

```
# jobot/adapters/greenhouse.py
from .base import BoardAdapter, ScrapeQuery, JobPosting, ApplyRequest, ApplyResult
from .registry import register_adapter
import httpx, asyncio

@register_adapter('greenhouse')
class GreenhouseAdapter(BoardAdapter):
    requires_browser = False # Greenhouse has a public JSON API
    BASE = 'https://boards-api.greenhouse.io/v1/boards'

    def __init__(self):
        self._client = httpx.AsyncClient(timeout=30.0)

    async def scrape(self, query: ScrapeQuery):
        # Greenhouse board = company name (e.g., 'airbnb')
        for company in query.companies or ['airbnb']:
            url = f'{self.BASE}/{company}/jobs?content=true'
            resp = await self._client.get(url)
            resp.raise_for_status()
            for job in resp.json()['jobs']:
                yield JobPosting(
                    id=uuid.uuid4().hex,
                    board='greenhouse',
                    external_id=str(job['id']),
                    title=job['title'],
                    company=company,
```

```

        location=job.get('location', {}).get('name', ''),
        url=job['absolute_url'],
        description=job.get('content', ''),
    )

    async def apply(self, req: ApplyRequest) -> ApplyResult:
        # Greenhouse apply: multipart form POST to /boards/{co}/jobs/{id}
        # (depends on job's specific form fields – fetch form spec first)
        ...

    async def health_check(self) -> bool:
        try:
            resp = await self._client.get(f'{self.BASE}/airbnb/jobs?per_page=1')
            return resp.status_code == 200
        except Exception:
            return False

```

10.3 Plugin Registry — User-Installable Modules

Plugins are Python packages that declare JoBot entry points via `pyproject.toml`. The plugin manifest is a pydantic schema; the install flow validates the manifest, checks the audit hash, and registers the plugin with JoBot's plugin registry. Lifted from career-ops's `plugins-registry/` pattern.

```

# jobot/plugins/manifest.py – pydantic schema for plugin manifests
from pydantic import BaseModel, Field, HttpUrl
from typing import Literal

class PluginManifest(BaseModel):
    name: str = Field(..., regex=r'^[a-z][a-z0-9-]*$')
    version: str = Field(..., regex=r'^\d+\.\d+\.\d+$')
    description: str
    author: str
    license: str
    homepage: HttpUrl
    entry_point: str = Field(..., description='Dotted path to plugin class')
    capabilities: list[Literal[
        'scraper', 'applier', 'resume_tailor', 'cover_letter_generator',
        'interview_prep', 'tracker', 'analytics', 'scheduler', 'other'
    ]]
    config_schema: dict | None = None # JSON schema for plugin config
    audit_hash: str = Field(..., description='SHA-256 of source tarball')
    min_jobot_version: str
    max_jobot_version: str | None = None

# jobot/plugins/installer.py – install / list / audit
from __future__ import annotations
import hashlib, subprocess, json
from pathlib import Path
from .manifest import PluginManifest

PLUGINS_DIR = Path.home() / '.jobot' / 'plugins'

class PluginInstaller:
    async def install(self, source: str) -> PluginManifest:
        # source = pip-installable spec ('git+https://...' or 'name==1.0.0')
        subprocess.check_call(['pip', 'install', '--user', source])
        # Locate entry point jobot_plugin.manifest in installed package
        # Validate manifest schema
        # Compute audit hash, store in plugins/installed.json

```

```

    manifest = self._load_manifest(source)
    self._register(manifest)
    return manifest

async def list_installed(self) -> list[PluginManifest]:
    return self._load_installed()

async def audit(self) -> dict:
    # Re-hash every installed plugin's source tarball; verify match
    # with stored audit_hash. Report drift.
    result = {'ok': [], 'drift': [], 'missing': []}
    for plugin in self._load_installed():
        current_hash = self._compute_hash(plugin)
        if current_hash == plugin.audit_hash:
            result['ok'].append(plugin.name)
        else:
            result['drift'].append({'name': plugin.name,
                                    'expected': plugin.audit_hash,
                                    'actual': current_hash})

    return result

async def uninstall(self, name: str):
    # Remove from installed.json, pip uninstall
    subprocess.check_call(['pip', 'uninstall', '-y', f'jobot-plugin-{name}'])
    self._unregister(name)

```

Chapter 11 — Security, PII & Policy Engine

JoBot operates on sensitive data: the user's resume, cover letters, LLM prompts containing personal information, browser sessions with logged-in credentials to job boards. The security model is defense-in-depth — multiple layers, each independently sufficient to prevent common failures. The policy engine enforces explicit caps and gates side effects.

11.1 PII Masker — Strip Sensitive Data From Traces

Every span attribute, every log line, every error message passes through the PII masker before being persisted. The masker is a regex+LLM hybrid: regex catches email addresses, phone numbers, SSN-like patterns, credit-card patterns; LLM (called once per long text on a slow path) catches subtler PII like full names, home addresses, employer names that should not appear in traces. Masked values are replaced with [PII:email], [PII:phone], etc., so the trace remains debuggable without leaking data.

```
# jobot/security/pii.py
from __future__ import annotations
import re
from typing import Any

REGEX_PATTERNS = {
    'email': re.compile(r'[\w.+~]+@[\w-]+\.[\w.-]+'),
    'phone': re.compile(r'\++?\d{1,4}?[-.\s]?(\d{1,3}?)?[-.\s]?(\d{1,4})[-.\s]?(\d{1,4})[-.\s]?(\d{1,9})'),
    'ssn': re.compile(r'\b\d{3}-\d{2}-\d{4}\b'),
    'credit_card': re.compile(r'\b(?:\d[ -]*){13,19}\b'),
    'ipv4': re.compile(r'\b(?:\d{1,3}\.){3}\d{1,3}\b'),
    'street_addr':
re.compile(r'\d+\s+[A-Z][a-z]+\s+(?:St|Ave|Rd|Dr|Blvd|Ln|Way|Ct)\b'),
}

async def mask_pii(value: Any, llm_assist: bool = False) -> Any:
    if isinstance(value, str):
        for kind, pat in REGEX_PATTERNS.items():
            value = pat.sub(f'[PII:{kind}]', value)
        if llm_assist and len(value) > 200:
            # Slow path: ask LLM to identify subtle PII (full names, etc.)
            from ..llm.router import router as llm_router
            resp = await llm_router.route([
                Message('system', 'Identify PII in the text. Return the text with '
                    'PII replaced by [PII:<kind>]. Kinds: name, address, '
                    'employer, dob, github_username.'),
                Message('user', value),
            ], max_tokens=len(value) // 3)
            value = resp.text
        return value
    elif isinstance(value, dict):
        return {k: await mask_pii(v, llm_assist) for k, v in value.items()}
    elif isinstance(value, list):
        return [await mask_pii(v, llm_assist) for v in value]
    return value
```

11.2 Policy Engine — Caps & Enforcement

The policy engine is a stateful gate. Every side-effecting action calls `policy.check(action, ...)` before executing; the call either returns success or raises `PolicyViolation`. Caps are tracked per-day, per-week, per-month, per-board, and per-cost. The `test_policy*.py` test files validate the enforcement logic.

```
# jobot/policy/engine.py
from __future__ import annotations
from dataclasses import dataclass
from datetime import datetime, timedelta
from .caps import Cap, CapViolation

@dataclass(frozen=True)
class PolicyConfig:
    daily_apply_cap: int = 50
    weekly_apply_cap: int = 200
    daily_cost_cap_usd: float = 5.0
    weekly_cost_cap_usd: float = 25.0
    allowed_boards: tuple[str, ...] = ('linkedin', 'greenhouse', 'lever', 'indeed')
    blocked_companies: tuple[str, ...] = () # user-configured blacklist
    blocked_keywords_in_jd: tuple[str, ...] = () # e.g., 'on-call', 'shift work'
    require_human_approval_above_cost_usd: float = 0.50

class PolicyEngine:
    def __init__(self, cfg: PolicyConfig, cap_store: Cap):
        self._cfg = cfg
        self._caps = cap_store

    async def check_apply(self, user_id: str, job_id: str,
                        board: str, company: str,
                        estimated_cost_usd: float) -> None:
        # Raises PolicyViolation if any cap would be exceeded
        if board not in self._cfg.allowed_boards:
            raise PolicyViolation(f'Board {board!r} not allowed')
        if company.lower() in [c.lower() for c in self._cfg.blocked_companies]:
            raise PolicyViolation(f'Company {company!r} blacklisted')
        if not await self._caps.has_remaining('apply', user_id, 'daily',
                                             self._cfg.daily_apply_cap):
            raise PolicyViolation(f'Daily apply cap ({self._cfg.daily_apply_cap})
reached')
        if not await self._caps.has_remaining('apply', user_id, 'weekly',
                                             self._cfg.weekly_apply_cap):
            raise PolicyViolation(f'Weekly apply cap ({self._cfg.weekly_apply_cap})
reached')
        if not await self._caps.has_remaining('cost', user_id, 'daily',
                                             self._cfg.daily_cost_cap_usd):
            raise PolicyViolation(f'Daily cost cap ({self._cfg.daily_cost_cap_usd})
reached')
        if estimated_cost_usd > self._cfg.require_human_approval_above_cost_usd:
            from .human_loop import request_human_approval
            await request_human_approval(
                f'Apply to {company} via {board} (est cost:
${estimated_cost_usd:.4f})?'
            )

    async def record_apply(self, user_id: str, actual_cost_usd: float):
        await self._caps.increment('apply', user_id, 'daily')
        await self._caps.increment('apply', user_id, 'weekly')
```

```
await self._caps.increment('cost', user_id, 'daily', amount=actual_cost_usd)
await self._caps.increment('cost', user_id, 'weekly', amount=actual_cost_usd)
```

```
class PolicyViolation(Exception): pass
```

11.3 Audit Log of Every Side Effect

Every side effect — every apply submitted, every email sent, every file written outside the workspace — is recorded in the `audit_log` SQLite table with: timestamp, action, target, idempotency key, status (attempted/committed/retried/compensated/skipped), user_id, estimated_cost, actual_cost. The audit log is append-only — no UPDATE or DELETE statements are ever issued against it. This is the policy-engine's source of truth for cap tracking.

11.4 Worked Example — Daily Apply Cap

Policy blocks the 51st application

User has set `daily_apply_cap: 50` in their profile. They queue 60 high-priority jobs to apply overnight. The orchestrator builds 60 `ApplySaga` instances. Saga 1 through 50 complete successfully. Saga 51 calls `policy.check_apply(...)` which calls `caps.has_remaining('apply', user_id, 'daily', 50)` — returns False. Saga 51 raises `PolicyViolation`. The orchestrator moves sagas 51-60 to the 'blocked' queue with reason 'daily apply cap reached'. The user sees this in `jobot tracker list --status blocked` and can either raise the cap or wait until midnight UTC when the scheduler resets daily caps.

11.5 Secret Hygiene

- API keys live in OS keyring, never in `.env`. `.env` holds only non-secret config (log level, default provider name, profile name).
- Board credentials (LinkedIn cookies, etc.) live in OS keyring with per-board keys.
- Fernet master key lives in OS keyring under key 'master.fernet.key'.
- Secrets are never logged. The PII masker treats known secret values as PII and masks them in trace spans.
- Browser profile volumes (cookies, localStorage) are chmod 0700. The OS user owns them; no other user has read access.
- Docker deployments mount the keyring via a secrets volume (Docker secrets or Kubernetes secrets), not environment variables.
- CI never sees production secrets. Integration tests use mock LLM providers (recorded responses) and mock board adapters.

Chapter 12 — Capability Module: Multi-Source Job Scraper

The scraper is JoBot's first contact with the job market. It feeds every downstream module — without fresh postings, the tracker is empty, the applier has nothing to apply to, the dashboard has no data. The scraper must be: (1) multi-source (no single board is sufficient), (2) deduplicated (the same job appears on LinkedIn, Indeed, and the company's own Greenhouse page), (3) resilient (a board going down or changing its HTML must not break the whole system), and (4) cheap (scrapping is high-volume, low-margin — every unnecessary LLM call bleeds money).

12.1 Strategy: Vendor JobSpy + Build Direct-API Adapters

The highest-leverage decision in this module is to vendor `speedyapply/JobSpy` (4,072★, MIT) as a pip dependency. JobSpy is a pure-Python scraper library that already handles LinkedIn, Indeed, Glassdoor, Google, ZipRecruiter, and Bayt — with deduplication, pandas output, and active maintenance. Re-implementing these six scrapers from scratch would take 4-8 engineer-weeks each and would arrive inferior to JobSpy's battle-tested selectors. Instead, JoBot wraps JobSpy in an adapter that respects its own circuit-breaker and dedup primitives.

For boards with public APIs (Greenhouse, Lever, Ashby), JoBot builds direct-API adapters — no scraping needed. These are faster, cheaper, more reliable, and more polite (no rate-limit pressure). The direct-API adapters are the preferred path whenever available; scraping is the fallback.

12.2 File Layout

```
jobot/scrapers/  
├── __init__.py  
├── jobspy_adapter.py # Wraps speedyapply/JobPy  
├── greenhouse.py # Direct JSON API  
├── lever.py # Direct JSON API  
├── ashby.py # Direct JSON API (planned)  
├── career_page.py # Generic career-page scanner (150+ sites)  
├── dedup.py # Vector+hash dedup  
└── README.md
```

12.3 JobSpy Adapter — Full Scaffold

```
# jobot/scrapers/jobspy_adapter.py
from __future__ import annotations
import asyncio, logging
from typing import AsyncIterator
from datetime import datetime
import uuid
from ..adapters.base import ScrapeQuery, JobPosting
from ..adapters.registry import register_adapter
from ..adapters.greenhouse import GreenhouseAdapter # noqa: re-export
from ..resilience.circuit_breaker import CircuitBreaker, CircuitOpenError
from ..storage.dedup import DedupService
from ..tracing.span import span
from jobspy import scrape_jobs as _sync_scrape

log = logging.getLogger(__name__)

SUPPORTED_SITES = ['linkedin', 'indeed', 'glassdoor', 'google',
                   'ziprecruiter', 'bayt']

@register_adapter('jobspy')
class JobSpyAdapter:
    """Wraps the JobSpy library behind JoBot's adapter contract."""
    board_name = 'jobspy'
    requires_browser = False # JobSpy uses httpx + occasional playwright

    def __init__(self, dedup: DedupService | None = None,
                 cb: CircuitBreaker | None = None):
        self._dedup = dedup or DedupService()
        self._cb = cb or CircuitBreaker(name='jobspy',
                                       failure_threshold=5,
                                       recovery_timeout=300)

    async def scrape(self, query: ScrapeQuery) -> AsyncIterator[JobPosting]:
        async with span('scrape.jobspy', keywords=query.keywords,
                       sites=query.sites, limit=query.limit):
            # Honor circuit breaker
            if not self._cb.allow_request():
                raise CircuitOpenError('jobspy circuit open – recent failures')
            # JobPy is sync – run in threadpool to avoid blocking event loop
            try:
                raw = await asyncio.to_thread(
                    _sync_scrape,
                    site_name=query.sites,
                    search_term=query.keywords,
                    location=query.location,
                    results_wanted=query.limit,
                    hours_old=query.hours_old,
                    # JobPy supports proxy + country config
                )
                self._cb.record_success()
            except Exception as e:
                self._cb.record_failure()
                log.error('JobPy scrape failed: %s', e)
                raise
            # Convert pandas DataFrame → JobPosting objects, dedup on the fly
            for _, row in raw.iterrows():
                posting = self._row_to_posting(row)
                if await self._dedup.is_dup(posting):
                    continue
```



```

        await self._dedup.remember(posting)
        yield posting

def _row_to_posting(self, row) -> JobPosting:
    # JobPy columns: site, title, company, location, url, description,
    # job_url_direct, company_url, ... etc.
    salary_low = salary_high = None
    if isinstance(row.get('interval'), str) and isinstance(row.get('min_amount'),
(int, float)):
        # JobPy reports salary as min/max + interval ('yearly'/'hourly')
        salary_low = int(row['min_amount'])
        salary_high = int(row.get('max_amount', row['min_amount']))
    return JobPosting(
        id=uuid.uuid4().hex,
        board=row.get('site', 'unknown'),
        external_id=str(row.get('id', ''))[:64] or row.get('job_url_direct', ''),
        title=row.get('title', ''),
        company=row.get('company', ''),
        location=row.get('location', ''),
        url=row.get('job_url_direct') or row.get('url', ''),
        description=row.get('description', '') or '',
        salary_low=salary_low,
        salary_high=salary_high,
        salary_currency=row.get('currency', 'USD') if salary_low else None,
        remote=self._infer_remote(row),
    )

def _infer_remote(self, row) -> str:
    loc = (row.get('location') or '').lower()
    if 'remote' in loc or 'anywhere' in loc:
        return 'yes'
    if 'hybrid' in loc:
        return 'hybrid'
    if loc:
        return 'no'
    return 'unknown'

async def apply(self, req):
    raise NotImplementedError('JobPy is a scraper, not an applier')

async def health_check(self) -> bool:
    try:
        # Scrape 1 Indeed posting as a smoke test
        raw = await asyncio.to_thread(_sync_scrape,
                                        site_name=['indeed'],
                                        search_term='engineer',
                                        location='United States',
                                        results_wanted=1)
        return len(raw) > 0
    except Exception:
        return False

```

12.4 Career-Page Scanner — Generic 150+ Sites

Many employers use Workday, SuccessFactors, or custom ATS pages not covered by JobSpy. Lifted from [replyre/job-hunter \(150★\)](#): a YAML-driven career-page scanner that takes a list of employer career URLs and extracts job postings via per-ATS-templates. Re-implemented from scratch (job-hunter is unlicensed).

```

# jobot/scrapers/career_page.py
from __future__ import annotations
import asyncio, yaml, httpx
from pathlib import Path
from typing import AsyncIterator
from pydantic import BaseModel
from ..adapters.base import JobPosting
from .dedup import DedupService

class CareerPageConfig(BaseModel):
    employer: str
    ats: str # 'workday' | 'successfactors' | 'greenhouse' | 'lever' | 'custom'
    url: str
    job_link_pattern: str # regex to extract individual job URLs from listing page
    enabled: bool = True

class CareerPageScanner:
    """Generic career-page scanner driven by YAML config.
    Lifted (re-implemented) from replyre/job-hunter."""
    def __init__(self, config_path: Path, dedup: DedupService):
        self._configs = self._load(config_path)
        self._dedup = dedup
        self._client = httpx.AsyncClient(timeout=30.0)

    def _load(self, path: Path) -> list[CareerPageConfig]:
        with path.open() as f:
            data = yaml.safe_load(f)
            return [CareerPageConfig(**c) for c in data['sites'] if c.get('enabled',
True)]

    async def scan(self) -> AsyncIterator[JobPosting]:
        for cfg in self._configs:
            try:
                async for posting in self._scan_one(cfg):
                    if not await self._dedup.is_dup(posting):
                        await self._dedup.remember(posting)
                        yield posting
            except Exception as e:
                # Don't let one bad site kill the whole scan
                logging.getLogger(__name__).warning(
                    f'Career page scan failed for {cfg.employer}: {e}')

    async def _scan_one(self, cfg: CareerPageConfig) -> AsyncIterator[JobPosting]:
        resp = await self._client.get(cfg.url)
        resp.raise_for_status()
        # Extract job URLs via regex pattern
        import re, uuid
        for match in re.finditer(cfg.job_link_pattern, resp.text):
            job_url = match.group(0) if match.groups() == () else match.group(1)
            # Fetch individual job page (rate-limited)
            await asyncio.sleep(1.0)
            job_resp = await self._client.get(job_url)
            if job_resp.status_code != 200:
                continue
            # Parse JD based on ATS type
            jd_text, title = self._parse_job_page(job_resp.text, cfg.ats)
            yield JobPosting(
                id=uuid.uuid4().hex,
                board=f'career_page_{cfg.ats}',
                external_id=job_url,
                title=title,

```

```

        company=cfg.employer,
        location='unknown',
        url=job_url,
        description=jd_text,
    )

def _parse_job_page(self, html: str, ats: str) -> tuple[str, str]:
    from bs4 import BeautifulSoup
    soup = BeautifulSoup(html, 'html.parser')
    # ATS-specific selectors
    if ats == 'workday':
        title = soup.select_one('[data-automation-id="jobTitle"]')
        desc = soup.select_one('[data-automation-id="jobPostingInstruction"]')
    elif ats == 'greenhouse':
        title = soup.find('h1')
        desc = soup.find('div', id='content')
    else:
        title = soup.find('h1')
        desc = soup.find('main') or soup.find('body')
    return (desc.get_text('\n', strip=True) if desc else '',
            title.get_text(strip=True) if title else 'Untitled')

```

12.5 Dedup Service — Hash + Vector

Two-tier dedup: (1) fast hash-based dedup on `(board, external_id)` catches exact reposts within seconds; (2) slow vector-based dedup on the JD embedding catches near-duplicates where the company tweaked the salary or location. The existing `test_dedup.py` test covers both paths.

Chapter 13 — Capability Module: Resume Tailoring

The resume tailoring module produces a JD-specific PDF resume from the user's base profile. It is one of JoBot's most LLM-intensive modules — every application requires a unique tailored resume. The design lifts the **drafter→reviewer two-agent loop** from MadsLorentzen/ai-job-search (31,411★) and the **LaTeX rendering pipeline** (lualatex + xelatex + pdftotext ATS check). The A-F rubric scoring from career-ops provides the reviewer's evaluation criteria.

13.1 Two-Agent Loop — Drafter & Reviewer

The drafter is an LLM that generates a tailored CV from the base profile + JD. The reviewer is a second LLM call that critiques the draft against an ATS-rubric and returns a letter grade (A-F) plus specific actionable feedback. The drafter iterates with the feedback; the loop terminates when the score reaches a configurable threshold (default B+ / 3.5) or after a configurable max iterations (default 3). This is lifted almost verbatim from MadsLorentzen's design — it is the single best architectural pattern for high-quality LLM generation.

```
# jobot/resume/drafter.py
from __future__ import annotations
from dataclasses import dataclass
from pydantic import BaseModel, Field
from ..llm.router import ModelRouter
from ..llm.base import Message
from ..storage.profile import UserProfile

class TailoredResume(BaseModel):
    summary: str = Field(..., description='2-3 sentence professional summary')
    skills: list[str]
    experience: list[dict] = Field(..., description='List of experience bullets per role')
    education: list[dict]
    certifications: list[str] = []
    raw_latex: str = '' # Filled by LaTeX renderer
    ats_score: float | None = None # Filled by ATS check

DRAFTER_SYSTEM_PROMPT = '''You are an expert resume writer.
Given a user's base profile and a target job description, produce a
tailored resume that maximizes ATS keyword match while remaining truthful.
Return ONLY a JSON object with keys: summary, skills, experience,
education, certifications. Do not invent experience or skills the user
does not have – instead emphasize existing ones most relevant to the JD.'''

class Drafter:
    def __init__(self, router: ModelRouter, model: str | None = None):
        self._router = router
        self._model = model # None = use router default

    async def draft(self, profile: UserProfile, jd_text: str,
                    feedback: str = '') -> TailoredResume:
        user_msg = (
            f'# Base Profile\n{profile.to_markdown()}\n\n'
            f'# Target Job Description\n{jd_text[:6000]}\n\n'
            + (f'# Reviewer Feedback (previous draft)\n{feedback}\n\n'
              if feedback else '')
```

```

        + 'Generate the tailored resume as JSON.'
    )
    resp = await self._router.route([
        Message('system', DRAFTER_SYSTEM_PROMPT),
        Message('user', user_msg),
    ], model=self._model, temperature=0.6, max_tokens=2048)
    # Parse JSON – be defensive
    import json, re
    m = re.search(r'\{.*\}', resp.text, re.DOTALL)
    if not m:
        raise ValueError(f'No JSON in drafter response: {resp.text[:200]}')
    data = json.loads(m.group(0))
    return TailoredResume(**data)

# jobot/resume/reviewer.py
from __future__ import annotations
from pydantic import BaseModel, Field
from ..llm.router import ModelRouter
from ..llm.base import Message

class ReviewResult(BaseModel):
    grade: str = Field(..., regex=r'^[A-F][+-]?$',) # A+, A, A-, ..., F
    score: float = Field(..., ge=0.0, le=5.0) # 0.0 = F, 5.0 = A+
    ats_keyword_coverage: float = Field(..., ge=0.0, le=1.0)
    truthfulness: float = Field(..., ge=0.0, le=1.0) # did drafter invent?
    feedback: str
    actionable_improvements: list[str]

REVIEWER_SYSTEM_PROMPT = '''You are a strict ATS resume reviewer.
Grade the resume against the job description on a 0.0-5.0 scale.
Criteria:
- ATS keyword coverage: how many JD keywords appear in the resume?
- Truthfulness: did the drafter invent experience/skills not in the base profile?
- Clarity & concision: are bullets crisp and impact-oriented?
- Relevance: is irrelevant experience de-emphasized?
Return JSON: {grade, score, ats_keyword_coverage, truthfulness,
feedback, actionable_improvements}. Be harsh. Better to flag a weak
resume for revision than to wave through a mediocre one.'''

class Reviewer:
    def __init__(self, router: ModelRouter, model: str | None = None):
        self._router = router
        self._model = model

    async def review(self, resume, profile, jd_text) -> ReviewResult:
        resp = await self._router.route([
            Message('system', REVIEWER_SYSTEM_PROMPT),
            Message('user',
                f'# Resume draft\n{resume.model_dump_json(indent=2)}\n\n'
                f'# Base profile\n{profile.to_markdown()}\n\n'
                f'# Job description\n{jd_text[:4000]}\n\n'
                'Grade and provide actionable improvements.'),
        ], model=self._model, temperature=0.3, max_tokens=1500)
        import json, re
        m = re.search(r'\{.*\}', resp.text, re.DOTALL)
        return ReviewResult(**json.loads(m.group(0)))

```

13.2 Drafter-Reviewer Loop Orchestrator

```
# jobot/resume/loop.py
from __future__ import annotations
from dataclasses import dataclass
from .drafter import Drafter, TailoredResume
from .reviewer import Reviewer, ReviewResult
from .latex import render_latex, ats_check_pdf
from ..storage.profile import UserProfile
from ..tracing.span import span

@dataclass(frozen=True)
class TailorConfig:
    max_iterations: int = 3
    target_score: float = 3.5 # B+ / 3.5 out of 5
    ats_target: float = 0.85 # 85% keyword coverage

class TailorLoop:
    def __init__(self, drafter: Drafter, reviewer: Reviewer,
                 cfg: TailorConfig = TailorConfig()):
        self._drafter = drafter
        self._reviewer = reviewer
        self._cfg = cfg

    async def tailor(self, profile: UserProfile, jd_text: str,
                    template: str = 'default') -> tuple[TailoredResume,
ReviewResult]:
        feedback = ''
        best_resume = None
        best_review = None
        for i in range(self._cfg.max_iterations):
            async with span('resume.iteration', iteration=i + 1,
                           max=self._cfg.max_iterations):
                resume = await self._drafter.draft(profile, jd_text, feedback)
                review = await self._reviewer.review(resume, profile, jd_text)
                # Render LaTeX + ATS check
                pdf_bytes, ats_score = await self._render_and_check(resume, template)
                resume.ats_score = ats_score
                best_resume, best_review = resume, review
                if review.score >= self._cfg.target_score and ats_score >=
self._cfg.ats_target:
                    break
            feedback = review.feedback + '\n\nSpecific improvements:\n- ' + \
                '\n- '.join(review.actionable_improvements)
        return best_resume, best_review

    async def _render_and_check(self, resume, template):
        latex = await render_latex(resume, template)
        resume.raw_latex = latex
        pdf_bytes = await self._compile_latex(latex)
        ats_score = await ats_check_pdf(pdf_bytes, resume)
        return pdf_bytes, ats_score

    async def _compile_latex(self, latex: str) -> bytes:
        import subprocess, tempfile, os
        with tempfile.TemporaryDirectory() as td:
            tex_path = os.path.join(td, 'resume.tex')
            with open(tex_path, 'w') as f:
                f.write(latex)
            # Try lualatex first (best Unicode support), fall back to xelatex
            for engine in ('lualatex', 'xelatex', 'pdflatex'):
```

```

        try:
            subprocess.run([engine, '-interaction=nonstopmode',
                            '-output-directory', td, tex_path],
                            check=True, capture_output=True, timeout=30)
            with open(os.path.join(td, 'resume.pdf'), 'rb') as f:
                return f.read()
        except (subprocess.CalledProcessError, FileNotFoundError):
            continue
    raise RuntimeError('No LaTeX engine available – install texlive or
tectonic')

```

13.3 LaTeX Template System

JoBot ships with three LaTeX templates by default: **default** (clean single-column, ATS-safe), **modern** (two-column with subtle accent color, ATS-safe with caveat), and **classic** (traditional serif, fully ATS-safe). Each template is a Jinja2 file with placeholders for summary, skills, experience, education. Templates live under `jobot/resume/templates/`.

```

% jobot/resume/templates/default.tex.j2
\documentclass[11pt,a4paper]{article}
\usepackage[utf8]{inputenc}
\usepackage[margin=0.5in]{geometry}
\usepackage{titlesec}
\usepackage{enumitem}
\usepackage{hyperref}
\hypersetup{colorlinks=true, urlcolor=blue}

\titleformat{\section}{\large\bfseries}{\thesection}{1em}{}
\titlespacing*{\section}{0pt}{6pt}{4pt}

\begin{document}
\pagestyle{empty}

{\Large \bfseries {{ profile.name }}}\\
{{ profile.email }} $| $ {{ profile.phone }} $| $ {{ profile.location }}\\
{\small {{ profile.linkedin_url }}{% if profile.github_url %} $| $ {{ profile.github_url }}{% endif %} }}

\vspace{6pt}
\hrule
\vspace{6pt}

\section*{Summary}
{{ resume.summary }}

\section*{Skills}
\begin{itemize}[leftmargin=0pt, itemsep=2pt]
{% for skill_chunk in resume.skills | slice(3) %}
    \item {{ skill_chunk | join(', ') }}
{% endfor %}
\end{itemize}

\section*{Experience}
{% for exp in resume.experience %}
    \textbf{\large {{ exp.title }}} at {{ exp.company }} ({{ exp.dates }})\
    \begin{itemize}[leftmargin=12pt, itemsep=1pt]
        {% for bullet in exp.bullets %}
            \item {{ bullet }}

```

```

    {% endfor %}
    \end{itemize}
    \vspace{4pt}
{% endfor %}

\section*{Education}
{% for edu in resume.education %}
    \textbf{\large {{ edu.degree }}} – {{ edu.school }} ({{ edu.year }})
{% endfor %}

{% if resume.certifications %}
\section*{Certifications}
\begin{itemize}[leftmargin=12pt]
{% for cert in resume.certifications %}
    \item {{ cert }}
{% endfor %}
\end{itemize}
{% endif %}

\end{document}

```

13.4 ATS Parseability Check

A tailored resume that defeats ATS parsing is worthless. After compilation, JoBot runs `pdftotext` (from `poppler-utils`) on the PDF and verifies: (1) text extracts successfully, (2) key fields (name, email, skills, experience titles) appear in the extracted text, (3) the LLM-generated summary contains $\geq 80\%$ of the JD's top-10 keywords. If any check fails, the loop iterates again with the failure as feedback.

```

# jobot/resume/ats_check.py
from __future__ import annotations
import subprocess, re, asyncio
from collections import Counter

async def ats_check_pdf(pdf_bytes: bytes, resume, jd_text: str = '') -> float:
    """Returns ATS score in [0.0, 1.0]."""
    # Extract text via pdftotext (poppler-utils)
    proc = await asyncio.create_subprocess_exec(
        'pdftotext', '-', '-', '-q',
        stdin=asyncio.subprocess.PIPE,
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE,
    )
    out, err = await proc.communicate(pdf_bytes)
    if proc.returncode != 0:
        return 0.0
    text = out.decode('utf-8', errors='replace').lower()
    # Check 1: did text extract?
    if len(text) < 100:
        return 0.0
    # Check 2: key fields present
    required = ['summary', 'skills', 'experience']
    section_score = sum(1 for s in required if s in text) / len(required)
    # Check 3: keyword coverage from JD (if provided)
    if jd_text:
        # Top-10 keywords from JD = most-common non-stopword nouns
        from ..skill_extraction import extract_keywords
        jd_keywords = await extract_keywords(jd_text, top_k=10)
        present = sum(1 for kw in jd_keywords if kw.lower() in text)

```



```
        kw_score = present / len(jd_keywords) if jd_keywords else 1.0
    else:
        kw_score = 1.0
    # Weighted: 0.3 sections + 0.7 keyword coverage
    return 0.3 * section_score + 0.7 * kw_score
```

Chapter 14 — Capability Module: Cover-Letter Generator

The cover-letter generator is a single-pass LLM call with tone presets and length caps. It is simpler than the resume tailoring module because cover letters are less structured — there is no ATS parsing constraint, no LaTeX rendering, no iterative reviewer loop. The architectural lift is from `career-ops/generate-cover-letter.mjs`, adapted to JoBot's ModelRouter.

```
# jobot/coverletter/generator.py
from __future__ import annotations
from pydantic import BaseModel, Field
from typing import Literal
from ..llm.router import ModelRouter
from ..llm.base import Message
from ..storage.profile import UserProfile

Tone = Literal['classic', 'narrative', 'technical', 'brief', 'enthusiastic']

class CoverLetterRequest(BaseModel):
    profile: UserProfile
    job_title: str
    company: str
    jd_text: str
    tone: Tone = 'classic'
    max_words: int = 300
    hiring_manager_name: str | None = None # if known

class CoverLetter(BaseModel):
    text: str
    tone: Tone
    word_count: int
    estimated_read_time_s: int

TONES = {
    'classic': 'professional, conventional, third-person perspective',
    'narrative': 'story-driven, opens with a specific anecdote, first-person',
    'technical': 'engineering-focused, mentions specific tools and projects',
    'brief': 'concise (150 words max), no fluff, every sentence impactful',
    'enthusiastic': 'high-energy, demonstrates genuine excitement about the company mission',
}

class CoverLetterGenerator:
    def __init__(self, router: ModelRouter, model: str | None = None):
        self._router = router
        self._model = model

    async def generate(self, req: CoverLetterRequest) -> CoverLetter:
        system = (
            'You are an expert cover-letter writer. Tone: '
            f'{TONES[req.tone]}. Max {req.max_words} words. '
            'Do not invent experience or skills the user does not have. '
            'Output only the cover letter text – no preamble, no "Dear Hiring '
            'Manager" if name unknown (open with the first paragraph).'
        )
        user = (
            f'# Candidate profile\n{req.profile.to_markdown()}\n\n'

```

```

        f'# Position: {req.job_title} at {req.company}\n'
        f'# Job description\n{req.jd_text[:4000]}\n'
        + (f'\n# Hiring manager\n{req.hiring_manager_name}'
           if req.hiring_manager_name else '')
    )
    resp = await self._router.route([
        Message('system', system),
        Message('user', user),
    ], model=self._model, temperature=0.7, max_tokens=600)
    text = resp.text.strip()
    # Enforce word cap as safety net
    words = text.split()
    if len(words) > req.max_words * 1.1:
        text = ' '.join(words[:req.max_words]) + '...'
    return CoverLetter(
        text=text, tone=req.tone, word_count=len(words),
        estimated_read_time_s=max(30, len(words) * 0.3),
    )

```

Chapter 15 — Capability Module: Auto-Apply Engine

The auto-apply engine is JoBot's most consequential module — every apply is an external side effect on the user's behalf, on a board the user does not control, with potentially career-affecting consequences. The design is correspondingly conservative: saga pattern, idempotency keys, compensating actions, preview-before-commit, human-in-the-loop checkpoint, evidence capture. The question-answering algorithm is lifted (re-implemented) from `feder-cr/AIHawk` (30,159★); the LinkedIn Easy Apply flow is lifted (re-implemented on Patchright) from `AIHawk + GodsScion/Auto_job_applier_linkedIn` (2,688★).

15.1 Apply Saga — The Multi-Step Transaction

An apply is decomposed into 13 saga steps. Each forward step has a compensating action; partial failure triggers compensation in reverse order.

#	Step	Compensating action
1	<code>fetch_job_from_storage(job_id)</code>	— (read-only, no-op)
2	<code>check_policy(user, action=APPLY)</code>	— (gate, no-op)
3	<code>tailor_resume(profile, jd, llm)</code>	delete resume PDF from blob store
4	<code>tailor_cover_letter(profile, jd, llm)</code>	delete cover letter from blob store
5	<code>render_latex_pdf(tailored_resume)</code>	delete PDF file
6	<code>acquire_browser_context(board)</code>	release browser context
7	<code>navigate_to_job_url(page, job.url)</code>	— (stateless)
8	<code>click_easy_apply_button(page)</code>	— (modal closes on its own)
9	<code>fill_form_fields(page, profile)</code>	clear form fields
10	<code>answer_custom_questions(page, llm)</code>	— (form fields cleared in step 9)
11	<code>upload_resume_file(page, pdf_path)</code>	— (cannot delete uploaded file reliably)
12	<code>submit_application(page)</code>	<code>withdraw_application(board, application_id)</code> if board API supports
13	<code>persist_event(APPLIED, job_id, ts)</code>	insert <code>ROLLED_BACK</code> event (audit log is append-only)

15.2 LinkedIn Easy Apply on Patchright

Lifted from `AIHawk` (single-script monolith, Selenium) and `GodsScion` (Selenium, LinkedIn-specific resilience). Re-implemented on Patchright (stealth Playwright) using JoBot's named-actions pattern. The flow assumes the user has logged in to LinkedIn manually once (via the persistent browser context), after which Patchright reuses the session.

```
# jobot/apply/linkedin_easy_apply.py
from __future__ import annotations
import asyncio, logging
from ..adapters.base import ApplyRequest, ApplyResult
from ..browser.pool import BrowserPool
from ..browser.named_actions import click_easy_apply_button, fill_text_input,
```

```

capture_evidence
from ..browser.captcha import HumanFallbackSolver
from ..question_answerer import QuestionAnswerer
from ..tracing.span import span

log = logging.getLogger(__name__)

class LinkedInEasyApplySaga:
    def __init__(self, browser: BrowserPool,
                  question_answerer: QuestionAnswerer,
                  captcha_solver=HumanFallbackSolver()):
        self._browser = browser
        self._qa = question_answerer
        self._captcha = captcha_solver

    async def execute(self, req: ApplyRequest) -> ApplyResult:
        async with span('apply.linkedin_easy_apply', job_id=req.job_id,
                        dry_run=req.dry_run):
            ctx = await self._browser.acquire('linkedin')
            page = await ctx.new_page()
            try:
                await page.goto(req.job_url, wait_until='networkidle')
                # Detect & solve captcha if present
                if await self._captcha_present(page):
                    ok = await self._captcha.solve(page, 'linkedin_check')
                    if not ok:
                        return ApplyResult(success=False, error='captcha_unsolvable')
                # Click Easy Apply button
                clicked = await click_easy_apply_button(page)
                if not clicked:
                    return ApplyResult(success=False,
                                        error='easy_apply_button_not_found')
                # Wait for modal
                await page.wait_for_selector('div.jobs-easy-apply-modal',
timeout=10000)
                # Iterate through modal pages (multi-step form)
                step = 0
                while True:
                    step += 1
                    await self._fill_modal_page(page, req)
                    # Click 'Next' or 'Submit'
                    next_btn = await page.query_selector('button[aria-label="Continue
to next step"]')
                    submit_btn = await page.query_selector('button[aria-label="Submit
your application"]')
                    if submit_btn:
                        await capture_evidence(page, f'pre-submit-step{step}')
                        if req.dry_run:
                            log.info('dry_run - would submit now')
                            return ApplyResult(success=True, applied_at='',
                                                evidence_path=f'pre-submit-step{step}'
)
                        await submit_btn.click()
                        break
                    elif next_btn:
                        await next_btn.click()
                        await asyncio.sleep(0.5)
                    else:
                        return ApplyResult(success=False,
                                            error='neither_next_nor_submit_found')
                # Confirm submission

```

```

        await page.wait_for_selector('text=Application sent', timeout=10000)
        await capture_evidence(page, 'post-submit')
        return ApplyResult(success=True,
                            applied_at=__import__('datetime').datetime.utcnow(
).isoformat(),
                            evidence_path='post-submit')

    finally:
        await page.close()
        await self._browser.release(ctx)

async def _fill_modal_page(self, page, req):
    """Detect form fields on the current modal page and fill them."""
    # Find all input/select/textarea elements inside the modal
    modal = await page.query_selector('div.jobs-easy-apply-modal')
    if not modal:
        return
    inputs = await modal.query_selector_all('input, select, textarea')
    for input_el in inputs:
        input_type = await input_el.get_attribute('type') or 'text'
        label = await self._get_input_label(input_el)
        if not label:
            continue
        # Use the question-answerer to get a value
        value = await self._qa.answer(label, '', req.profile, req.job_description)
        if value:
            if input_type in ('text', 'tel', 'email', 'url'):
                await input_el.fill(value)
            elif input_type == 'select':
                await input_el.select_option(value)
            # Skip checkbox/radio – usually EE0 questions, handled separately

async def _get_input_label(self, input_el):
    # Try aria-label, then associated label element, then placeholder
    for attr in ('aria-label', 'placeholder'):
        v = await input_el.get_attribute(attr)
        if v:
            return v
    # Walk to nearby <label>
    # ... (simplified – production version handles complex form layouts)
    return None

async def _captcha_present(self, page) -> bool:
    return bool(await page.query_selector('iframe[src*="recaptcha"]',
    iframe[src*="funcaptcha"]'))

```

15.3 Question-Answering Module (Lifted from AIHawk)

AIHawk's strongest asset is its question-answering module — given a form field label, it produces a contextually appropriate answer from the user's profile + JD. Re-implemented behind JoBot's ModelRouter (AIHawk is OpenAI-locked). The LLM call is single-pass with strict JSON output schema; answers are cached per (label, profile_hash) to avoid redundant LLM calls across similar forms.

```

# jobot/apply/question_answerer.py
from __future__ import annotations
import json, re, hashlib
from pydantic import BaseModel, Field
from typing import Literal
from ..llm.router import ModelRouter

```

```

from ..llm.base import Message
from ..storage.profile import UserProfile

QA_CACHE: dict[str, str] = {} # in-memory; persist to SQLite in production

class QAAnswer(BaseModel):
    value: str
    confidence: float = Field(..., ge=0.0, le=1.0)
    reasoning: str = ''

class QuestionAnswerer:
    """Given a form question, produce a contextually appropriate answer."""
    def __init__(self, router: ModelRouter, model: str | None = None):
        self._router = router
        self._model = model

    async def answer(self, question: str, question_type: str,
                     profile: UserProfile, jd_text: str) -> str:
        # Check cache
        cache_key = self._cache_key(question, profile)
        if cache_key in QA_CACHE:
            return QA_CACHE[cache_key]
        # LLM call
        system = (
            'You are filling out a job application form on behalf of a candidate. '
            'Given a form question, return a SHORT appropriate answer (1-3 words for '
            'single-line fields, 1-2 sentences for free-text). Never invent '
            'experience. '
            'For yes/no questions, answer yes or no. For multiple choice, pick the '
            'closest. '
            'For salary expectations, leave blank if unknown. Return JSON: '
            '{"value": "...", "confidence": 0.0-1.0, "reasoning": "..."}'
        )
        user = (
            f'# Question\n{question} (type: {question_type or "text"})\n\n'
            f'# Candidate profile\n{profile.to_markdown()}\n\n'
            f'# Job description (context)\n{jd_text[:2000]}'
        )
        resp = await self._router.route([
            Message('system', system),
            Message('user', user),
        ], model=self._model, temperature=0.2, max_tokens=200)
        # Parse JSON defensively
        m = re.search(r'\{.*\}', resp.text, re.DOTALL)
        if not m:
            return ''
        try:
            ans = QAAnswer(**json.loads(m.group(0)))
            if ans.confidence >= 0.6:
                QA_CACHE[cache_key] = ans.value
                return ans.value
        except Exception:
            pass
        return ''

    def _cache_key(self, question: str, profile: UserProfile) -> str:
        h = hashlib.sha256()
        h.update(question.lower().encode())
        h.update(profile.identity_hash.encode())
        return h.hexdigest()[:16]

```

15.4 Direct-API Adapters (Greenhouse, Lever)

Greenhouse and Lever expose public JSON APIs for both scraping and applying. The apply endpoints accept multipart form POSTs with structured fields — no browser needed. These adapters are faster, cheaper, and more reliable than browser-based apply. JoBot's existing `GreenhouseAdapter` covers scraping; the apply half is implemented here.

```
# jobot/apply/greenhouse_apply.py
from __future__ import annotations
import httpx, json
from pathlib import Path
from ..adapters.base import ApplyRequest, ApplyResult

class GreenhouseApplyAdapter:
    """Apply via Greenhouse's public JSON API – no browser needed."""
    BASE = 'https://boards-api.greenhouse.io/v1/boards'

    async def apply(self, req: ApplyRequest,
                    board_token: str, # company name on Greenhouse
                    job_id: str) -> ApplyResult:
        if req.dry_run:
            return ApplyResult(success=True, applied_at='')
        # Step 1: fetch the form spec for this job
        async with httpx.AsyncClient(timeout=30.0) as client:
            spec_resp = await client.get(f'{self.BASE}/{board_token}/jobs/{job_id}')
            spec_resp.raise_for_status()
            spec = spec_resp.json()
            # Step 2: build the multipart form
            files = {'resume': (Path(req.resume_pdf_path).name,
                                open(req.resume_pdf_path, 'rb'),
                                'application/pdf')}
            if req.cover_letter_path:
                files['cover_letter'] = (Path(req.cover_letter_path).name,
                                         open(req.cover_letter_path, 'rb'),
                                         'application/pdf')
            data = {'first_name': req.profile.first_name,
                    'last_name': req.profile.last_name,
                    'email': req.profile.email,
                    'phone': req.profile.phone}
            # Add custom questions from spec
            for q in spec.get('questions', []):
                # Use QuestionAnswerer to fill custom fields
                value = await self._answer(q, req)
                if value:
                    data[q['key']] = value
            # Step 3: submit
            post_resp = await client.post(
                f'{self.BASE}/{board_token}/jobs/{job_id}',
                data=data, files=files,
            )
            if post_resp.status_code in (200, 201):
                return ApplyResult(success=True,
                                   applied_at=__import__('datetime').datetime.utcnow().isoformat(),
                                   board_application_id=post_resp.json().get('id'))
            return ApplyResult(success=False, error=post_resp.text)

    async def _answer(self, question_spec, req):
        # Delegate to QuestionAnswerer
```



```
from .question_answerer import QuestionAnswerer
from ..llm.router import ModelRouter
qa = QuestionAnswerer(ModelRouter.from_config())
return await qa.answer(question_spec.get('label', ''),
                        question_spec.get('type', 'text'),
                        req.profile, req.job_description)
```

15.5 Human-in-the-Loop Checkpoint

Every `apply` calls `policy.check_apply(...)` which may invoke `request_human_approval(...)` if the estimated cost exceeds the configurable threshold (`require_human_approval_above_cost_usd`, default \$0.50). In interactive mode, the user gets a yes/no prompt with full context (company, role, estimated cost, model used). In unattended mode (`--yes`), the `apply` proceeds but the audit log records that the human-approval gate was bypassed — useful for forensics if a mis-application occurs.

Chapter 16 — Capability Module: Interview Prep

Interview prep is JoBot's most undervalued capability — none of the Tier-2 source repos treat it as first-class. Only `career-ops` and `MadsLorentzen/ai-job-search` ship interview-prep modules. JoBot's interview-prep module lifts the question-bank + STAR-method coach design from `jennifer88huang/interview-skills` (272★) (FAANG focus) and the structured-practice flow from `career-ops/interview-prep/`. Re-implemented as a JoBot task-graph subgraph driven by the ModelRouter.

16.1 Module File Layout

```
jobot/interview/  
├── __init__.py  
├── question_bank.py # Curated question sets (FAANG, behavioral, system design,  
technical)  
├── mock_session.py # Multi-turn mock interview  
├── star_coach.py # STAR-method answer coach  
└── data/  
    ├── faang_behavioral.yaml # 100+ FAANG behavioral questions  
    ├── system_design.yaml # 50+ system-design prompts  
    └── technical_<role>.yaml # role-specific technical questions
```

16.2 Mock Interview Session

```
# jobot/interview/mock_session.py
from __future__ import annotations
from pydantic import BaseModel, Field
from typing import Literal
from ..llm.router import ModelRouter
from ..llm.base import Message
from ..storage.profile import UserProfile

InterviewType = Literal['behavioral', 'system_design', 'technical', 'mixed']

class MockSessionConfig(BaseModel):
    profile: UserProfile
    job_title: str
    company: str
    jd_text: str
    interview_type: InterviewType = 'mixed'
    num_questions: int = 5
    difficulty: Literal['easy', 'medium', 'hard'] = 'medium'

class MockTurn(BaseModel):
    question: str
    candidate_answer: str
    score: float = Field(..., ge=0.0, le=1.0)
    feedback: str
    improvement_suggestions: list[str]

INTERVIEWER_SYSTEM_PROMPT = '''You are a {difficulty} {interview_type} interviewer at
{company}.
Your goal is to assess the candidate's fit for {job_title}. Ask probing but fair
questions.
After the candidate answers, score the response on clarity, depth, and relevance.
Provide
specific, actionable feedback. Stay in character – do not reveal you are an LLM.'''

class MockSession:
    def __init__(self, router: ModelRouter, model: str | None = None):
        self._router = router
        self._model = model
        self._history: list[Message] = []

    async def start(self, cfg: MockSessionConfig) -> str:
        system_prompt = INTERVIEWER_SYSTEM_PROMPT.format(
            difficulty=cfg.difficulty,
            interview_type=cfg.interview_type,
            company=cfg.company,
            job_title=cfg.job_title,
        )
        self._history = [Message('system', system_prompt),
                        Message('user', f'Candidate
profile:\n{cfg.profile.to_markdown()}\n\n'
                                f'Job
description:\n{cfg.jd_text[:2000]}\n\n'
                                f'Please start the interview with your first
question.')]
        resp = await self._router.route(self._history, model=self._model,
                                       temperature=0.7, max_tokens=400)
        self._history.append(Message('assistant', resp.text))
        return resp.text
```

```

async def respond(self, candidate_answer: str) -> str:
    self._history.append(Message('user', candidate_answer))
    resp = await self._router.route(self._history, model=self._model,
                                    temperature=0.7, max_tokens=400)
    self._history.append(Message('assistant', resp.text))
    return resp.text

async def evaluate(self) -> list[MockTurn]:
    """At end of session, ask the LLM to evaluate each turn."""
    eval_prompt = (
        'Review the entire interview above. For each question you asked and each
candidate '
        'answer, output a JSON array of objects: {question, candidate_answer,
score (0-1), '
        'feedback, improvement_suggestions}. Be honest – a candidate who hears
only praise '
        'cannot improve.'
    )
    self._history.append(Message('user', eval_prompt))
    resp = await self._router.route(self._history, model=self._model,
                                    temperature=0.3, max_tokens=2000)

    # Parse JSON array
    import json, re
    m = re.search(r'\[.*\]', resp.text, re.DOTALL)
    if m:
        return [MockTurn(**t) for t in json.loads(m.group(0))]
    return []

```

16.3 STAR-Method Coach

The STAR coach takes a candidate's draft answer to a behavioral question and rewrites it in Situation-Task-Action-Result format. Inputs: the question, the candidate's draft, the user's profile (for relevant past experiences to draw on). Outputs: a restructured answer + critique. Useful before submitting take-home written interviews or as a pre-practice tool.

```

# jobot/interview/star_coach.py
from __future__ import annotations
from pydantic import BaseModel
from ..llm.router import ModelRouter
from ..llm.base import Message

class STARCoaching(BaseModel):
    situation: str
    task: str
    action: str
    result: str
    feedback: str
    strengths: list[str]
    weaknesses: list[str]

STAR_PROMPT = '''You are an interview coach specializing in the STAR method
(Situation, Task, Action, Result). Given a behavioral question and the
candidate's draft answer, restructure it as a clear STAR response.
Identify strengths and weaknesses. Be specific and actionable.
Return JSON: {situation, task, action, result, feedback, strengths[],
weaknesses[]}.'''

class STARCoach:

```

```

def __init__(self, router: ModelRouter, model: str | None = None):
    self._router = router
    self._model = model

    async def coach(self, question: str, draft_answer: str) -> STARCoaching:
        resp = await self._router.route([
            Message('system', STAR_PROMPT),
            Message('user', f'# Question\n{question}\n\n# Draft
answer\n{draft_answer}'),
        ], model=self._model, temperature=0.4, max_tokens=800)
        import json, re
        m = re.search(r'\{.*\}', resp.text, re.DOTALL)
        return STARCoaching(**json.loads(m.group(0)))

```

Chapter 17 — Capability Module: Application Tracking & Dashboard

JoBot already has a 12-phase Application State Protocol (ASP). What's missing is the user-facing tracker dashboard, weekly digest, and analytics. The lift is from `career-ops` (`build-dashboard.mjs`, `weekly-digest.mjs`, `stats.mjs`, `funnel-velocity.mjs`, `rejection-latency.mjs`) with UX references from `Gsync/jobsync` (854★). The tracker persists state to the existing SQLite control plane and renders to terminal (rich tables), HTML (export), and weekly email digest.

17.1 Application State Machine

The 12-phase ASP is event-sourced. Status transitions are append-only events; current state is the latest event's payload. This means full history is reconstructable from the events table — no separate status column is the source of truth.

Phase	Meaning	Allowed transitions
DISCOVERED	Found by scraper, saved to SQLite	→ SAVED, → SKIPPED
SAVED	User marked interested	→ APPLYING, → ARCHIVED
APPLYING	Saga started, in progress	→ APPLIED, → FAILED, → BLOCKED
BLOCKED	Policy gate denied (cap reached, blacklisted)	→ APPLYING (after unblock), → ARCHIVED
FAILED	Apply saga failed after retries	→ APPLYING (manual retry), → ARCHIVED
APPLIED	Submission confirmed by board	→ SCREENING, → REJECTED, → WITHDRAWN
SCREENING	Recruiter has viewed / contacted	→ INTERVIEW, → REJECTED, → GHOSTED
INTERVIEW	Phone screen / onsite scheduled	→ OFFER, → REJECTED, → WITHDRAWN
OFFER	Verbal or written offer received	→ ACCEPTED, → REJECTED, → NEGOTIATING
NEGOTIATING	In offer negotiation	→ ACCEPTED, → REJECTED, → WITHDRAWN
ACCEPTED	Offer signed — terminal success	(terminal)
REJECTED	Board declined — terminal	(terminal)
WITHDRAWN	User withdrew — terminal	(terminal)
GHOSTED	No response after 30 days — terminal	(terminal)
ARCHIVED	User archived (not pursuing) — terminal	(terminal)

17.2 Weekly Digest Email

```
# jobot/tracker/digest.py
from __future__ import annotations
from datetime import datetime, timedelta
import smtplib, asyncio
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
from ..storage import SessionLocal, Application, ApplicationEvent

class WeeklyDigest:
    async def generate_and_send(self, user_email: str):
        stats = await self._compute_stats()
        html = self._render_html(stats)
        await self._send_email(user_email, 'JoBot weekly digest', html)

    async def _compute_stats(self) -> dict:
        cutoff = datetime.utcnow() - timedelta(days=7)
        with SessionLocal() as s:
            new_apps = s.query(Application).filter(Application.applied_at >=
cutoff).all()
            # Status transitions in last 7 days
            events = (s.query(ApplicationEvent)
                .filter(ApplicationEvent.ts >= cutoff)
                .filter(ApplicationEvent.event_type == 'status_change')
                .all())

            return {
                'week_range': (cutoff.isoformat(), datetime.utcnow().isoformat()),
                'new_applications': len(new_apps),
                'status_transitions': [
                    {'from': e.payload.get('from'), 'to': e.payload.get('to'),
                    'job_title': e.application.job.title,
                    'company': e.application.job.company,
                    'ts': e.ts.isoformat()}
                    for e in events
                ],
                'funnel_velocity': self._funnel_velocity(events),
                'rejection_latency_days': self._rejection_latency(events),
                'cost_this_week_usd': self._weekly_cost(s),
                'cap_remaining': self._cap_remaining(s),
            }

    def _funnel_velocity(self, events) -> dict:
        # Avg days from APPLIED → INTERVIEW, APPLIED → REJECTED, etc.
        ...

    def _rejection_latency(self, events) -> float:
        # Avg days from APPLIED → REJECTED in last 7 days
        ...

    def _render_html(self, stats: dict) -> str:
        import jinja2
        template = jinja2.Template(WEEKLY_DIGEST_TEMPLATE)
        return template.render(**stats)

    async def _send_email(self, to: str, subject: str, html: str):
        from ..storage.secrets import SecretsStore
        smtp_user = SecretsStore.get('smtp.user')
        smtp_pass = SecretsStore.get('smtp.password')
        if not smtp_user:
            return # email not configured – skip silently
```

```

msg = MIMEMultipart('alternative')
msg['From'] = smtp_user
msg['To'] = to
msg['Subject'] = subject
msg.attach(MIMEText(html, 'html'))
await asyncio.to_thread(self._smtp_send, smtp_user, smtp_pass, to, msg)

@staticmethod
def _smtp_send(user, password, to, msg):
    with smtplib.SMTP_SSL('smtp.gmail.com', 465) as s:
        s.login(user, password)
        s.send_message(msg)

```

17.3 Analytics Dashboard (Terminal + HTML)

The dashboard renders three views: (1) **terminal** via rich tables — `jobot tracker list` shows the kanban; (2) **HTML export** via Jinja2 — `jobot tracker dashboard --html > dashboard.html` for sharing; (3) **weekly email** digest. All three consume the same query layer so numbers match across surfaces. The analytics include: funnel velocity (avg days from APPLIED → INTERVIEW), rejection latency (avg days from APPLIED → REJECTED), response rate by board, response rate by company size, resume template A/B (which resume template gets more interviews).

```

# jobot/tracker/dashboard.py
from __future__ import annotations
from datetime import datetime, timedelta
import rich.table, rich.console
from ..storage import SessionLocal, Application

class TrackerDashboard:
    def render_terminal(self, status_filter: str | None = None):
        console = rich.console.Console()
        with SessionLocal() as s:
            query = s.query(Application)
            if status_filter:
                query = query.filter(Application.status == status_filter)
            apps = query.order_by(Application.applied_at.desc()).limit(50).all()
            table = rich.table.Table(show_header=True, header_style='bold')
            table.add_column('Job')
            table.add_column('Company')
            table.add_column('Status', style='cyan')
            table.add_column('Applied', style='dim')
            table.add_column('Cost', justify='right')
            for app in apps:
                table.add_row(
                    app.job.title[:40],
                    app.job.company,
                    app.status,
                    app.applied_at.strftime('%Y-%m-%d') if app.applied_at else '-',
                    f'${app.cost_usd/100:.3f}',
                )
            console.print(table)

    def render_html(self) -> str:
        import jinja2
        # Same data, Jinja2 template → standalone HTML file
        ...

```


Chapter 18 — Capability Module: Career Analytics & Salary Benchmarking

Career analytics is JoBot's reflective layer — it looks at the aggregate of the user's job-search activity and answers strategic questions: 'Am I getting enough interviews?', 'Which companies respond fastest?', 'What skills am I missing?', 'Am I being paid below market?'. The lift is from MadsLorentzen/salary_lookup.py + career-ops/salary-gap.mjs, re-implemented behind JoBot's ModelRouter and SQLite.

18.1 Salary Lookup

Salary data is scraped from Levels.fyi (public, no API) and Built In (public RSS-like JSON). Cache for 24h to avoid rate-limiting. Output: low/median/high bands per role per location, with sample size.

```
# jobot/career/salary_lookup.py
from __future__ import annotations
import httpx, asyncio, json
from datetime import datetime, timedelta
from pathlib import Path
from pydantic import BaseModel

CACHE_DIR = Path.home() / '.jobot' / 'cache' / 'salary'
CACHE_TTL = timedelta(hours=24)

class SalaryBand(BaseModel):
    role: str
    location: str
    low: int # USD annual
    median: int
    high: int
    sample_size: int
    source: str # 'levels.fyi' | 'builtin'
    fetched_at: datetime

class SalaryLookup:
    def __init__(self):
        self._client = httpx.AsyncClient(timeout=15.0)
        CACHE_DIR.mkdir(parents=True, exist_ok=True)

    async def lookup(self, role: str, location: str) -> SalaryBand | None:
        cache_key = f'{role.lower()}_{location.lower()}'
        cache_file = CACHE_DIR / f'{cache_key}.json'
        if cache_file.exists():
            age = datetime.now() - datetime.fromtimestamp(cache_file.stat().st_mtime)
            if age < CACHE_TTL:
                return SalaryBand(**json.loads(cache_file.read_text()))
        # Scrape Levels.fyi (public JSON endpoint)
        try:
            resp = await self._client.get(
                'https://www.levels.fyi/js/salaryData.json',
                params={'role': role, 'location': location},
            )
            data = resp.json()
            bands = self._aggregate(data)
            band = SalaryBand(role=role, location=location,
```

```

        low=bands['low'], median=bands['median'],
        high=bands['high'], sample_size=bands['n'],
        source='levels.fyi', fetched_at=datetime.utcnow())
    cache_file.write_text(band.model_dump_json())
    return band
except Exception:
    return None

def _aggregate(self, raw_entries: list[dict]) -> dict:
    # Compute low (10th pct), median (50th), high (90th)
    salaries = sorted(int(e['total']) for e in raw_entries
                       if e.get('total') and str(e.get('total')).isdigit())
    n = len(salaries)
    if n == 0:
        return {'low': 0, 'median': 0, 'high': 0, 'n': 0}
    return {
        'low': salaries[int(n * 0.1)],
        'median': salaries[n // 2],
        'high': salaries[int(n * 0.9)],
        'n': n,
    }

```

18.2 Skill-Gap Analysis

Extract skills mentioned across saved JDs, compare to the user's profile, output the gap. Useful for prioritizing learning — if 70% of senior backend JDs mention Kubernetes and the user doesn't have it, that's a learning signal. Output is a prioritized list with estimated study time and recommended resources (lifted from `career-ops/skill-gap.mjs`).

```

# jobot/career/skill_gap.py
from __future__ import annotations
from collections import Counter
from pydantic import BaseModel
from ..llm.router import ModelRouter
from ..llm.base import Message
from ..storage.profile import UserProfile
from ..storage import SessionLocal, JobPosting

class SkillGap(BaseModel):
    skill: str
    jd_demand_pct: float # % of recent JDs mentioning it
    user_has: bool
    estimated_study_hours: int
    recommended_resource: str

class SkillGapAnalyzer:
    def __init__(self, router: ModelRouter, model: str | None = None):
        self._router = router
        self._model = model

    async def analyze(self, profile: UserProfile,
                     lookback_days: int = 30) -> list[SkillGap]:
        # 1. Get all JDs user has saved in last N days
        with SessionLocal() as s:
            from datetime import datetime, timedelta
            cutoff = datetime.utcnow() - timedelta(days=lookback_days)
            jobs = s.query(JobPosting).filter(JobPosting.scraped_at >= cutoff).all()
        if not jobs:

```

```

        return []
    # 2. LLM-extract skills from each JD (dedupe across JDs)
    all_skills = []
    for job in jobs[:50]: # cap at 50 to control LLM cost
        skills = await self._extract_skills(job.description)
        all_skills.extend(skills)
    # 3. Aggregate: count JDs mentioning each skill
    counter = Counter(all_skills)
    total_jds = len(jobs[:50])
    # 4. Compare to user profile
    user_skills_lower = [s.lower() for s in profile.skills]
    gaps = []
    for skill, count in counter.most_common(20):
        if count < 3: # noise floor
            continue
        has = any(s in skill.lower() or skill.lower() in s
                  for s in user_skills_lower)
        if has:
            continue
        gaps.append(SkillGap(
            skill=skill,
            jd_demand_pct=count / total_jds,
            user_has=False,
            estimated_study_hours=self._estimate_hours(skill),
            recommended_resource=self._recommend_resource(skill),
        ))
    return gaps[:10] # top 10 gaps

async def _extract_skills(self, jd_text: str) -> list[str]:
    resp = await self._router.route([
        Message('system', 'Extract a list of named skills from the job '
                      'description. Return as JSON: {"skills": [...],
...]}.'),
        Message('user', jd_text[:3000]),
    ], model=self._model, temperature=0.1, max_tokens=300)
    import json, re
    m = re.search(r'\{.*\}', resp.text, re.DOTALL)
    if m:
        return json.loads(m.group(0)).get('skills', [])
    return []

def _estimate_hours(self, skill: str) -> int:
    # Heuristic – extend with curated table
    easy = {'git', 'github', 'sql', 'rest', 'json', 'yaml'}
    medium = {'python', 'java', 'go', 'kubernetes', 'docker', 'aws', 'gcp'}
    hard = {'system design', 'distributed systems', 'ml', 'ml ops', 'kafka'}
    s = skill.lower()
    if s in easy: return 8
    if s in medium: return 40
    if s in hard: return 100
    return 20 # default

def _recommend_resource(self, skill: str) -> str:
    s = skill.lower()
    if 'kubernetes' in s: return 'https://kubernetes.io/docs/tutorials/'
    if 'python' in s: return 'https://docs.python.org/3/tutorial/'
    if 'system design' in s: return 'Designing Data-Intensive Applications (book)'
    return f'Search YouTube for "{skill} tutorial"'

```

Chapter 19 — Capability Module: Outreach & Cold-DM Generation

Outreach is the cold-DM layer: generating personalized LinkedIn messages to recruiters at companies the user is interested in. Re-implemented from `replyre/job-hunter` (150★) (no license — re-implemented from scratch). The module generates recruiter-search URLs, drafts DM templates based on the user's profile + target company, and (optionally) sends them via LinkedIn's web UI (browser-automated) or via SMTP for email outreach.

19.1 Profile-Driven YAML Presets

Recruiter outreach works best when personalized to a persona. The user defines YAML presets for each persona they're targeting (e.g., 'FAANG senior backend', 'startup founding engineer', 'quant finance'). Each preset has tone, length, signature, and highlighted profile aspects.

```
# profiles/outreach_presets.yaml
presets:
- name: faang_senior_backend
  target_companies: [Google, Meta, Apple, Amazon, Microsoft]
  tone: confident_professional
  max_words: 75
  highlights: [distributed_systems, system_design, leadership]
  signature: 'Aryan — github.com/aryansinghnagar'
- name: startup_founding_eng
  target_companies: [] # any startup
  tone: direct_enthusiastic
  max_words: 50
  highlights: [full_stack, fast_ship, ownership]
  signature: 'Aryan — github.com/aryansinghnagar'
- name: quant_finance
  target_companies: [Jane Street, Citadel, Two Sigma, Hudson River Trading]
  tone: precise_technical
  max_words: 100
  highlights: [low_latency, c_plus_plus, math]
  signature: 'Aryan — github.com/aryansinghnagar'
```

19.2 LinkedIn People-Search URL Generator

```
# jobot/outreach/linkedin_search.py
from __future__ import annotations
from urllib.parse import urlencode

def recruiter_search_url(company: str, role: str = 'recruiter') -> str:
    """Generate LinkedIn people-search URL for recruiters at a company.
    Lifted (re-implemented) from replyre/job-hunter."""
    # LinkedIn people search accepts keywords in URL params
    params = {
        'keywords': f'{role} {company}',
        'origin': 'GLOBAL_SEARCH_HEADER',
    }
    return f'https://www.linkedin.com/search/results/people/?{urlencode(params)}'

def hiring_manager_search_url(company: str, role: str) -> str:
    """Generate URL for hiring managers of a specific role at a company."""
    params = {
        'keywords': f'{role} hiring {company}',
        'title': f'Hiring Manager|Head of {role}|VP of {role}|Director of {role}',
        'currentCompany': f'[{company}]',
    }
    return f'https://www.linkedin.com/search/results/people/?{urlencode(params)}'
```

19.3 DM Template Generator

```
# jobot/outreach/dm_templates.py
from __future__ import annotations
from pydantic import BaseModel
from ..llm.router import ModelRouter
from ..llm.base import Message

class DMTemplate(BaseModel):
    subject: str | None # None for LinkedIn DMs (no subject)
    body: str
    preset_name: str

class DMGenerator:
    def __init__(self, router: ModelRouter, model: str | None = None):
        self._router = router
        self._model = model

    async def generate(self, profile_markdown: str, recipient_name: str,
                      company: str, preset: dict) -> DMTemplate:
        system = (
            f'You are writing a cold outreach DM. Tone: {preset["tone"]}. '
            f'Max {preset["max_words"]} words. End with: {preset["signature"]}. '
            'Be specific – mention one concrete thing the candidate did that maps '
            'to the company\'s known work. No buzzwords. No "passionate about".'
        )
        user = (
            f'# Candidate\n{profile_markdown}\n\n'
            f'# Recipient: {recipient_name} at {company}\n'
            f'# Highlights to emphasize\n{preset["highlights"]}\n\n'
            'Generate the DM. Output only the body text.'
        )
        resp = await self._router.route([
            Message('system', system),
            Message('user', user),
        ], model=self._model, temperature=0.6, max_tokens=200)
        return DMTemplate(body=resp.text.strip(), preset_name=preset['name'])
```

19.4 SMTP Email Sender

For email outreach (some recruiters prefer email), JoBot can send via SMTP. The sender credentials live in OS keyring. Each send is an idempotent side effect — if the same email (same subject + same recipient hash) is sent twice, the second send is a no-op. This prevents the apply saga's retry logic from spamming recruiters.

Chapter 20 — Capability Module: Scheduler & Daily Loop

The scheduler turns JoBot from a CLI invoked manually into a self-driving job-search agent. The daily loop: scrape new postings → dedup → score → rank → apply to top N → log to tracker → send digest. Run via cron or systemd timer. Lifted from `replyre/job-hunter` (daily digest scheduler) and `career-ops` (modes concept).

20.1 Modes — Scan, Apply, Digest, Full-Loop

Different users want different daily rhythms. A user in passive search wants scan-only (track new postings, no auto-apply). A user in active search wants full-loop (scrape + apply + digest). A user vacationing wants digest-only (no scraping, just summarize last week's activity). The four modes are first-class CLI commands.

Mode	What it does	Typical cron	Cost/day
scan-only	Scrape new postings, dedup, save. No apply. Send short email summary of new jobs found.	0 9 * * *	~\$0.01
apply-only	Skip scraping. Apply to top N jobs from existing SAVED pool.	0 10 * * *	\$0.50-\$2.00
digest-only	No scraping, no applying. Generate weekly/monthly digest email from existing data.	0 8 * * 1	~\$0.00
full-loop	Scrape + apply to top N + digest. The default for active job seekers.	0 9 * * *	\$0.50-\$2.50
interview-prep	Run a mock interview session for an upcoming scheduled interview.	on-demand	~\$0.10
outreach-batch	Send N cold DMs to recruiters at target companies.	0 14 * * 2,4	~\$0.05

20.2 Daily Loop Implementation

```
# jobot/scheduler/loop.py
from __future__ import annotations
import asyncio, logging
from datetime import datetime
from ..scrapers.jobspy_adapter import JobSpyAdapter
from ..adapters.base import ScrapeQuery
from ..apply.orchestrator import ApplyOrchestrator
from ..tracker.digest import WeeklyDigest
from ..policy.engine import PolicyEngine, PolicyViolation
from ..tracing.span import span

log = logging.getLogger(__name__)

class DailyLoop:
    def __init__(self, scraper: JobSpyAdapter,
                 applier: ApplyOrchestrator,
                 digest: WeeklyDigest,
                 policy: PolicyEngine,
                 profile):
        self._scraper = scraper
        self._applier = applier
        self._digest = digest
        self._policy = policy
        self._profile = profile

    async def run_full_loop(self, max_apply: int = 10):
        async with span('daily_loop.full', profile=self._profile.name):
            # 1. Scrape
            new_jobs = []
            async with span('daily_loop.scrape'):
                query = ScrapeQuery(
                    keywords=self._profile.search_keywords,
                    location=self._profile.location,
                    sites=self._profile.search_sites,
                    limit=100,
                    hours_old=24,
                )
            async for posting in self._scraper.scrape(query):
                new_jobs.append(posting)
            log.info(f'Scraped {len(new_jobs)} new jobs')

            # 2. Rank (skill extraction + salary match + remote match)
            ranked = await self._rank(new_jobs)

            # 3. Apply to top N
            applied = 0
            for job in ranked[:max_apply]:
                try:
                    await self._policy.check_apply(self._profile.user_id, job.id,
                                                  job.board, job.company,
                                                  estimated_cost_usd=0.05)
                    result = await self._applier.apply_to_job(job, self._profile)
                    if result.success:
                        await self._policy.record_apply(self._profile.user_id,
                                                         result.cost_usd)
                        applied += 1
                except PolicyViolation as e:
                    log.warning(f'Policy blocked apply to {job.company}: {e}')
                    break # cap reached
```



```

        except Exception as e:
            log.error(f'Apply to {job.company} failed: {e}')
            continue # don't let one failure kill the loop
        log.info(f'Applied to {applied} jobs')

    # 4. Send digest
    await self._digest.generate_and_send(self._profile.email)

    async def _rank(self, jobs: list) -> list:
        # Score by: keyword match in JD, salary vs target, remote-ness
        # Use ModelRouter to score each JD against profile
        ...
        return jobs # simplified

```

20.3 Cron Entry

```

# Add to user crontab (crontab -e):

# JoBot daily full-loop at 9am
0 9 * * * /usr/local/bin/jobot loop full-loop --profile default --max-apply 10

# JoBot weekly digest every Monday at 8am
0 8 * * 1 /usr/local/bin/jobot loop digest --profile default

# JoBot outreach batch Tue+Thu at 2pm
0 14 * * 2,4 /usr/local/bin/jobot loop outreach-batch --profile default --preset
faang_senior_backend --count 5

# Reset daily caps at midnight UTC
0 0 * * * /usr/local/bin/jobot policy reset-daily --profile default

```

systemd timer alternative (more reliable than cron)

For production deployments, prefer systemd timers over cron — they support dependency ordering, automatic retries, logging via journald, and won't run if the previous instance is still going (via `ConditionPathExists`). The Dockerfile ships a systemd unit file at `/etc/systemd/system/jobot-daily.service`.

Chapter 21 — Phased Migration Roadmap (Phases 0–4)

The merge is sequenced across 15+ weeks in five phases. Each phase has explicit entry criteria (must be true before the phase begins), work items (the concrete deliverables), and exit criteria (must be true before the phase ends). Risks are surfaced per-phase. The phases are sequential — Phase 2 cannot begin until Phase 1's exit criteria are met.

21.1 Phase 0 — Audit & Cleanup (Week 1)

Goal: clean the JoBot repo to a known-good baseline before adding new code. The existing planning documents in root are confusing newcomers; the worklog.md is mixed with implementation log; some stub files exist that block clean imports.

Step	Detail	Owner
0.1	Move all *.md planning docs from repo root to docs/history/. Leave a single docs/README.md pointing to current authoritative plan (this document).	Maintainer
0.2	Audit and delete stub files in jobot/adapters/ that don't yet have implementations. Empty stubs block clean imports.	Maintainer
0.3	Run full test suite; record baseline pass rate and execution time. Target: 100% pass, <60s runtime.	Maintainer
0.4	Document the current ASP (12-phase) in docs/asp.md with state diagram (Mermaid). Make it the single source of truth.	Maintainer
0.5	Freeze the public interface for: BoardAdapter, ApplyRequest/Result, ModelRouter.route(), BlobStore, PolicyEngine.check_apply(). These are the contracts the merge must not break.	Maintainer
0.6	Run ruff + mypy strict clean. Fix any flagged issues.	Maintainer

Entry criteria: none — this is the start. **Exit criteria:** repo root contains only README.md, pyproject.toml, LICENSE, JoBot_Refactor_Plan.md (a redirect), src/, tests/, docs/, .github/, .gitignore. Test suite passes 100%. mypy strict clean.

Phase 0 risk

Cutting the stubs in step 0.2 will temporarily break `from jobot.adapters import linkedin` if any test imports them. The test file is updated in the same PR — no broken main branch.

21.2 Phase 1 — Provider Abstraction + Config Schema (Weeks 2–3)

Goal: extend the existing 4-provider ModelRouter to the full 6 provider classes (adding OpenAI-compatible, Mistral, Cohere, Bedrock, Vertex). Land the config schema (profiles/*.yaml) and the secrets pipeline (OS keyring for all API keys, never .env).

Step	Detail	Source ref
1.1	Implement OpenAICompatProvider (covers OpenRouter, Groq, Together, Ollama, vLLM). Single class with base_url+api_key params.	OpenAI SDK + openai-compat pattern
1.2	Implement MistralProvider (direct API).	Mistral docs
1.3	Implement CohereProvider (direct API).	Cohere docs

Step	Detail	Source ref
1.4	Implement BedrockProvider (boto3-based, IAM auth).	AWS Bedrock docs
1.5	Implement VertexProvider (google-cloud-aiplatform).	Vertex AI docs
1.6	Add pricing.yaml with per-model cost data for all 12 provider instances.	Provider pricing pages
1.7	Land ModelRouter v2 with cost-aware routing + fallback chain + daily cost cap.	Spec in Chapter 6
1.8	Land profiles/ YAML schema + secrets.py OS-keyring integration.	Spec in Chapter 8
1.9	Add <code>jobot config get/set/show</code> CLI commands.	Spec in Chapter 28
1.10	Add <code>jobot doctor</code> CLI command (verify env: Python, deps, keyring, providers reachable).	Spec in Chapter 28
1.11	Tests for every provider (mock HTTP responses) + integration tests against real APIs (skipped if no API key).	Native
1.12	Update install docs (Chapter 24) + provider setup docs (Chapter 26).	Native

Entry: Phase 0 exit criteria met. **Exit:** `jobot doctor` passes with at least one provider configured (Gemini default). `jobot config set llm.default_provider anthropic` + `jobot config show` round-trips. All provider tests pass. Cost tracking works — every LLM call emits a span with `cost_usd` attribute.

Phase 1 risk

Bedrock and Vertex auth is complex — IAM roles, JSON key files, region pinning. Schedule a full day per provider for the integration tests alone.

21.3 Phase 2 — Scraper Merge (Weeks 4–5)

Step	Detail	Source ref
2.1	Add <code>jobspy>=0.4.0</code> to pyproject.toml dependencies.	speedyapply/JobSpy
2.2	Implement JobSpyAdapter wrapping JobPy's <code>scrape_jobs()</code> behind JoBot's adapter contract (Chapter 12).	Spec in Chapter 12
2.3	Wire JobSpyAdapter into adapter registry. Circuit breaker + dedup active.	Spec in Chapter 12
2.4	Implement GreenhouseAdapter (direct JSON API — already in JoBot, polish + extend).	Greenhouse public API
2.5	Implement LeverAdapter (direct JSON API).	Lever public API
2.6	Implement CareerPageScanner with YAML-driven 150+ site config.	replyre/job-hunter (re-implemented)
2.7	Vector dedup service — embed each JD, dedup on cosine sim > 0.92.	Native (test_dedup.py)
2.8	Add <code>jobot scrape <board></code> CLI command.	Spec in Chapter 28
2.9	Tests: scraper returns ≥50 real postings per board; dedup reduces repost rate by ≥80%.	Native

Entry: Phase 1 exit criteria met. **Exit:** `jobot scrape linkedin --keywords 'senior backend' --location 'San Francisco' --limit 50` returns 50+ real LinkedIn postings persisted to SQLite. Same for greenhouse, lever, indeed, glassdoor. Vector dedup catches a synthetic repost test case. **Highest-leverage single decision in the entire plan.**

Phase 2 risk

JobSpy's selectors will break when LinkedIn/Glassdoor change their HTML. Pin to a specific JobPy version, and add a daily health-check that emails the maintainer when a board goes red.

21.4 Phase 3 — Resume + Cover-Letter + Auto-Apply (Weeks 6–9)

Step	Detail	Source ref
3.1	Implement Drafter + Reviewer with A-F rubric scoring (Chapter 13).	MadsLorentzen + career-ops
3.2	Implement LaTeX rendering pipeline (lualatex + xelatex + pdftotext ATS check).	MadsLorentzen
3.3	Land 3 LaTeX templates (default, modern, classic) via Jinja2.	Native
3.4	Implement CoverLetterGenerator with 5 tone presets.	career-ops
3.5	Implement QuestionAnswerer (lifted+re-implemented from AIHawk) behind ModelRouter.	feder-cr/AIHawk
3.6	Implement LinkedInEasyApplySaga on Patchright (Chapter 15).	AIHawk + GodsScion
3.7	Implement GreenhouseApplyAdapter (direct multipart form POST).	Greenhouse public API
3.8	Implement LeverApplyAdapter (direct).	Lever public API
3.9	Implement ApplyOrchestrator with saga pattern + compensating actions + idempotency keys.	Spec in Chapter 9
3.10	Implement preview-before-commit + human-in-the-loop checkpoint for sensitive applies.	Spec in Chapter 7
3.11	Add <code>jobot apply <job-id> + jobot apply --batch N + --dry-run</code> .	Spec in Chapter 28
3.12	Tests: dry-run produces PDF + cover letter; saga rollback works on synthetic failure; real apply to test account on Greenhouse succeeds.	Native

Entry: Phase 2 exit. **Exit:** `jobot apply --dry-run <job-id>` produces a tailored resume PDF + cover letter + answers to all form questions, with an ATS score ≥ 0.85 . Real apply to a test Greenhouse posting succeeds and the saga's compensating action (withdraw) works. Policy cap blocks the 51st apply.

Phase 3 risk

LinkedIn actively detects and blocks automation. The Patchright stealth layer will need ongoing maintenance. Plan for: weekly selector-audit, monthly Patchright version bumps, and a human-in-the-loop fallback when automation is detected (slow-mode with longer waits, more realistic mouse movements).

21.5 Phase 4 — ATS, Analytics, Plugins, Docker (Weeks 10–13)

Step	Detail	Source ref
4.1	Implement InterviewPrep module (mock session + STAR coach).	career-ops + jennifer88huang
4.2	Implement TrackerDashboard (terminal + HTML export).	career-ops + jobsync
4.3	Implement WeeklyDigest email sender.	career-ops + replyre/job-hunter
4.4	Implement CareerAnalytics (salary lookup + skill-gap analysis).	MadsLorentzen + career-ops
4.5	Implement Outreach module (LinkedIn people-search URL gen + DM templates + SMTP).	replyre/job-hunter (re-implemented)
4.6	Implement Scheduler with 4 modes (scan/apply/digest/full-loop).	career-ops
4.7	Implement PluginInstaller + PluginManifest schema + audit flow.	career-ops
4.8	Write Dockerfile (multi-stage: builder + runtime).	career-ops + DaKheera47/job-ops
4.9	Write docker-compose.yml (jobot + browser-profile volume + persistent data).	Spec in Chapter 27
4.10	Add CI hardening: CodeQL workflow, SBOM generation, signature-CI.	career-ops
4.11	Add jobot plugin install/list/audit + jobot interview mock + jobot digest + jobot tracker + jobot scheduler CLI commands.	Spec in Chapter 28
4.12	End-to-end integration test: full-loop on a Docker compose stack against mock ATS.	Native

Entry: Phase 3 exit. **Exit:** `docker compose up -d` brings up a fully working JoBot stack. `docker compose exec jobot jobot doctor` passes. `jobot plugin install <git-url>` installs a sample plugin and audit succeeds. Weekly digest arrives in inbox. CI has CodeQL + SBOM + signature-CI workflows.

Phase 4 risk

Plugin system exposes a code-execution surface. Audit flow must verify plugin source hashes; user must explicitly trust each plugin. Document the threat model in docs/security.md. Default-deny: nothing runs without explicit `jobot plugin install + --trust`.

21.6 Timeline Summary

Phase	Weeks	Theme	Key deliverable
0	1	Audit & cleanup	Clean repo baseline
1	2–3	Providers + config	6-provider ModelRouter + secrets pipeline
2	4–5	Scrapers	JobSpy vendored + 6 board adapters + dedup
3	6–9	Apply pipeline	Drafter-reviewer resume + LaTeX + auto-apply saga
4	10–13	Surround features	Interview prep + tracker + outreach + scheduler + Docker + plugins
5+	14+	Polish & ecosystem	Documentation site, community plugins, k8s manifests (future)

Chapter 22 — Risks, Mitigations & License Compliance

An honest risk register. Each risk has probability, impact, and concrete mitigation. Risks are categorized as legal, technical, operational, and product. The license compliance section re-iterates the decision tree from Chapter 4 with the explicit list of legally untouchable repositories.

22.1 Risk Register

#	Category	Risk	Prob	Impact	Mitigation
R1	Legal	Vendoring code from a NOASSERTION/no-license repo into JoBot's AGPL-3.0 codebase.	Med	High	License Compliance Decision Tree (Chapter 4 §4.5). Re-implement from scratch; never vendor.
R2	Legal	AGPL-3.0 contamination from a permissively-licensed repo (e.g., MIT).	Low	Med	Apache/MIT/BSD are one-way compatible with AGPL — vendoring is fine. Keep copyright + license headers in vendored files.
R3	Legal	User auto-applies to a job in a jurisdiction where AI-submitted applications are regulated (e.g., EU AI Act).	Med	High	Document in setup docs. Add <code>require_human_approval_above_cost_usd</code> default to \$0.50. Default to dry-run mode; user must opt-in to real apply.
R4	Technical	LinkedIn detects Patchright automation; user's account gets rate-limited or banned.	High	High	Stealth patches (Patchright), session reuse via CDP, named-action pattern, slow-mode with realistic delays, weekly selector-audit. Human-in-the-loop fallback. Document that auto-apply violates LinkedIn ToS.
R5	Technical	JobSpy scraper selectors break when a board redesigns its HTML.	High	Med	Pin JobSpy version, daily health-check, alert on circuit-breaker opening. Patch JobSpy upstream when possible.
R6	Technical	LLM API rate limits cause cascade failures.	Med	High	ModelRouter fallback chain (try Gemini → OpenAI → Anthropic). Circuit breaker per provider. Daily cost cap prevents accidental runaway spend.
R7	Technical	Captcha solver returns garbage; auto-apply submits with wrong answers.	Med	High	Human-in-the-loop fallback for unsolvable captchas. Never auto-submit if captcha is unsolved — fail the saga and quarantine.
R8	Technical	Saga left a job in half-applied state (e.g., form filled but not submitted).	Med	High	Compensating actions on every step. Checkpoint after every step. Quarantine failed sagas. User can resume from checkpoint via <code>jobot apply --resume <saga-id></code> .
R9	Technical	SQLite database is locked (concurrent writer contention).	Low	Med	WAL mode allows concurrent readers + single writer. All writes go through SessionLocal() context manager. Long transactions split into smaller ones.
R10	Operational	OS keyring unavailable on WSL2 (Linux Secret Service not running).	Med	Med	<code>jobot doctor</code> checks keyring availability. Document workaround: <code>keyring.set_keyring(keyring.backends.fail.Keyring())</code> + encrypted fallback file.
R11	Operational	LaTeX resume generation fails on Windows (TeX Live setup is painful).	High	Low	Recommend Tectonic (single-binary LaTeX) as alternative. Document both paths. <code>jobot doctor</code> checks LaTeX engine availability.
R12	Operational	Browser profile volume corrupts (Playwright crashed mid-write).	Low	High	Browser profile is backed up to <code>data/browser-backup/<board>/</code> weekly. Recovery: <code>jobot browser restore <board></code> .
R13	Operational	Daily cost cap silently reached at 3am; morning apply batch silently no-ops.	Med	Med	Alert email on cap-hit. <code>jobot tracker list --status blocked</code> shows blocked jobs. Scheduler raises cap by 20% on consecutive cap-hit days (user-approved ceiling).
R14	Operational	PII masker over-aggressive — masks non-PII strings, breaking debug traces.	Med	Low	Configurable masker strictness (off / regex-only / regex+LLM). Default regex-only. LLM-assist only on explicit opt-in.
R15	Operational	Data loss from SQLite corruption.	Low	High	WAL mode is robust. Daily backup to <code>data/backup/<date>.sqlite</code> . <code>jobot backup</code> and <code>jobot restore</code> commands. Backup retention 30 days.

#	Category	Risk	Prob	Impact	Mitigation
R1 6	Product	User overtrusts auto-apply — submits to 200 jobs/day, gets blacklisted by boards.	Med	High	Default daily cap = 50. Default weekly cap = 200. Soft warning if user raises cap above 100/day. Document recommended daily volume in setup.
R1 7	Product	User submits a half-tailored resume with LLM-hallucinated experience; gets caught in interview.	Med	High	Reviewer step explicitly checks truthfulness against base profile. Drafter prompt forbids invention. Document that JoBot is a force multiplier, not a substitute for honesty.
R1 8	Product	Cover letter sounds like every other LLM-generated cover letter (LLM cliches).	High	Med	Multiple tone presets. Reviewer can critique cliches. User can override with custom prompt fragments. Document recommended post-edit.

22.2 License Compliance — Re-iteration

JoBot is AGPL-3.0. Source code from AGPL-3.0 or GPL-3.0 repos may be vendored (with attribution + license header). Apache/MIT/BSD code may be vendored. MPL-2.0 must keep file-level copyleft. NOASSERTION and no-license repositories are legally untouchable for code vendoring — algorithms must be re-implemented from scratch.

Repo	Stars	License issue	Action
wodsuz/EasyApplyJobsBot	804	NOASSERTION	Reference only — re-implement
imon333/Job-apply-AI-agent	177	No LICENSE	Reference only — re-implement
replyre/job-hunter	150	No LICENSE	Reference only — re-implement
beatwad/LinkedIn-AI-Job-Applier-Ultimate	147	No LICENSE	Reference only — re-implement
jennifer88huang/interview-skills	272	No LICENSE	Reference only — re-implement (port prompt design only)
algsch/job_agentic	3	No LICENSE	Reference only — re-implement
DaKheera47/job-ops	3,840	NOASSERTION	Reference only — re-implement

Vendorable repos (with attribution): AIHawk (AGPL-3.0, feder-cr), JobSpy (MIT, speedyapply), career-ops (MIT, santifer), MadsLorentzen/ai-job-search (MIT), GodsScion/Auto_job_applier_linkedin (MIT), JobFunnel (MIT, archived — reference only since stale), sreekarrs/JobSearch-Agent (GPL-3.0), JobFunnel (MIT).

Chapter 23 — Setup: Prerequisites & System Requirements

JoBot is a Python 3.11+ application. Its hard dependencies are Python, pip, git, and an OS-native keyring. Its soft dependencies are a LaTeX engine (for resume PDF generation), Playwright system deps (for browser automation), and Docker (for containerized deployment). This chapter lists what you need before you start.

23.1 Hard Prerequisites

Component	Version	Why needed	How to verify
Python	3.11 or newer	JoBot uses PEP 695 type aliases, match-case, async improvements	<code>python3 --version</code>
pip	23.0+	Package installation	<code>pip --version</code>
git	2.20+	Cloning the repo	<code>git --version</code>
OS keyring	Any	API key + secret storage. macOS Keychain (built-in), Linux Secret Service (gnome-keyring or kwalletd), Windows Credential Manager (built-in).	<code>python3 -c 'import keyring; print(keyring.get_keyring())'</code>
At least one LLM API key	—	Default: Google Gemini (free tier sufficient for testing)	Sign up at aistudio.google.com/apikey

23.2 Soft Prerequisites

Component	Version	Why needed	Install
LaTeX engine	TeX Live 2023+ or Tectonic 0.5+	Resume PDF generation via lualatex/xelatex. Without it, <code>jobot resume tailor</code> fails.	macOS: <code>brew install --cask mactex-no-gui</code> or <code>brew install tectonic</code> ; Linux: <code>apt install texlive-full</code> or <code>snap install tectonic</code>
poppler-utils	20+	<code>pdftotext</code> for ATS parseability check on generated resumes.	macOS: <code>brew install poppler</code> ; Linux: <code>apt install poppler-utils</code>
Playwright system deps	—	Browser automation. <code>patchright install chromium</code> will install Chromium itself but system libs are user-installed.	<code>patchright install-deps</code> (after pip install)
Docker	24+	Optional but recommended — runs JoBot in a container without polluting your host Python env.	docs.docker.com/get-docker
Docker Compose	v2+	Container orchestration for multi-service setup.	Bundled with Docker Desktop; Linux: <code>apt install docker-compose-plugin</code>

23.3 OS Support Matrix

OS	Support level	Notes
macOS (Intel)	Tier 1 — fully tested	Use Homebrew for prereqs. Patchright works natively.
macOS (Apple Silicon)	Tier 1 — fully tested	Same as Intel. Architecture transparent via Python's universal2 wheels.
Linux (Ubuntu 22.04+)	Tier 1 — fully tested	Best Docker experience. gnome-keyring recommended for keyring integration.
Linux (Debian 12+)	Tier 1 — fully tested	Same as Ubuntu.
Linux (Fedora 39+)	Tier 2 — should work	Not in CI but no known issues.
Linux (Arch)	Tier 3 — community-supported	JoBot works but maintainers don't test.
Windows 10/11 (native)	Tier 3 — works with WSL2	WSL2 is strongly recommended. Native Windows works but LaTeX + Playwright setup is painful. OS keyring (Credential Manager) works.
Windows 10/11 (WSL2)	Tier 1 — fully tested	Best Windows experience. Treat as Linux Ubuntu 22.04 inside WSL2.
FreeBSD	Tier 4 — unsupported	Not tested; keyring integration may not work.
Other Unix	Tier 4 — unsupported	Compile Python 3.11+, hope for the best.

23.4 Resource Requirements

Resource	Minimum	Recommended	Notes
RAM	4 GB	8 GB	Browser automation is memory-hungry. Headless Chrome uses ~500MB per context.
Disk	2 GB	10 GB	JoBot core (~50MB) + Playwright browsers (~400MB) + LaTeX (~3GB if full TeX Live) + SQLite + blob store.
CPU	Any modern dual-core	Quad-core+	Browser automation and LaTeX compilation benefit from extra cores.
Network	Broadband	Broadband	Job scraping is bandwidth-light; LLM calls are tiny payloads; browser automation is moderate.

Chapter 24 — Setup: Local Dev Installation

Step-by-step install for macOS, Linux, and Windows (WSL2). Each step includes the exact command, the expected output, and a troubleshooting tip if the step commonly fails.

24.1 macOS (Intel & Apple Silicon)

Step 1. Install Homebrew (if not present).

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
# Expected: Homebrew installs to /opt/homebrew (Apple Silicon) or /usr/local (Intel)
# Verify:
brew --version
# → Homebrew 4.x.x
```

Step 2. Install Python 3.11, poppler, and (optionally) Tectonic.

```
brew install python@3.11 poppler
# Optional: lightweight LaTeX alternative to full MacTeX
brew install tectonic
# Verify:
python3.11 --version && pdftotext -v 2>&1 | head -1
# → Python 3.11.x ; pdftotext v22.x
```

Step 3. Clone JoBot.

```
git clone https://github.com/aryansinghnagar/JoBot.git
cd JoBot
```

Step 4. Create venv + install.

```
python3.11 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -e '[dev]'
# Expected: installs jobot + all dev deps (pytest, ruff, mypy)
```

Step 5. Install Patchright browser.

```
patchright install chromium
# Optional: also install system deps for Chromium
patchright install-deps
```

Step 6. Run `jobot init` + set Gemini API key.

```
jobot init
# → Creates ~/.jobot/ directory with default profile.yaml
# → Prompts for Gemini API key (stored in macOS Keychain)
# Enter your key from https://aistudio.google.com/apikey
```

Step 7. Run `jobot doctor`.

```
jobot doctor
# Expected output:
# ✓ Python 3.11.9
# ✓ SQLite 3.43.0
# ✓ OS keyring: macOS Keychain
# ✓ Patchright browser installed
```

```
# ✓ LaTeX engine: lualatex (TeX Live 2024)
# ✓ pdftotext available
# ✓ LLM provider 'gemini' configured and reachable (test call took 0.8s)
# JoBot is ready.
```

Troubleshooting macOS install

If `jobot doctor` fails on 'OS keyring', ensure macOS Keychain is unlocked. If it fails on 'Patchright browser', re-run `patchright install chromium` with `sudo`. If it fails on LaTeX, install Tectonic instead of full MacTeX: `brew install tectonic`.

24.2 Linux (Ubuntu 22.04+)

Step 1. Install Python 3.11 + system deps.

```
sudo apt update
sudo apt install -y python3.11 python3.11-venv python3.11-dev \
    poppler-utils git build-essential \
    gnome-keyring # for keyring integration
# Optional: full TeX Live (large download)
sudo apt install -y texlive-full
# Or lightweight alternative:
sudo snap install tectonic
```

Step 2. Clone + venv + install.

```
git clone https://github.com/aryansinghnagar/JoBot.git
cd JoBot
python3.11 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -e '.[dev]'
patchright install chromium
patchright install-deps
```

Step 3. Initialize gnome-keyring (if running headless server).

```
# On a desktop Linux, gnome-keyring starts automatically on login.
# On a headless server, start it manually:
eval $(gnome-keyring-daemon --start)
export SSH_AUTH_SOCK
# Add this to ~/.bashrc for persistence.
```

Step 4. Same `jobot init` + `jobot doctor` as macOS.

24.3 Windows 10/11 with WSL2 (Recommended)

Strongly recommended over native Windows. WSL2 gives you Ubuntu 22.04 with full Linux compatibility, including OS keyring via gnome-keyring (which runs inside WSL2).

Step 1. Install WSL2 + Ubuntu 22.04.

```
# In PowerShell (admin):
wsl --install -d Ubuntu-22.04
# Restart, set Ubuntu user/password when prompted.
# Open Ubuntu shell (Start → WSL → Ubuntu).
```

Step 2. Inside Ubuntu shell, follow the Linux steps (24.2).

Step 3. For Patchright browser on WSL2: install native Windows Chromium and configure WSLg for X11 forwarding, OR use headless Chromium inside WSL2 (default behavior).

```
# Default behavior is headless Chromium inside WSL2 – works fine.
# For interactive browser sessions (human-in-the-loop captcha solving),
# set DISPLAY to Windows host:
export DISPLAY=$(cat /etc/resolv.conf | grep nameserver | awk '{print $2}'):0.0
# Requires VcXsrv or similar X server running on Windows.
```

24.4 Windows Native (Not Recommended)

JoBot works on native Windows but the setup is more painful. LaTeX (MiKTeX or TeX Live for Windows) is a large install. Patchright on Windows works but requires extra system DLLs. OS keyring uses Windows Credential Manager (built-in). If you must use native Windows, the steps are similar to macOS but with Windows installers for Python (3.11+ from python.org), Git (from git-scm.com), and MiKTeX (from miktex.org).

Why we recommend WSL2

Native Windows has these known pain points: (1) LaTeX setup is 5-10GB vs Tectonic's 100MB; (2) Patchright's `install-deps` doesn't work on Windows — you must install Visual C++ Redistributable manually; (3) cron isn't available — Task Scheduler is the only scheduler; (4) the OS keyring works but `keyring.get_keyring()` returns `WindowsCredentialVaultKeyring` which has subtle permission issues. WSL2 sidesteps all of these.

Chapter 25 — Setup: Configuration & Secrets

JoBot uses a three-tier config system. Tier 1 (.env) holds runtime config that's safe to log (log level, default provider name, profile name). Tier 2 (secrets.yaml, Fernet-encrypted at rest) holds board credentials and other config that should be secret but isn't API keys. Tier 3 (OS keyring) holds the actual API keys and the Fernet master key — never written to disk in plaintext.

25.1 Tier 1 — .env (Runtime Config)

.env is loaded by python-dotenv on JoBot startup. It contains only safe-to-log config. Add to .gitignore (it is by default).

```
# .env – runtime config (NEVER put API keys here!)

JOBOT_ENV=development # development | production
JOBOT_LOG_LEVEL=INFO # DEBUG | INFO | WARNING | ERROR
JOBOT_PROFILE=default # which profile to use from profiles/
JOBOT_DATA_DIR=~/.jobot/data # SQLite + blobs
JOBOT_BROWSER_PROFILE_DIR=~/.jobot/browser-profiles
JOBOT_EVIDENCE_DIR=~/.jobot/evidence # screenshots
JOBOT_CACHE_DIR=~/.jobot/cache # salary lookups, etc.

# Default LLM provider (one of: gemini, openai, anthropic, openrouter,
# groq, together, ollama, vllm, mistral, cohere, bedrock, vertex)
JOBOT_DEFAULT_LLM_PROVIDER=gemini
```

25.2 Tier 3 — OS Keyring (API Keys & Master Key)

All API keys live in OS keyring, accessed via `keyring.get_password('jobot', <key>)`. JoBot's `jobot config set` command writes to the keyring — you never edit a file directly. The master Fernet key (used to encrypt secrets.yaml and blob store) is auto-generated on first run and stored under key `master.fernet.key`.

```
# Set Gemini API key (default provider):
jobot config set llm.api_key.gemini AIzaSy...
# → Stored in macOS Keychain / Linux Secret Service / Windows Credential Manager

# Set OpenAI API key (for fallback chain):
jobot config set llm.api_key.openai sk-proj-...

# Set Anthropic API key:
jobot config set llm.api_key.anthropic sk-ant-...

# Set OpenRouter (OpenAI-compatible):
jobot config set llm.api_key.openrouter sk-or-...

# Set LinkedIn cookies (for browser session reuse – optional,
# usually not needed because Patchright persists session in browser-profile volume):
jobot config set board_cookies.linkedin 'li_session=...; li_at=...'

# Set SMTP credentials (for digest email):
jobot config set smtp.user your-email@gmail.com
jobot config set smtp.password your-app-password

# Verify config:
jobot config show
```

```
# → Lists all keys (values masked: 'AIzaSy***')

# Get a specific value (for debugging):
jobot config get llm.api_key.gemini
# → AIzaSy...
```

25.3 Tier 2 — secrets.yaml (Encrypted Board Credentials)

Secrets that aren't API keys — e.g., LinkedIn cookies, Workday credentials, custom recruiter portal logins — live in `secrets.yaml`. This file is Fernet-encrypted at rest; the master key is in OS keyring.

```
# Edit secrets.yaml (jobot decrypts on open, re-encrypts on save):
jobot secrets edit
# Opens $EDITOR with decrypted YAML:

board_credentials:
  linkedin:
    email: you@example.com
    # password in OS keyring as 'board_password.linkedin'
  greenhouse:
    api_token: gh-...
  workday_custom:
    username: you@example.com
    password_env: WORKDAY_PASSWORD # reads from env at apply time

smtp:
  host: smtp.gmail.com
  port: 465
  # credentials in OS keyring as 'smtp.user' and 'smtp.password'

scheduler:
  daily_apply_cap: 50
  weekly_apply_cap: 200
  daily_cost_cap_usd: 5.00
```

25.4 Tier 4 — Profiles (User Personas)

Profiles live in `profiles/<name>.yaml`. Each profile is a complete user persona: identity (name, email, phone), job-search target (keywords, location, salary expectations), resume base (the master resume that gets tailored per JD), and LLM config overrides. Multiple users can share one JoBot install by switching profiles via `--profile`.

```
# profiles/default.yaml – default user profile
identity:
  name: Aryan Singh Nagar
  email: aryansinghnagar@example.com
  phone: "+1-555-0100"
  location: "San Francisco, CA"
  linkedin_url: https://linkedin.com/in/aryansinghnagar
  github_url: https://github.com/aryansinghnagar

search:
  keywords: ["senior backend engineer", "distributed systems", "python"]
  location: "San Francisco Bay Area"
  sites: [linkedin, greenhouse, lever, indeed]
  remote_only: false
  visa_sponsor_required: false
```

```

target:
  min_salary_usd: 180000
  min_role_level: senior
  blacklisted_companies: []
  blacklisted_keywords: ["on-call", "weekend work"]

resume_base: # base resume, tailored per JD
summary: |
  Senior backend engineer with 8 years building distributed systems at scale.
  Led 0-to-1 of three services now handling 10M req/day. Deep Python + Go.
skills:
  - Python
  - Go
  - Kubernetes
  - PostgreSQL
  - Redis
  - Kafka
  - AWS
  - GCP
  - Terraform
  - Distributed Systems Design
experience:
  - company: Stripe
    title: Senior Software Engineer
    dates: "2022-Present"
    bullets:
      - "Led redesign of payment reconciliation service; reduced P99 latency from
4.2s to 180ms."
      - "Owned migration from monolith to event-driven microservices (Kafka + Go)."
```

```

  - company: Airbnb
    title: Software Engineer
    dates: "2018-2022"
    bullets:
      - "Built search ranking pipeline serving 50M queries/day."
education:
  - degree: "B.S. Computer Science"
    school: "UC Berkeley"
    year: 2018

llm:
  default_provider: gemini
  fallback_chain: [gemini, openai, anthropic]
  daily_cost_cap_usd: 5.00
  task_overrides:
    resume_tailoring:
      provider: anthropic
      model: claude-3-5-sonnet-20241022
    question_answering:
      provider: gemini
      model: gemini-2.0-flash

outreach:
  presets: [faang_senior_backend, startup_founding_eng]
  daily_dm_cap: 5

```


25.5 Config Wizard

`jobot init` runs an interactive wizard on first invocation. It creates the `~/.jobot/` directory tree, prompts for identity + search + target fields, generates a `profiles/default.yaml` from the responses, and prompts for at least one LLM API key (stored in OS keyring). The wizard is restartable — answering 'skip' to any field leaves it blank for later editing via `jobot config set`.

Chapter 26 — Setup: AI Provider API Keys (All 6 Supported Providers)

JoBot supports six provider classes with twelve concrete instances. Google Gemini is the default — out of the box, with only `GEMINI_API_KEY` set, JoBot works. This chapter walks through each provider setup: where to get the key, what env var / keyring entry to set, minimum scopes, cost model, and recommended default model.

26.1 Google Gemini (DEFAULT — Required for First Run)

Where to get the key: Google AI Studio at <https://aistudio.google.com/apikey>. Sign in with any Google account, click 'Create API Key', copy the `AIza...` string.

How to set it in JoBot:

```
jobot config set llm.api_key.gemini AIzaSy...  
# Verify:  
jobot config get llm.api_key.gemini  
# → AIzaSy...
```

Free tier: 15 requests/minute, 1500 requests/day, 1 million tokens/minute. Sufficient for ~50 LLM calls/day (enough for the default daily apply cap of 50). **Recommended default model:** `gemini-2.0-flash` — \$0.075/1k input tokens, \$0.30/1k output tokens. ~\$0.005 per resume tailoring + cover letter. **Cost cap:** the default `daily_cost_cap_usd: 5.00` allows ~1000 LLM calls before the cap engages.

26.2 OpenAI

Where to get the key: <https://platform.openai.com/api-keys>. Requires an OpenAI account with billing set up (no free tier).

```
jobot config set llm.api_key.openai sk-proj-...
```

Cost model: GPT-4o-mini is \$0.150/1k input + \$0.600/1k output (~3x Gemini Flash). GPT-4o is \$2.50/1k input + \$10/1k output (~30x Gemini Flash). **Recommended default model:** `gpt-4o-mini` for most tasks; `gpt-4o` for resume tailoring if Anthropic is unavailable. **Use case in JoBot:** fallback in the default fallback chain after Gemini.

26.3 Anthropic Claude

Where to get the key: <https://console.anthropic.com/settings/keys>. Requires Anthropic account + billing.

```
jobot config set llm.api_key.anthropic sk-ant-api03-...
```

Cost model: Claude 3.5 Sonnet is \$3/1k input + \$15/1k output (comparable to GPT-4o). Claude 3 Haiku is \$0.25/1k input + \$1.25/1k output (cheap, fast, less capable). **Recommended default model:** `claude-3-5-sonnet-20241022` for resume tailoring (it produces the highest-quality tailored resumes in internal testing). **Use case in JoBot:** Tier-1 provider for high-stakes LLM tasks (resume tailoring, interview prep); fallback after OpenAI.

26.4 OpenAI-Compatible (OpenRouter, Groq, Together, Ollama, vLLM)

All five of these providers speak the OpenAI Chat Completions API protocol. JoBot uses a single `OpenAICompatProvider` class parameterized by `base_url` and `api_key`.

Provider	Base URL	Env var	Free tier?
OpenRouter	https://openrouter.ai/api/v1	OPENROUTER_API_KEY	Free models available (e.g., meta-llama/llama-3.1-8b-instruct:free)
Groq	https://api.groq.com/openai/v1	GROQ_API_KEY	Yes — 30 RPM, 14400 RPD on Llama 3.1 70B
Together	https://api.together.xyz/v1	TOGETHER_API_KEY	\$5 free credit for new users
Ollama (local)	http://localhost:11434/v1	OLLAMA_BASE_URL	Free (self-hosted)
vLLM (local)	Set via env	VLLM_BASE_URL	Free (self-hosted)

```
# OpenRouter:
jobot config set llm.api_key.openrouter sk-or-...
jobot config set llm.base_url.openrouter https://openrouter.ai/api/v1

# Groq:
jobot config set llm.api_key.groq gsk-...
jobot config set llm.base_url.groq https://api.groq.com/openai/v1

# Ollama (local – install from ollama.com first):
ollama pull llama3.1:8b
ollama serve # runs at http://localhost:11434
jobot config set llm.base_url.ollama http://localhost:11434/v1
# No api_key needed for Ollama – use 'dummy' as placeholder:
jobot config set llm.api_key.ollama dummy
# Set Ollama as default provider (fully offline mode):
jobot config set llm.default_provider ollama
```

26.5 Mistral

Where to get the key: <https://console.mistral.ai/api-keys>. Free tier: 1 RPS, 500K tokens/month. Recommended model: `mistral-large-latest` (\$2/1k input + \$6/1k output).

```
jobot config set llm.api_key.mistral <your-key>
```

26.6 Cohere

Where to get the key: <https://dashboard.cohere.com/api-keys>. Trial key: 1000 calls/month. Recommended model: `command-r-plus-08-2024` (\$2.50/1k input + \$10/1k output). Cohere is also useful for embeddings (text-embedding-3-small) — JoBot can use it instead of Gemini's embedding API for the vector-memory dedup layer.

```
jobot config set llm.api_key.cohere <your-key>
```

26.7 AWS Bedrock

Bedrock is the enterprise path — auth via IAM, no API key file. Requires an AWS account with Bedrock model access granted.

```
# Set up AWS credentials (standard AWS CLI flow):
aws configure
# → AWS Access Key ID: AKIA...
# → AWS Secret Access Key: ...
# → Default region: us-east-1

# JoBot reads from standard AWS_* env vars:
export AWS_ACCESS_KEY_ID=AKIA...
export AWS_SECRET_ACCESS_KEY=...
export AWS_REGION=us-east-1

# Grant Bedrock model access in AWS console:
# Bedrock → Model access → Manage model access →
# enable Anthropic Claude, Mistral, etc.

# Use Bedrock as default:
jobot config set llm.default_provider bedrock
# Default model on Bedrock:
jobot config set llm.default_model anthropic.claude-3-5-sonnet-20241022-v2:0
```

26.8 Google Vertex AI

Vertex AI is Google Cloud's enterprise LLM platform — same Gemini models but via GCP auth (IAM-based, JSON service account key).

```
# Create a service account in GCP:
# IAM → Service accounts → Create service account →
# name: jobot-sa
# roles: Vertex AI User
# → Create JSON key → download as jobot-sa-key.json

# Tell JoBot where the key is:
export GOOGLE_APPLICATION_CREDENTIALS=/path/to/jobot-sa-key.json
export VERTEX_PROJECT=my-gcp-project-id
export VERTEX_REGION=us-central1

# Use Vertex as default:
jobot config set llm.default_provider vertex
jobot config set llm.default_model gemini-1.5-pro-002
```

26.9 Switching Providers at Runtime

```
# Switch default provider:
jobot config set llm.default_provider anthropic
# Next LLM call will use Anthropic.

# Switch for a specific task only (per-profile YAML):
# In profiles/default.yaml:
llm:
  default_provider: gemini
  task_overrides:
    resume_tailoring:
      provider: anthropic # use Claude for resumes
      model: claude-3-5-sonnet-20241022
    question_answering:
      provider: gemini # cheap model for form Q&A
      model: gemini-2.0-flash

# Force a specific provider for a single CLI invocation:
jobot --provider anthropic apply <job-id>
```

Chapter 27 — Setup: Docker

Docker is the recommended deployment path for production. It isolates JoBot's Python env, browser binaries, and SQLite data from the host system. The compose stack has three services: `jobot` (the application), `jobot-browser` (a Playwright-in-Docker image with persistent browser profile), and `jobot-db` (optional — Postgres for users with >10k tracked jobs).

27.1 Dockerfile (Multi-Stage Build)

```
# Dockerfile
FROM python:3.11-slim AS builder
WORKDIR /build
COPY pyproject.toml .
COPY src/ ./src/
RUN pip install --no-cache-dir build && python -m build --wheel

FROM python:3.11-slim AS runtime
# System deps: poppler-utils (pdftotext), gnupg, fonts
RUN apt-get update && apt-get install -y --no-install-recommends \
    poppler-utils gnupg fonts-noto-core fonts-noto-cjk \
    && rm -rf /var/lib/apt/lists/*
# Tectonic (lightweight LaTeX)
RUN curl -fsSL https://drop-sh.fullyjustified.net | sh && \
    mv tectonic /usr/local/bin/
# Patchright browser
COPY --from=builder /build/dist/*.whl /tmp/
RUN pip install --no-cache-dir /tmp/*.whl patchright && \
    patchright install chromium && \
    patchright install-deps && \
    rm /tmp/*.whl
# Data dirs
RUN mkdir -p /data /browser-profile /evidence /cache
VOLUME ["/data", "/browser-profile", "/evidence", "/cache"]
ENV JOBOT_DATA_DIR=/data \
    JOBOT_BROWSER_PROFILE_DIR=/browser-profile \
    JOBOT_EVIDENCE_DIR=/evidence \
    JOBOT_CACHE_DIR=/cache
WORKDIR /app
ENTRYPOINT ["jobot"]
CMD ["--help"]
```

27.2 docker-compose.yml

```
# docker-compose.yml
version: '3.9'

services:
  jobot:
    build: .
    image: jobot:latest
    container_name: jobot
    restart: unless-stopped
    env_file: .env
    volumes:
      - jobot-data:/data
      - jobot-browser-profile:/browser-profile
      - jobot-evidence:/evidence
      - jobot-cache:/cache
      - ./profiles:/app/profiles:ro # user profiles (read-only)
    # Mount OS keyring via Docker secrets (see §27.4)
    secrets:
      - source: jobot_keyring
        target: /run/secrets/keyring
    healthcheck:
      test: ["CMD", "jobot", "doctor"]
      interval: 5m
      timeout: 30s
      retries: 3
      start_period: 30s

    # Optional: scheduled daily loop via cron-in-container
  jobot-scheduler:
    image: jobot:latest
    container_name: jobot-scheduler
    restart: unless-stopped
    env_file: .env
    depends_on:
      jobot:
        condition: service_healthy
    volumes:
      - jobot-data:/data:ro
      - jobot-browser-profile:/browser-profile
      - ./profiles:/app/profiles:ro
    entrypoint: ["/bin/sh", "-c"]
    command:
      - |
        echo '0 9 * * * jobot loop full-loop --profile default --max-apply 10' >
/etc/cron.d/jobot
        echo '0 8 * * 1 jobot loop digest --profile default' >> /etc/cron.d/jobot
        echo '0 0 * * * jobot policy reset-daily --profile default' >>
/etc/cron.d/jobot
        crontab /etc/cron.d/jobot
        crond -f

volumes:
  jobot-data:
  jobot-browser-profile:
  jobot-evidence:
  jobot-cache:

secrets:
  jobot_keyring:
```

```
file: ./secrets/keyring.enc # see §27.4
```

27.3 Bringing Up the Stack

```
# 1. Create .env from template
cp .env.example .env
# Edit .env: set JOBOT_PROFILE, JOBOT_DEFAULT_LLM_PROVIDER, etc.

# 2. Set API keys via the in-container keyring (one-time):
docker compose run --rm jobot config set llm.api_key.gemini AIzaSy...
docker compose run --rm jobot config set llm.api_key.openai sk-proj-...
# (Stored in container's keyring; persisted via the jobot-browser-profile volume)

# 3. Build + start the stack:
docker compose up -d --build
# Expected: 2 containers running (jobot + jobot-scheduler)

# 4. Verify:
docker compose exec jobot jobot doctor
# Expected: all checks pass

# 5. Run a manual scrape test:
docker compose exec jobot jobot scrape linkedin --keywords 'senior backend' --limit 10
# Expected: 10 postings persisted to SQLite at /data/jobot.db

# 6. View scheduler logs:
docker compose logs -f jobot-scheduler
```

27.4 Secrets Management in Docker

The container has no OS keyring by default (no Keychain / gnome-keyring running). JoBot uses a fallback keyring backend that reads from an encrypted file at `/run/secrets/keyring` (the Docker secret). This file is encrypted with a master key derived from the `JOBOT_DOCKER_MASTER_KEY` env var (set in `.env`, not committed). On first run, JoBot creates the encrypted file; subsequent runs decrypt it on startup.

```
# On host (one-time setup):
echo "$(openssl rand -hex 32)" > .docker-master-key
echo "JOBOT_DOCKER_MASTER_KEY=$(cat .docker-master-key)" >> .env
rm .docker-master-key
# .docker-master-key is gone – only the env var (in .env, .gitignore'd) knows it.

# First container run creates the encrypted keyring file:
docker compose run --rm jobot config set llm.api_key.gemini AIzaSy...
# → /run/secrets/keyring now exists with the encrypted API key.
# → Mounted via Docker secret + persistent volume (jobot-browser-profile).
```

27.5 Healthcheck + Restart Policy

The healthcheck runs `jobot doctor` every 5 minutes. If doctor fails (e.g., LLM provider down, SQLite locked), Docker marks the container unhealthy and restarts it (per `restart: unless-stopped`). The scheduler container depends on `condition: service_healthy` — it won't start cron jobs while the main container is unhealthy.

27.6 Backup Strategy

The `jobot-data` volume contains the SQLite database + Fernet-encrypted blobs. Back it up daily via a sidecar container or host cron:

```
# Host crontab – backup the data volume nightly:
0 2 * * * docker run --rm -v jobot-data:/data -v /backups:/backups \
    alpine tar czf /backups/jobot-data-$(date +%F).tar.gz /data
# Retention: keep last 30 days
0 3 * * * find /backups -name 'jobot-data-*.tar.gz' -mtime +30 -delete
```

Chapter 28 — Setup: CLI Reference

Complete CLI command tree. All commands are subcommands of `jobot`. Each subcommand has `--help` for full argument list. Common flags: `--profile <name>` selects which profile to use (default: `default`); `--json` outputs structured JSON instead of human-readable text (for piping to other tools); `--dry-run` simulates without side effects.

28.1 Initialization & Diagnostics

Command	Purpose	Example
<code>jobot init</code>	First-run setup wizard. Creates <code>~/.jobot/</code> , generates profile, prompts for API key.	<code>jobot init</code>
<code>jobot doctor</code>	Verify environment: Python version, deps, keyring, browser, LaTeX, LLM reachability.	<code>jobot doctor</code>
<code>jobot --version</code>	Print version + git commit hash.	<code>jobot --version</code>
<code>jobot --help</code>	List all subcommands.	<code>jobot --help</code>

28.2 Configuration & Secrets

Command	Purpose	Example
<code>jobot config get <key></code>	Get a config value. Masked if secret.	<code>jobot config get llm.api_key.gemini</code>
<code>jobot config set <key> <value></code>	Set a config value. Secrets go to OS keyring.	<code>jobot config set llm.api_key.gemini AIza...</code>
<code>jobot config show</code>	List all config. Secrets masked.	<code>jobot config show</code>
<code>jobot config unset <key></code>	Remove a config value.	<code>jobot config unset llm.api_key.openai</code>
<code>jobot config reload</code>	Force re-instantiation of all providers.	<code>jobot config reload</code>
<code>jobot secrets edit</code>	Open \$EDITOR on decrypted secrets.yaml; re-encrypts on save.	<code>jobot secrets edit</code>
<code>jobot policy reset-daily</code>	Reset daily caps (cost + apply count). Cron runs at midnight UTC.	<code>jobot policy reset-daily --profile default</code>

28.3 Scraping & Discovery

Command	Purpose	Example
jobot scrape <board>	Scrape a specific board. Boards: linkedin, indeed, greenhouse, lever, glassdoor, career-page.	jobot scrape linkedin --keywords 'senior backend' --location 'San Francisco' --limit 50
jobot scrape --all	Scrape all configured boards in parallel.	jobot scrape --all --keywords 'engineer' --limit 25
jobot scrape career-page	Scan 150+ configured employer career pages.	jobot scrape career-page
jobot discover ats <employer>	Detect what ATS an employer uses (Greenhouse/Lever/Workday/...). Lifted from career-ops.	jobot discover ats airbnb
jobot dedup	Run dedup on existing jobs (catches near-duplicates via vector similarity).	jobot dedup --threshold 0.92

28.4 Ranking & Applying

Command	Purpose	Example
jobot rank	Score and rank saved jobs by keyword match, salary, remote, visa sponsor.	jobot rank --top 20
jobot apply <job-id>	Apply to a specific job via the saga.	jobot apply abc123 --dry-run
jobot apply --batch N	Apply to top N jobs from ranked list.	jobot apply --batch 5 --yes
jobot apply --resume <saga-id>	Resume an interrupted apply saga from checkpoint.	jobot apply --resume saga-xyz789
jobot tracker list	List applications (kanban view via rich tables).	jobot tracker list --status applied
jobot tracker show <job-id>	Show application detail: status history, events, evidence, cost.	jobot tracker show abc123
jobot tracker move <job-id> <status>	Manually move an application to a new ASP state.	jobot tracker move abc123 interview

28.5 Resume & Cover Letter

Command	Purpose	Example
jobot resume tailor	Run drafter→reviewer loop on a JD, produce tailored PDF.	jobot resume tailor --jd-url https://... --template modern
jobot resume ats-check <pdf>	Run ATS parseability check on an existing PDF.	jobot resume ats-check my-resume.pdf
jobot resume templates list	List available LaTeX templates.	jobot resume templates list
jobot coverletter generate	Generate cover letter for a specific job.	jobot coverletter generate --job-id abc123 --tone classic

28.6 Interview Prep

Command	Purpose	Example
jobot interview mock <job-id>	Start a multi-turn mock interview session for a job.	jobot interview mock abc123 --type behavioral --num-questions 5
jobot interview star <question> <draft>	Coach a draft behavioral answer into STAR format.	jobot interview star 'Tell me about a conflict...' 'I had a disagreement with...'

28.7 Tracker & Analytics

Command	Purpose	Example
jobot tracker dashboard	Render dashboard (terminal by default).	jobot tracker dashboard
jobot tracker dashboard --html > dashboard.html	Export dashboard to standalone HTML file.	jobot tracker dashboard --html > dashboard.html
jobot digest	Generate + send weekly digest email.	jobot digest --period weekly
jobot career salary <role> <location>	Look up salary band.	jobot career salary 'senior backend' 'San Francisco'
jobot career skill-gap	Analyze skill gap from saved JDs vs profile.	jobot career skill-gap --lookback 30

28.8 Outreach

Command	Purpose	Example
jobot outreach search <company> <role>	Generate LinkedIn people-search URL for recruiters/hiring managers.	jobot outreach search Stripe recruiter
jobot outreach dm <company> <preset>	Generate a DM template.	jobot outreach dm Stripe faang_senior_backend
jobot outreach batch <preset> --count N	Generate N DMs for top recruiters at target companies in the preset.	jobot outreach batch faang_senior_backend --count 5

28.9 Scheduler & Loops

Command	Purpose	Example
jobot loop scan-only	Scrape + dedup + email summary. No apply.	jobot loop scan-only --profile default
jobot loop apply-only --max-apply N	Apply to top N jobs from saved pool.	jobot loop apply-only --max-apply 5
jobot loop digest --period weekly	Send weekly/monthly digest email.	jobot loop digest --period weekly
jobot loop full-loop --max-apply N	Full daily loop: scrape + apply + digest.	jobot loop full-loop --max-apply 10
jobot loop interview-prep <job-id>	Run mock interview for an upcoming scheduled interview.	jobot loop interview-prep abc123
jobot loop outreach-batch <preset> --count N	Send N cold DMs to recruiters.	jobot loop outreach-batch faang_senior_backend --count 5

28.10 Plugin Management

Command	Purpose	Example
jobot plugin install <source>	Install plugin from pip spec or git URL.	jobot plugin install git+https://github.com/foo/jobot-plugin-bar.git
jobot plugin list	List installed plugins.	jobot plugin list
jobot plugin audit	Verify all installed plugins match their stored audit hash.	jobot plugin audit
jobot plugin uninstall <name>	Uninstall a plugin.	jobot plugin uninstall bar

28.11 Maintenance & Backup

Command	Purpose	Example
jobot backup	Backup SQLite + blobs to ~/.jobot/backup/<date>/.	jobot backup
jobot restore <date>	Restore from a backup.	jobot restore 2026-08-12
jobot migrate	Run pending schema migrations.	jobot migrate
jobot migrate --rollback N	Roll back N migrations.	jobot migrate --rollback 1
jobot browser restore <board>	Restore browser profile from backup (after corruption).	jobot browser restore linkedin
jobot trace export <trace-id>	Export a trace (all spans) as OpenTelemetry JSON.	jobot trace export trace-abc123
jobot quarantine list	List quarantined (failed) tasks.	jobot quarantine list
jobot quarantine replay <id>	Replay a quarantined task.	jobot quarantine replay task-abc123
jobot quarantine discard <id>	Discard a quarantined task.	jobot quarantine discard task-abc123

Chapter 29 — Setup: Troubleshooting & FAQ

Top 20 failure modes drawn from real JoBot deployment experience and the agents.md reliability-math principles. Each entry: symptom → root cause → fix command. If a fix doesn't work, the next step is `jobot doctor --verbose` and inspecting the trace spans via `jobot trace export`.

Symptom. Browser launches but LinkedIn login fails

Root cause. Patchright's persistent context lost the login session (browser-profile volume corrupted or first-time setup not done).

Fix. Open a visible browser, log in manually once: `jobot browser login --board linkedin --headed`. After login, future runs reuse the session.

Symptom. LLM API key works in curl but not in JoBot

Root cause. Key is in `.env` file (Tier 1) but JoBot reads from OS keyring (Tier 3).

Fix. Set via the keyring command: `jobot config set llm.api_key.gemini AIza...` Remove from `.env`. Verify: `jobot config get llm.api_key.gemini` returns the key.

Symptom. SQLite database is locked

Root cause. Concurrent writer contention (rare on WAL mode but possible if a long-running task holds a write transaction open).

Fix. Find the writer: `fuser ~/.jobot/data/jobot.db`. If a stale process holds the lock, kill it: `kill -9 <pid>`. If corruption suspected: `sqlite3 ~/.jobot/data/jobot.db 'PRAGMA integrity_check;'`.

Symptom. LaTeX resume generation fails on missing font

Root cause. TeX Live minimal install missing a required package (e.g., fontawesome).

Fix. Install full TeX Live: `brew install --cask mactex` (macOS) or `apt install texlive-full` (Linux). Or use Tectonic (auto-downloads packages on demand): `brew install tectonic`.

Symptom. Captcha solver returns garbage; apply submits with wrong answer

Root cause. 2captcha API key invalid OR captcha type unsupported (e.g., reCAPTCHA v3).

Fix. Disable auto-captcha-solving and fall back to human: `jobot config set captcha.solver human`. Or use the LLM-assisted solver for text/math captchas: `jobot config set captcha.solver llm`.

Symptom. Apply saga left job in half-applied state

Root cause. Saga crashed between 'fill_form' and 'submit' (e.g., Ctrl+C). Compensating actions didn't run.

Fix. Resume from checkpoint: `jobot apply --resume <saga-id>`. If checkpoint is corrupt, manually inspect: `sqlite3 ~/.jobot/data/jobot.db 'SELECT * FROM applications`

WHERE id="abc123";'. Mark as FAILED if unrecoverable: `jobot tracker move abc123 failed`.

Symptom. OS keyring not available on WSL2

Root cause. gnome-keyring daemon not running inside WSL2.

Fix. Start it: `eval $(gnome-keyring-daemon --start); export SSH_AUTH_SOCK`. Add to `~/.bashrc`. If still failing, use the encrypted-file fallback: `export JOBOT_KEYRING_BACKEND=file; export JOBOT_KEYRING_FILE=~/.jobot/keyring.enc`.

Symptom. JobSpy rate-limited by LinkedIn (HTTP 429)

Root cause. LinkedIn detected the scraper pattern.

Fix. Enable proxy rotation: `jobot config set scraper.jobspy.proxy_list ~/proxies.txt`. Reduce rate: `jobot config set scraper.jobspy.delay_s 5`. Or skip LinkedIn and use Greenhouse/Lever direct-API adapters: `jobot scrape greenhouse --companies airbnb,stripe,...`

Symptom. PII masker is over-aggressive — masks non-PII strings

Root cause. LLM-assisted masker (when enabled) flags common words as PII.

Fix. Reduce strictness: `jobot config set pii.masker regex-only`. Re-enable LLM-assist only on traces containing free-text user input.

Symptom. Daily cap reached unexpectedly

Root cause. Yesterday's auto-apply batch ran past midnight UTC (when cap resets) — today's cap was already half-consumed by yesterday's run.

Fix. Cap is enforced correctly. The fix is to either (a) run the daily loop earlier (8am UTC instead of midnight), or (b) raise the daily cap in profile: `jobot config set policy.daily_apply_cap 100`.

Symptom. Resume LaTeX produces PDF but ATS check fails

Root cause. PDF uses two-column template that confuses pdftotext extraction order.

Fix. Switch to single-column template: `jobot resume tailor --template default`. The modern two-column template is for human review; `default` is ATS-safe.

Symptom. Cover letter sounds like every other LLM-generated cover letter

Root cause. Default temperature 0.7 + generic system prompt produces bland output.

Fix. Lower temperature to 0.4 for more constrained variation, OR add a custom tone fragment: `jobot coverletter generate --tone technical --extra-prompt 'Mention the candidate's experience with distributed consensus (Paxos).'`

Symptom. Browser profile corrupt after crash

Root cause. Patchright crashed mid-write to user-data-dir, leaving partial state.

Fix. Restore from backup: `jobot browser restore linkedin`. If no backup: `rm -rf ~/.jobot/browser-profiles/linkedin/` + re-login via `jobot browser login --board linkedin --headed`.

Symptom. Plug-in install fails with 'audit hash mismatch'

Root cause. Plugin source changed between install attempt and audit.

Fix. Re-pull from source: `jobot plugin install git+https://... --trust` (bypass audit — only for trusted plugins). For untrusted: contact plugin author to publish a stable versioned release.

Symptom. Digest email never arrives

Root cause. SMTP credentials not set in OS keyring, OR daily cost cap hit before digest send.

Fix. Set SMTP: `jobot config set smtp.user you@gmail.com; jobot config set smtp.password your-app-password`. Use a Gmail App Password (not your main password). Verify: `jobot digest --force-send`.

Symptom. Apply to Greenhouse fails with '400 Bad Request'

Root cause. Form field validation failed — a custom question was answered with wrong type (e.g., numeric answer to a text field).

Fix. Inspect the form spec first: `jobot apply abc123 --dry-run --verbose`. See which question triggered the error. Adjust the answer manually in profile or override: `jobot apply abc123 --override-answer 'years_of_experience' 8`.

Symptom. Multi-turn interview mock session ends abruptly

Root cause. LLM hit max_tokens limit mid-question.

Fix. Raise limit: `jobot config set llm.max_tokens.interview 1000`. Or split into multiple shorter sessions.

Symptom. Docker container keeps restarting

Root cause. Healthcheck fails because LLM provider unreachable (e.g., no API key set).

Fix. Set at least one provider key: `docker compose run --rm jobot config set llm.api_key.gemini AIza...` Restart: `docker compose restart jobot`. Check logs: `docker compose logs jobot | tail -50`.

Symptom. JobSpy returns 0 results for a search that obviously has results

Root cause. Site parameter case mismatch (JobSpy expects lowercase site names like 'linkedin' not 'LinkedIn').

Fix. Use lowercase site names: `jobot scrape linkedin --keywords 'engineer'`. Check JobPy's supported site list: `python -c 'from jobspy import scrape_jobs; print(scrape_jobs.__doc__)'`.

Symptom. JobBot is slow on first scrape

Root cause. First-time scraping triggers a vector-memory embedding call for every job (Gemini embedding API cold start).

Fix. This is expected. Subsequent scrapes reuse the embedding cache. Speed up by pre-warming the cache:
`jobot dedup --warm-cache.`

Appendix A — Full Source Repository Star Inventory

Complete inventory of all 33 surveyed repositories, sorted by star count descending. All star counts are API-verified as of 2026-08-13 (UTC 04:35–05:00). Categories: NA = Named (in original prompt list); DC = Discovered via topic/keyword search. The complete research artifact (with feature-overlap matrix, architectural patterns, and adoption recommendations) is at `/home/z/my-project/upload/repo_research.md`.

#	Repo	Stars	Lang	License	Type	Tier
1	santifer/career-ops	63,659	JavaScript	MIT	NA	1
2	MadsLorentzen/ai-job-search	31,411	TypeScript	MIT	NA	1
3	feder-cr/Jobs_Applier_AI_Agent_AIHawk	30,159	Python	AGPL-3.0	NA	1
4	speedyapply/JobSpy	4,072	Python	MIT	DC	2
5	DaKheera47/job-ops	3,840	TypeScript	NOASSERTION	DC	2
6	GodsScion/Auto_job_applier_linkedIn	2,688	Python	MIT	DC	2
7	PaulMcInnis/JobFunnel (ARCHIVED)	2,178	Python	MIT	DC	2
8	can4hou6joeng4/boss-agent-cli	1,558	Python	MIT	DC	2
9	eatmoreduck/boss-zhipin-scraper	1,094	Python	MIT	DC	2
10	Gsync/jobsync	854	TypeScript	MIT	DC	3
11	wodsuz/EasyApplyJobsBot	804	Python	NOASSERTION	DC	3
12	andrew-shwetzer/career-ops-plugin-...	463	HTML	MIT	DC	3
13	feder-cr/resume_render_from_jd	412	Python	MIT	DC	3
14	rainmanjam/jobspy-api	376	Python	(check)	DC	3
15	agentenatalie/get-job.skill	348	Python	NOASSERTION	DC	3
16	strelov1/freehire	337	Go	MIT	DC	3
17	jennifer88huang/interview-skills	272	JavaScript	None	DC	3
18	slothsheepking/jobclaw	208	?	—	DC	3
19	feder-cr/lib_resume_builder_AIHawk	193	Python	MIT	DC	3
20	imon333/Job-apply-AI-agent	177	Python	None	DC	3
21	surapuramakhil-org/Job_search_agent (ARCHIVED)	171	Python	AGPL-3.0	NA	3
22	colophon-group/jobseek	167	Python	?	DC	3
23	replyre/job-hunter	150	Python	None	NA	3
24	beatwad/LinkedIn-AI-Job-Applier-Ultimate	147	Python	None	DC	3
25	Liam-Frost/AutoApply	111	Python	?	DC	4
26	tmwclaxton/autoapplycv	73	JavaScript	NOASSERTION	DC	4
27	Rayyan9477/AutoApply-AI-Agentic-...	65	Python	?	DC	4
28	Azoo92i/AutoApplyMax	58	JavaScript	?	DC	4

#	Repo	Stars	Lang	License	Type	Tier
29	krishnavalliappan/JobScout (STALE)	47	Python	None	DC	4
30	BjornMelin/ai-job-scraper	44	Python	MIT	DC	4
31	kbhujbal/go-get-jobs	40	Go	None	DC	4
32	LuisMiguelFurlanettoSousa/auto-apply-bot	34	TypeScript	MIT	DC	4
33	cboyd0319/JobSentinel	20	TypeScript	MIT	DC	4
34	algsoch/job_agentic	3	Python	None	NA	4
35	PunithVT/career-ops	2	JavaScript	?	DC	4
—	aryansinghnagar/JoBot (TARGET)	0	Python	AGPL-3.0	Target	—

The 'Tier' column reflects JoBot's internal prioritization (see Chapter 4). Tier 1 = 10k+ stars or 5k+ active; Tier 2 = 1k–10k; Tier 3 = 100–1k; Tier 4 = <100 / archived / stale / no license. Note that `algsoch/job_agentic` (3 stars) is in Tier 4 because it has no `LICENSE` file and its functionality is a subset of JoBot's existing plans.

A.1 Repos That Could Not Be Located

Several repos mentioned in the original scoping conversation could not be found on GitHub as of 2026-08-13. They are recorded here for completeness.

Prompted name	URL tried	Result
feder-cr/AIHawk (the original)	https://github.com/feder-cr/AIHawk	404 — AIHawk brand lives only in <code>Jobs_Applier_AI_Agent_AIHawk</code> + sibling resume libs
feder-cr/aihawk (lowercase)	https://github.com/feder-cr/aihawk	404
surapuramakhil-org/AI-job-apply-agent	https://github.com/surapuramakhil-org/AI-job-apply-agent	404 — org's only job-search repo is <code>Job_search_agent</code> (archived)
Nexloop/JobScan-AI	https://github.com/Nexloop/JobScan-AI	404 — no search hits
SurferMatt/job-application-automator	https://github.com/SurferMatt/job-application-automator	404
aiagentjobseeker (org)	https://github.com/aiagentjobseeker/aiagentjobseeker	404
AutoJobr	searched GitHub	No results — no GitHub repo of that name with meaningful stars
JobLLM	searched GitHub	Only 3 micro-repos (≤ 1 star each) — none material
JobPilot-AI	searched GitHub	No results (note: a Python package 'jobpilot' exists on PyPI as an unrelated Australian job-board scraper)

Prompted name	URL tried	Result
aryansinghnarel/JoBot (typo)	https://github.com/aryansinghnarel/JoBot	404 — the canonical owner is 'aryansinghnagar' (with an 'a')

Appendix B — Glossary

40+ terms used throughout the document, with concise definitions. Terms are listed alphabetically. Cross-references to relevant chapters are in (parentheses).

Term	Definition
Adapter	A class implementing BoardAdapter interface, providing scrape/apply/health_check methods for a specific job board (LinkedIn, Greenhouse, etc.). (Ch 5, 10)
AGPL-3.0	Affero General Public License v3.0. JoBot's license. Network-use triggers source-disclosure obligation. Compatible with MIT/Apache/BSD/AGPL/GPL code; incompatible with no-license code.
ASP	Application State Protocol. JoBot's 12-phase state machine for tracking application lifecycle: DISCOVERED → SAVED → APPLYING → APPLIED → SCREENING → INTERVIEW → OFFER → ACCEPTED/REJECTED/WITHDRAWN/GHOSTED/ARCHIVED. (Ch 17)
ATS	Applicant Tracking System. The software employers use to filter resumes. JoBot's ATS check verifies its PDFs are machine-parseable. (Ch 13)
Bedrock	AWS's enterprise LLM platform. Auth via IAM, not API keys. (Ch 26)
BrowserPool	JoBot's session-pooling layer for Patchright. Reuses logged-in contexts across CLI invocations. (Ch 7)
Cap	Policy-engine limit (daily/weekly apply count or cost). Enforced before side effects. (Ch 11)
CDP	Chrome DevTools Protocol. Low-level control of Chrome via WebSocket. Used by JoBot's alternative stealth mode (eatmoreduck pattern). (Ch 7)
Circuit Breaker	Per-adapter pattern that opens after N consecutive failures, preventing cascading damage. Stays open for a cooldown period before half-opening to test recovery. (Ch 5, 10)
Compensating Action	A reverse operation declared for each saga step, executed if a later step fails. Example: 'withdraw application' for 'submit application'. (Ch 9, 15)
Cover Letter Generator	Module producing JD-specific cover letters with tone presets + length caps. (Ch 14)
Drafter-Reviewer Loop	Two-agent LLM pattern: drafter generates, reviewer critiques with A-F grade, drafter iterates until threshold. Lifted from MadsLorentzen. (Ch 13)
Durable Wait	A pause in execution that preserves exact run state, resumable after approval/webhook/time. (Ch 9)
Easy Apply	LinkedIn's one-click application feature. JoBot automates this via Patchright. (Ch 15)
Evidence Capture	Screenshot + DOM snapshot before/after risky browser actions. Stored at ~/.jobot/evidence/. (Ch 7)
Fernet	Symmetric authenticated encryption from cryptography lib. JoBot encrypts blob store with Fernet, master key in OS keyring. (Ch 8)
Greenhouse	ATS used by Airbnb, Stripe, Pinterest, etc. Has public JSON API. (Ch 10, 12, 15)
Idempotency Key	Unique hash attached to every side effect, enabling safe retries. If the same key is seen twice, second execution is a no-op. (Ch 5, 9)
JobSpy	speedyapply/JobPy — MIT-licensed Python scraper library supporting 6 boards. JoBot vendors it as a pip dependency. (Ch 12)
LaTeX	Typesetting system. JoBot uses lualatex/xelatex to render resumes to PDF. (Ch 13)
Lever	ATS used by Notion, Figma, Plaid, etc. Has public JSON API. (Ch 10, 12, 15)
ModelRouter	JoBot's provider-neutral LLM abstraction. Strategy pattern; 6 provider classes. Cost-aware. Fallback chain. (Ch 6)
NOASSERTION	GitHub's status when a LICENSE file exists but isn't recognized as a standard license. Legally risky for vendoring. (Ch 4, 22)

Term	Definition
OS Keyring	OS-native credential store: macOS Keychain, Linux Secret Service (gnome-keyring), Windows Credential Manager. (Ch 8, 25)
Patchright	Stealth fork of Playwright. Patches navigator.webdriver and other detection vectors. (Ch 7)
PII	Personally Identifiable Information. JoBot's masker strips PII from trace spans before persistence. (Ch 11)
Plugin	User-installable capability module distributed as a separate Python package. Audit-verified. (Ch 10)
Policy Engine	Stateful gate that enforces caps and approval checkpoints before side effects. (Ch 11)
Preview-Before-Commit	Capture screenshot of final form state before submit click. Human-in-the-loop checkpoint in interactive mode. (Ch 7)
Quarantine	Move repeatedly-failing tasks to a dead-letter queue for explicit replay or discard. Prevents retry storms. (Ch 9)
Saga	Multi-step transaction with compensating actions. JoBot's apply is a 13-step saga. (Ch 9, 15)
SCRUM / STAR	Situation-Task-Action-Result. Format for structuring behavioral interview answers. (Ch 16)
Selector Healing	When a primary CSS/XPath selector fails, the named action tries alternatives in order. Logs failure + emits alert. (Ch 7)
Span	OpenTelemetry-style record of a single operation (LLM call, browser action, etc.). Aggregated into traces. (Ch 9)
SQLite WAL	Write-Ahead Logging mode. Allows concurrent readers + single writer. JoBot's default control-plane storage. (Ch 8)
Strategy Pattern	OO pattern: define a family of algorithms, encapsulate each, make interchangeable. JoBot uses it for LLMPProvider. (Ch 5, 6)
Task Graph	DAG of typed TaskNodes with explicit dependencies. JoBot's orchestrator executes it with bounded parallelism. (Ch 9)
Tier (1/2/3/4)	JoBot's internal repo prioritization: T1 = 10k+ stars, T2 = 1k–10k, T3 = 100–1k, T4 = <100 or archived. (Ch 3, 4)
Trace	Collection of spans for a single user-facing operation (e.g., one apply). (Ch 9)
Vector Memory	JoBot's embedding store for semantic dedup of job postings. (Ch 8)
Vertex AI	Google Cloud's enterprise LLM platform. Auth via GCP service account JSON. (Ch 26)
Visa Sponsor	Whether an employer sponsors work visas. DaKheera47/job-ops uniquely filters for this. (Ch 18)
WAL	See SQLite WAL.
Workday	Enterprise ATS used by large employers. No public API; scraping required. (Ch 12)